

Lecture Embedded Computing

Prof. Dr. Markus Pfeil
with kind permission of Prof. Dr.-Ing. Franz Brümmer

WS 2020/21

Contents

1	Introduction	3
1.1	Historical Background	4
1.2	Typical Applications	4
1.3	Charateristics of Embedded Systems	6
2	Modeling	10
2.1	General Requirements	10
2.2	State Space Oriented Models	14
2.3	Activity Oriented Models	21
3	Cyber-Physical Systems	25
3.1	Overview, Delimitation of the Topic	25
3.2	OpenModelica	28
4	Processors	34
4.1	Processors for Embedded Systems	34
4.2	Overview about ARM Processors	39
4.3	Functions of the ARM-Cortex-A8-Processor	39
4.4	Overview about the DSP	45
5	Programming	50
5.1	Programming in Assembler	50
5.2	Programming in C	55
5.3	Python for Embedded Systems	57
5.4	Node.JS in embedded Systems	82
5.5	A systematic approach to debugging (especially Linux)	90
5.6	Debugging using JTAG	109
6	OMAP - Application Prozessor	116
6.1	General Aspects	116
6.2	Digital Ports	116
6.3	Serial Ports	118
6.4	Timers	123
7	Operating Systems	125
7.1	Operating Systems for Embedded Systems	125
7.2	Functions of RTOS	128
7.3	Functions of Embedded Linux	171
7.4	Cross Compilation	176
7.5	User Programming in uCLinux	178

Chapter 1

Introduction

The lecture on Embedded Computing focuses on different aspects in computer science. Starting with a short introduction on the development of computers, we will try to get an idea about the special requirements of embedded systems. In modern times, Embedded Systems finds application in almost every field. They are used not only in cars, washing machines and toasters but also in every mobile phone you'll find one microprocessor/microcontroller. It is interesting to note that the number of microcontrollers in use is much more than the number of Personal Computers. If you are looking on an application which includes a microcontroller you will have an embedded system.

Embedded systems on the contrary to a standard PC don't have graphical user interfaces (GUI), hard discs, keyboards etc. A typical example is an embedded system which controls the breaking mechanism in your car. This controller doesn't ask you in a command line (or with a click on an icon) what should be done. This system simply controls the brakes and hopefully stops your car in a dangerous situation. To fulfil such a typical requirement embedded systems have to deal with different aspects and have different requirements than standard PC. In the chapter "Modelling" we will try to find out what are the special conditions which embedded systems should fulfil.

After a general view about the requirements for embedded systems we shall shift our focus to processors. In principle, an embedded system can be realized using several different types of processors. This decision is based on factors such as application requirements, costs, computational needs etc. A standard processor of choice for an embedded system is a microcontroller. We shall focus on 'Coldfire', a 32-bit microcontroller from Motorola (or Freescale). This controller allows us to build simple stand alone applications or applications with the support of operating systems (real-time operating system).

Real-time operating systems are an important part in modern embedded systems. PC based computing is not very time critical e.g. If you are running a word-processor (like MS Word) a time delay of a few seconds in opening a new document is not critical. However, imagine if the same delay occurs in deploying of Air-bags during a car crash. Hence, embedded systems have special needs and the operating system of choice for such applications are the Real-time operating systems. They offer us a lot of features which makes it easier to build more complex applications. The drawback is that real-time operating systems have bigger requirements on the onboard memory and the support of special functions of the microcontroller like a memory management unit and so on. The last aspect of this lecture is, to give an introduction about the opportunities to store an algorithm on a chip directly. This so called "solution on chip" works with a FPGA-Chip (field programmable gate array) and offers a solution with the fast execution times.

1.1 Historical Background

First programmable computers

- Development of computers
 - started in 1940 (1941 Zuse Z3, Aiken Mark I)
 - classification in generations

Generation	Period	Technology	Characteristics
1	1950 - 1959	vacuum tube	mainframe
2	1960 - 1968	transistor	mainframe, process control
3	1969 - 1977	Integrated circuits	minicomputer
4	1978 -	LSI, VLSI	Workstation, PC
5	1993 -	LSI, VLSI	parallel processing

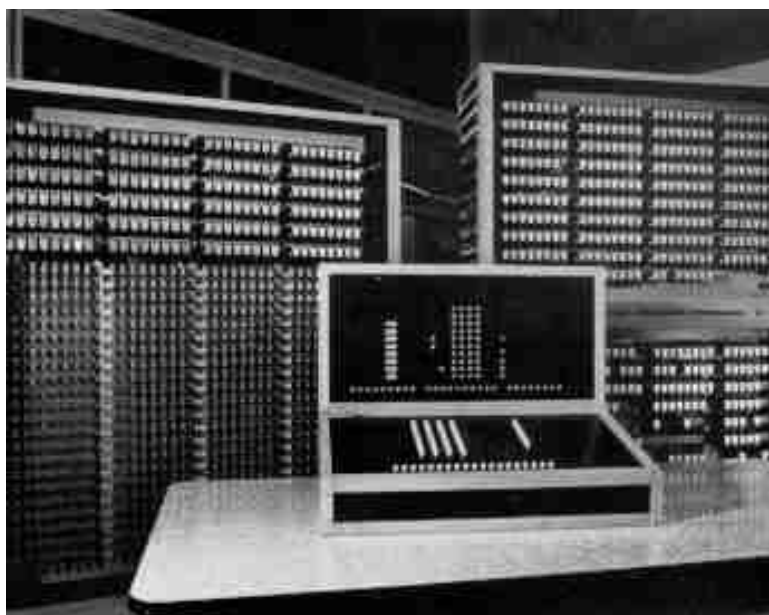


Figure 1.1.1: First programmable computer Zuse Z3

1.2 Typical Applications

Microcontrollers in Embedded systems

- no Graphical User Interface (GUI)
- no hard discs
- use of special real-time operating systems instead of complex standard operating system (like Windows or full feature-Linux)

Features of microcontrollers

- On-chip memory,
- I/O-devices (analogue, digital)
- Timer/Counter-channels

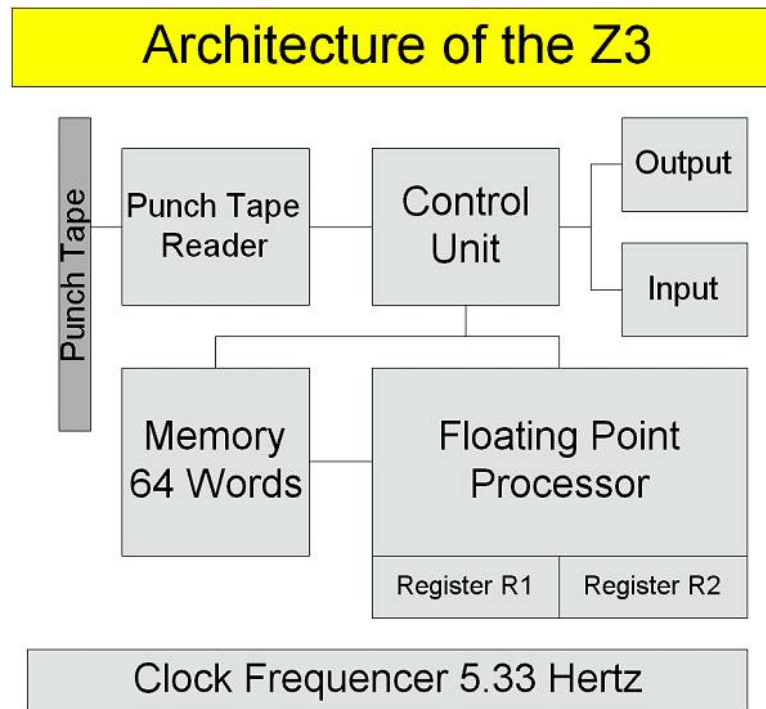


Figure 1.1.2: Blockdiagram of Zuse Z3



Figure 1.1.3: Computer Mark I

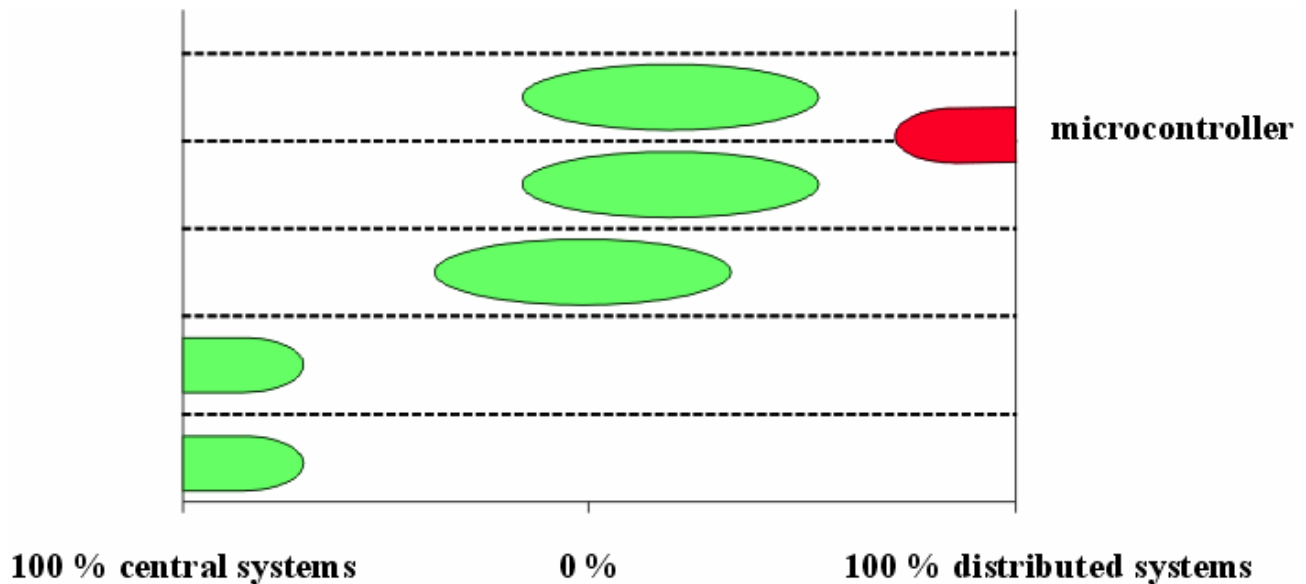


Figure 1.1.4: Central vs. distributed applications

- One-chip-solutions

Differences in the programming of microcontrollers

- Cross development (using PC, workstation)
- Function oriented programming language (most 'C')
 - application without an operating system
 - application with parts of an operating systems (e.g. ECOS)
 - applications with a special real-time operating system

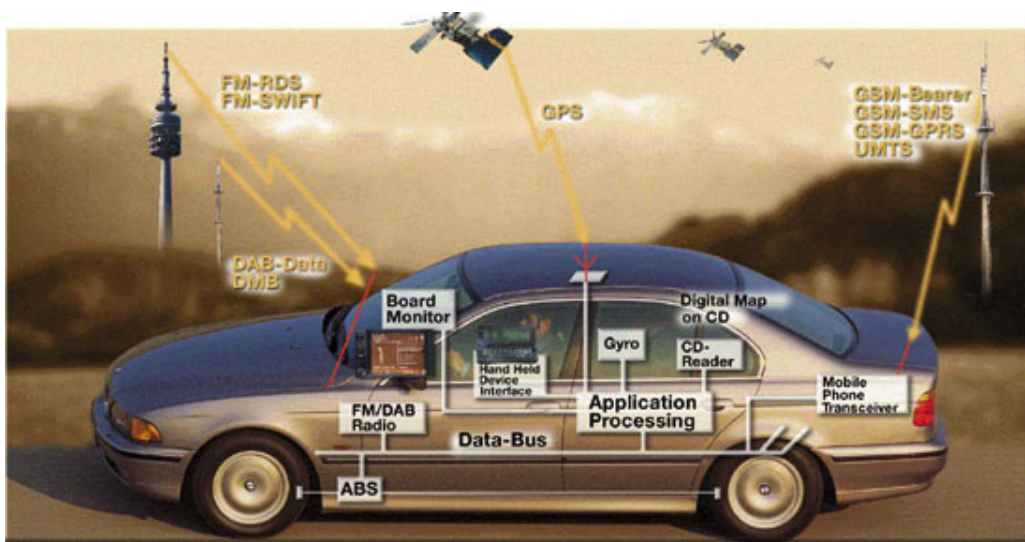


Figure 1.2.1: Embedded systems in modern car

1.3 Characteristics of Embedded Systems

Dependability of embedded systems



Figure 1.2.2: Microcontroller in a standard size

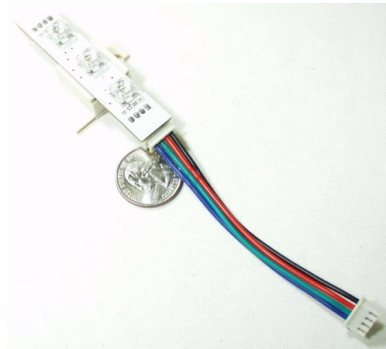


Figure 1.2.3: Small size microcontroller

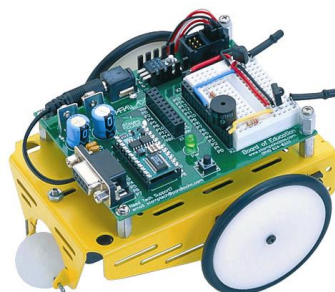


Figure 1.2.4: A last example for embedded systems

- Reliability
 - probability of system working correctly provided that it was working at $t=0$
 - continuous execution for long times (sometimes more than years)
 - system could recover by themselves (implemented with watch dog-functions)
- Maintainability
 - indicates repair friendliness
 - could be expressed as the probability of system working correctly d time units after error occurred.
- Availability
 - probability of system working at time t
 - the availability expresses the probability that the system at the actual time is available
- Safety
 - probability that the system works correct or that no harms will be caused
 - indicates that a system could fail without dangerous effects

Conclusion

Even perfectly designed systems can fail if the assumptions about the workload and possible errors turn out to be wrong.

Making the system dependable must not be an after-thought, it must be considered from the very beginning.

Efficiency of embedded systems

- Energy
 - low power consumption by microcontrollers
 - different operating modes (sleep-mode)
- Code-size
 - small external memories (SRAM or EPROM)
 - no external hard disc
 - in some cases one chip solutions
- Run-time
 - have to fulfil real-time constraints
 - in principal, 'a right answers that arrives too late, is wrong'
- Size and Weight
 - especially in industrial applications there exists strong requirements
- Cost
 - depends on the number of installations and the cost of device (hardware)
 - software cost mostly dedicated to one embedded system project (no general purpose software available)

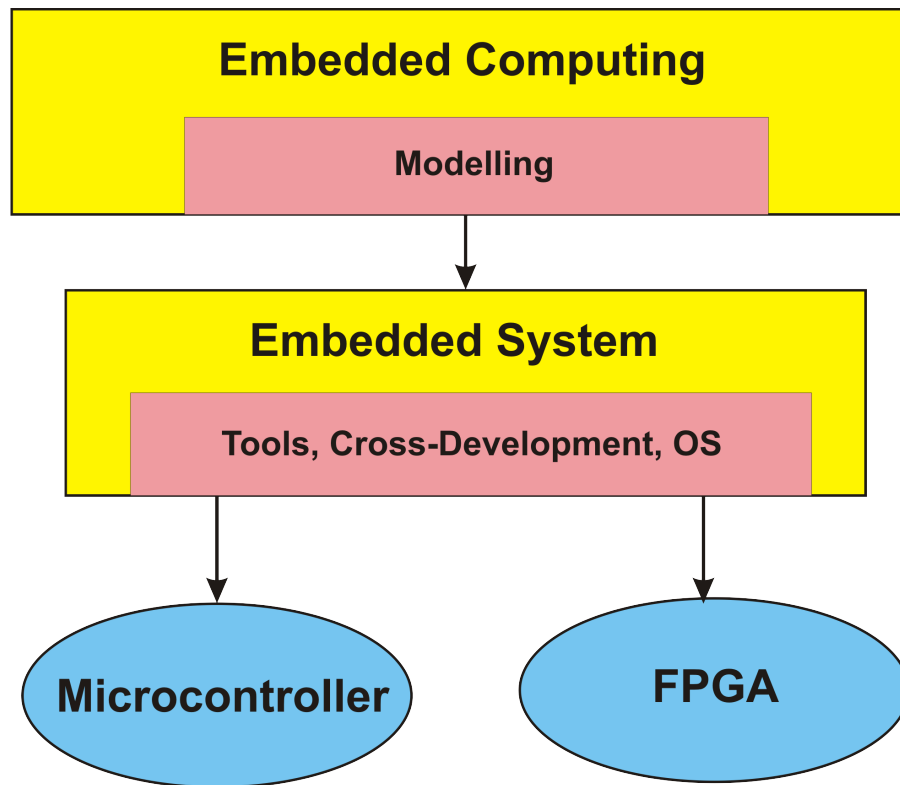


Figure 1.3.1: Blockdiagram Embedded Computing vs. Embedded System

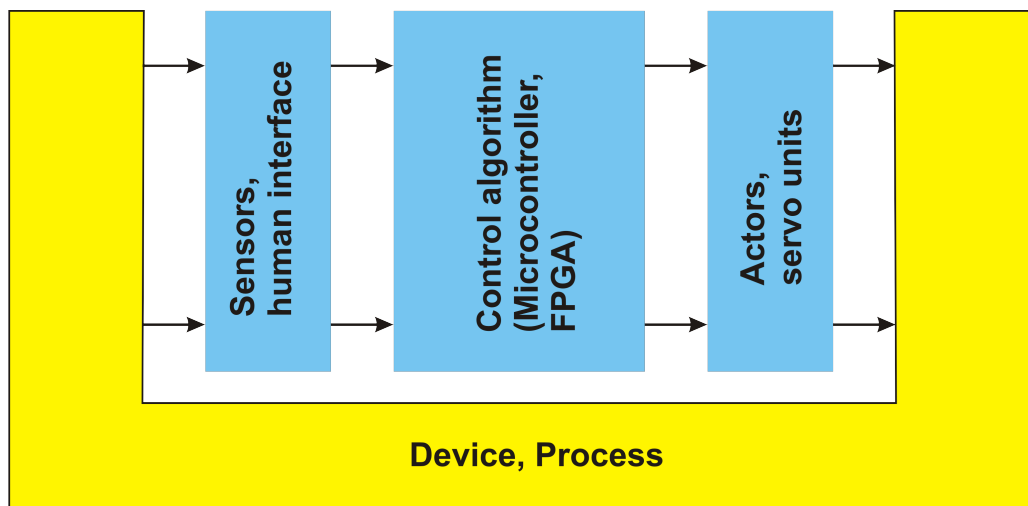


Figure 1.3.2: Blockdiagram Embedded System

Chapter 2

Modeling

2.1 General Requirements

The most embedded systems start with integration of a small microcontroller which performs some functions which are implemented in hardware. To emulate this function only a single program or several small-size programs are necessary. There is no need to build a model and to simulate the behaviour of the emulated functions. Also the documentation for such projects is small. The development starts with the programming and stops with a successful test. Then you could forget the project and start with the next.

But with changing times embedded systems have become more complex. So the development process has to be more structured. One of the major steps involved is the documentation. Companies spend a lot of money for documentation of embedded systems. A tool which helps the documentation process is hence a basic requirement. Model based development tools help us achieve this goal.

The next step is to overcome the special constraints of an embedded system. Real-time aspects, dead-lock situations, failure safe behaviour are requirements of any typical embedded system. A model should help us achieve these goals. This means that not only the correct result of a designed algorithm have to be tested, the results have to be in compliance of the requirements of an embedded system.

- Classification of systems from the point of programming

Definition 2.1 An **embedded system** is a special-purpose computer system designed to perform a dedicated function. Unlike a general-purpose computer, such as a personal computer, an embedded system performs one or a few pre-defined tasks, usually with very specific requirements, and often includes task-specific hardware and mechanical parts not usually found in a general-purpose computer. [Wikipedia]

Definition 2.2 “A **reactive system** is one which is in continual interaction with its environment and executes at a pace determined by that environment” [Bergé, 1995]

- interaction with the system controlled by sensors or from the environment
- specification of minimal and maximal response time
- system could contain parallel and distributed parts (subsystems)

Definition 2.3 A **real-time system** has to give a response as fast as required. So the system must respond to a signal, event or request fast enough to satisfy some requirement. Realtime often refers to process control and embedded systems

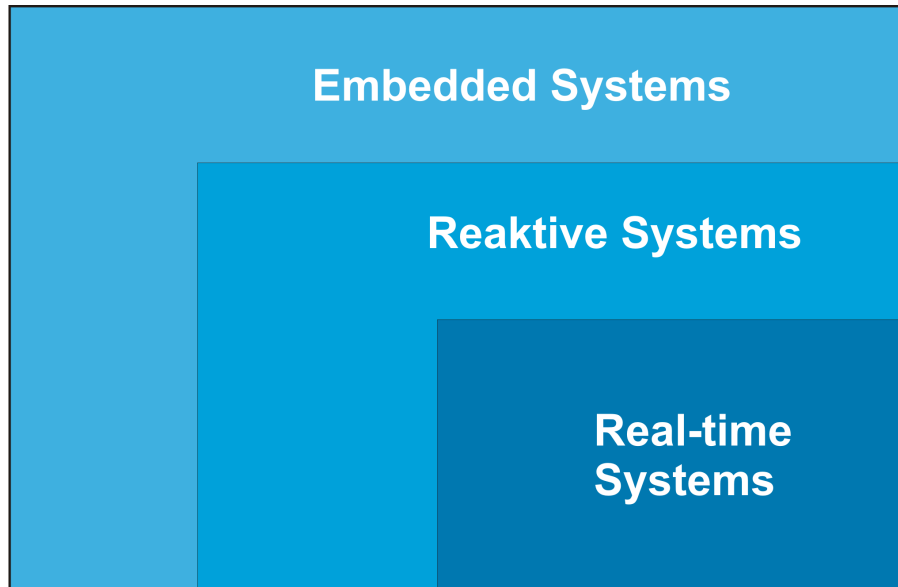


Figure 2.1.1: Classification of embedded systems and real-time-systems

- a real-time operating system (RTOS) facilitates the creation of a real-time system
- but real-time systems could operate without an RTOS
- if a real-time system responds deterministically → hard real-time system
- if a real-timesystem guarantees a deadline for responds → soft real-time system

Requirements

- concurrency → real-time systems are mostly concurrent
- synchronization and communication → modeling the communication between distributed systems
- portability and flexibility
- programming elements → arithmetic operations, loops, and function calls should be available
- non-standard I/O devices → switches, analogue and digital I/O
- non-functional properties → e.g. fault-tolerance, disposability, EMC-properties, user friendliness, extendibility, expected life time, power consumption...
- Co-Design aspects → modeling hard- and software functions

but

- models should be readable
- models should have a hierarchical structure → “humans not capable to understand systems containing more than ~ 5 objects”

A model must be as simple as possible, but not simpler. [A. Einstein]

- a model could change to different level in the Y-Chart
- Y-Chart are focused on the hardware-design

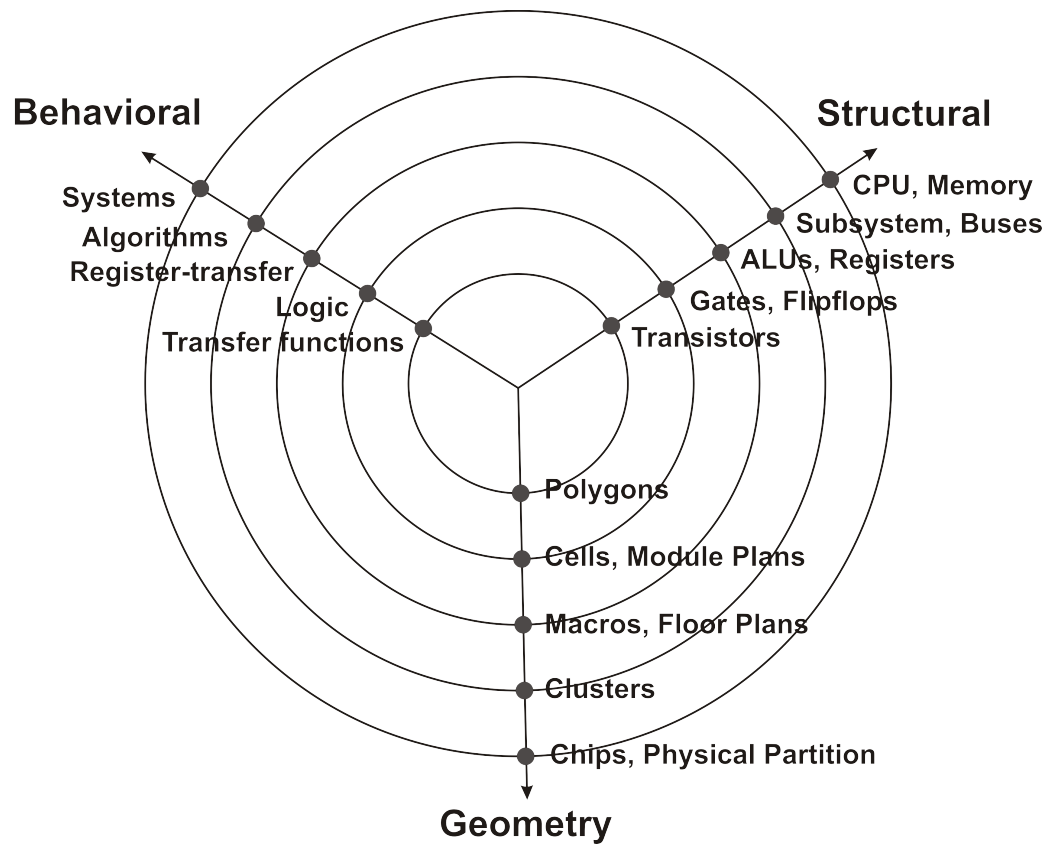


Figure 2.1.2: Y-Chart from Gajski-Kuhn

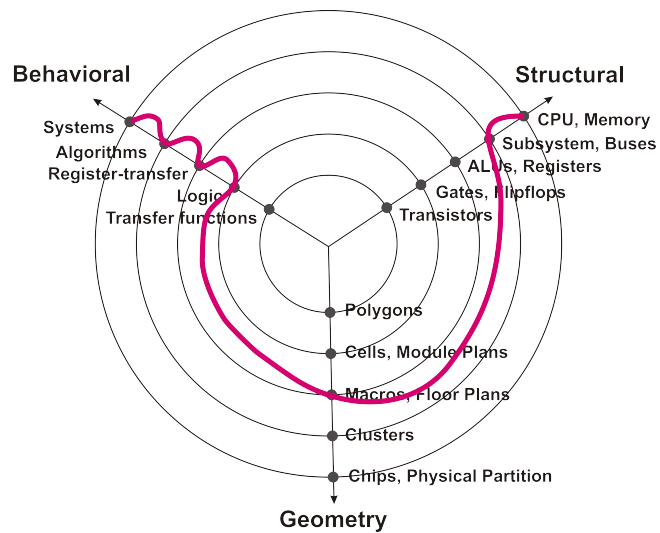


Figure 2.1.3: Different levels in a Y-Chart

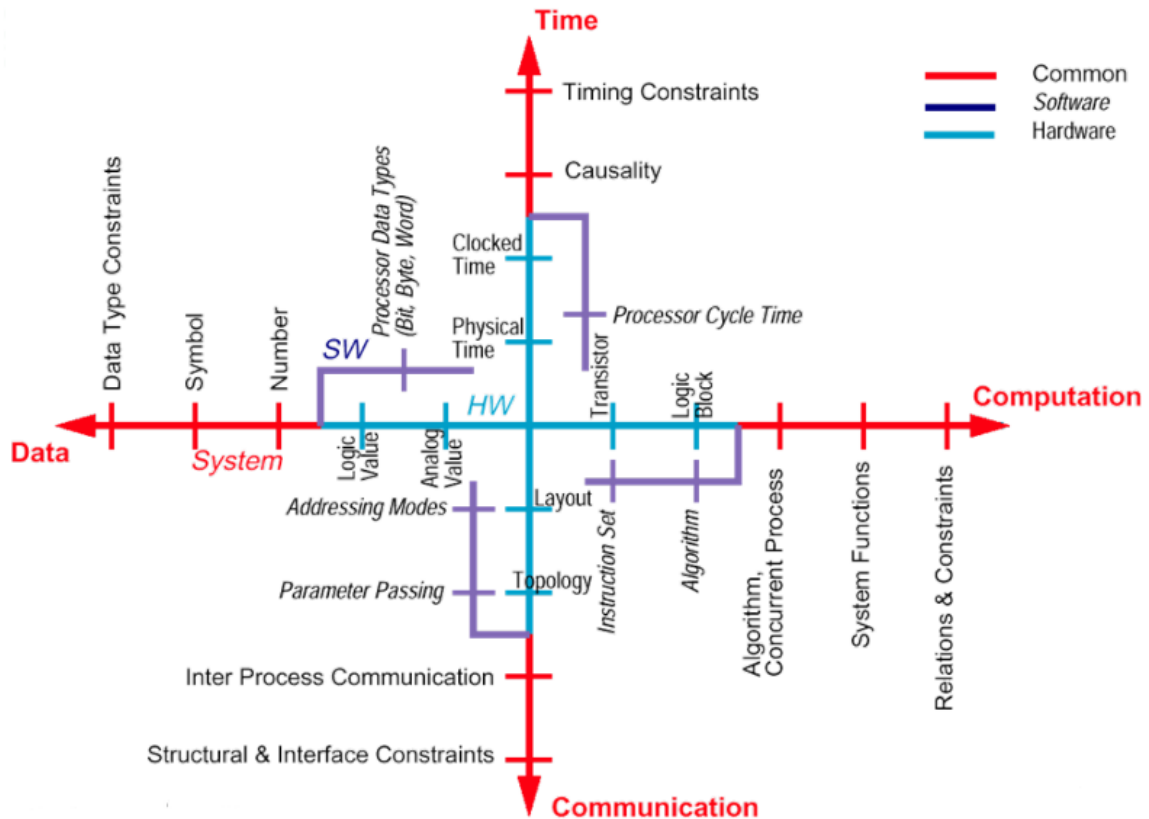


Figure 2.1.4: Hardware and Software Co-Design

- concurrency and time behavior difficult to modeling

Design methods

- capture and simulate
 - specification of the embedded system (development)
 - generation of block diagrams (details not specified)
 - simulation and verification
- describe and synthesize
 - generate functions (software and hardware) from CAD-tools
 - logic synthesis as an integral component of the draft
 - use of hardware description languages (like VHDL)
- specify-explore-refine
 - CAD/CASE-tools supports the explore-step (verification of different alternatives)
 - unfortunately these steps are not supported for all levels of the Co-Design

Different views on a model

- behavioral view

- use models or part of models as black boxes
- focus on states, processes, procedures
- structural view
 - specify the implementation of the system (e.g. connection of the components implies the behavior of the system)
 - processors, racks, printed circuit boards in the foreground
- view on physical components
 - specify the physical characteristics of the components (e.g. expansion, layer, physical characteristics of the connections)

Classification of models

- a model which fulfils all the requirements of embedded system doesn't exist
- actual models are focused on specific aspects
 - state space oriented
 - activity oriented
 - structural oriented
 - data driven oriented
 - heterogeneous oriented

2.2 State Space Oriented Models

- Classical automata (FSM, finite state machine) are state space models
- Two different versions: Moore- and Mealy-Automata

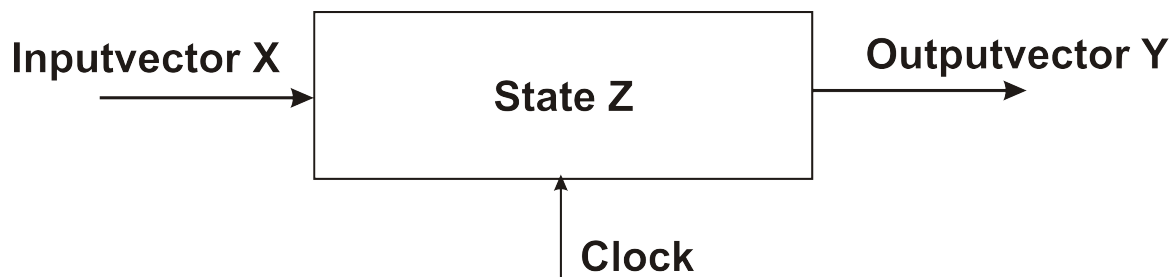


Figure 2.2.1: Blockdiagram of a FSM

Moore – automata

$$Y = \lambda(Z); Z^+ = \delta(X, Z)$$

Mealy – automata

$$Y = \lambda(X, Z); Z^+ = \delta(X, Z)$$

$\lambda = \text{output – function}$

$\delta = \text{computes the next state } Z^+$

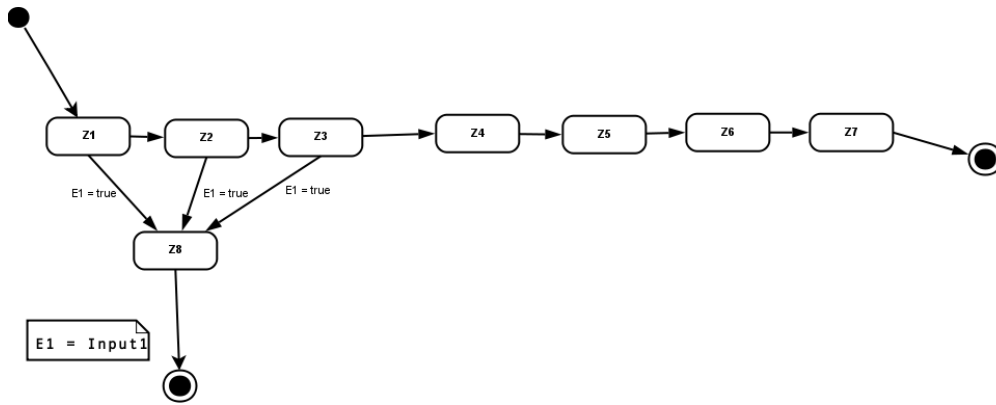


Figure 2.2.2: FSM more in detail

- FSM not useful for complex systems (to much details)
- Introduction of hierarchy in the FSM → StateCharts [Harel, 1987]

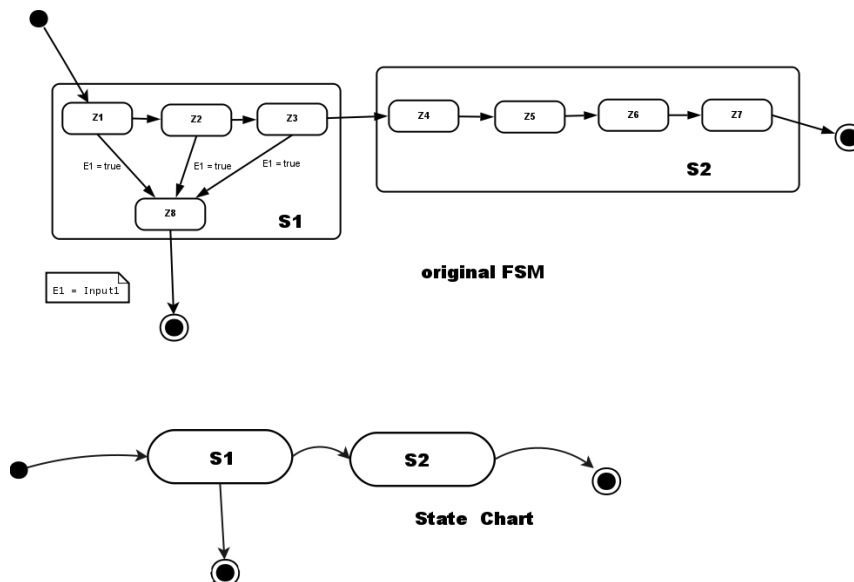


Figure 2.2.3: State Chart

Definition 2.4 Current states of FSMs are called **active states**.

States which are not composed of other states are called **basic states**.

States containing other states are called **super-states**.

Super-states S are called **OR-super-states**, if exactly one of the sub-states of S is active whenever S is active.

- In a OR-super-state only one sub-state is active
- Modeling of concurrency not possible

Definition 2.5 A Super-states S are called **AND-super-states**, if in every AND-super-states exactly one sub-states is active.

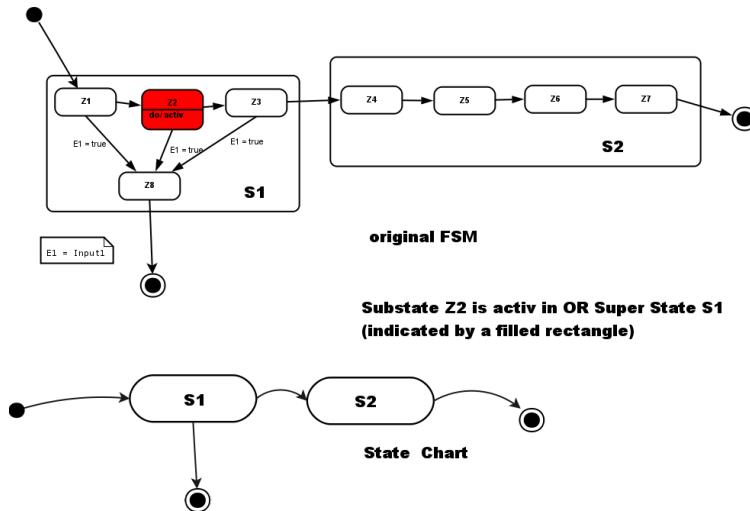


Figure 2.2.4: Active Substate in a Super-State

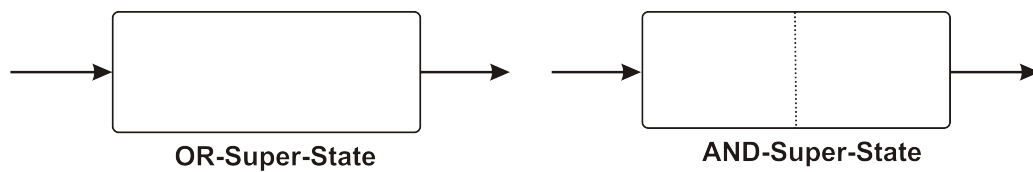


Figure 2.2.5: Concurrency in a AND-Super-State

- In a AND-super-state → modeling of concurrent FSM possible
- Implementation of a AND-super-state with a “Product-FSM”

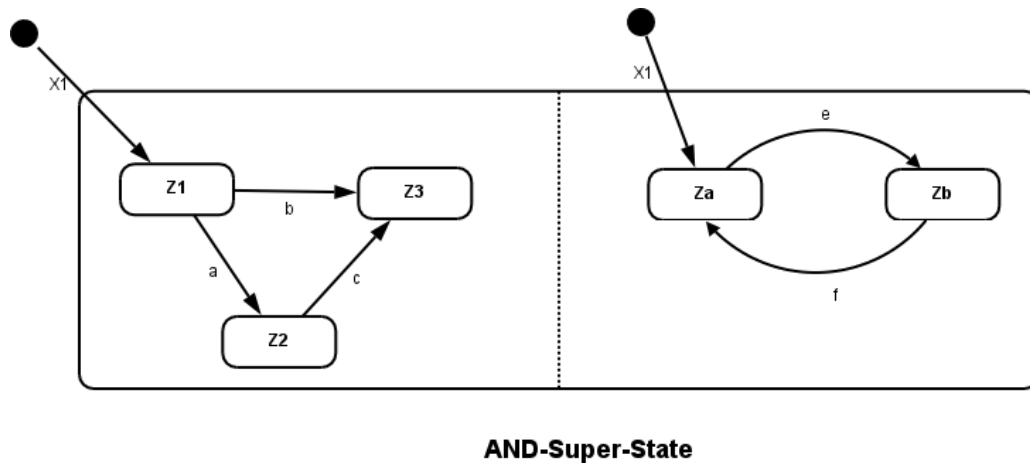


Figure 2.2.6: Example of a AND-Super-State

- **Advantages of state charts modeling**
 - hierarchy allows arbitrary nesting of AND- and OR-super states.
 - semantics defined in a follow-up paper to original paper.

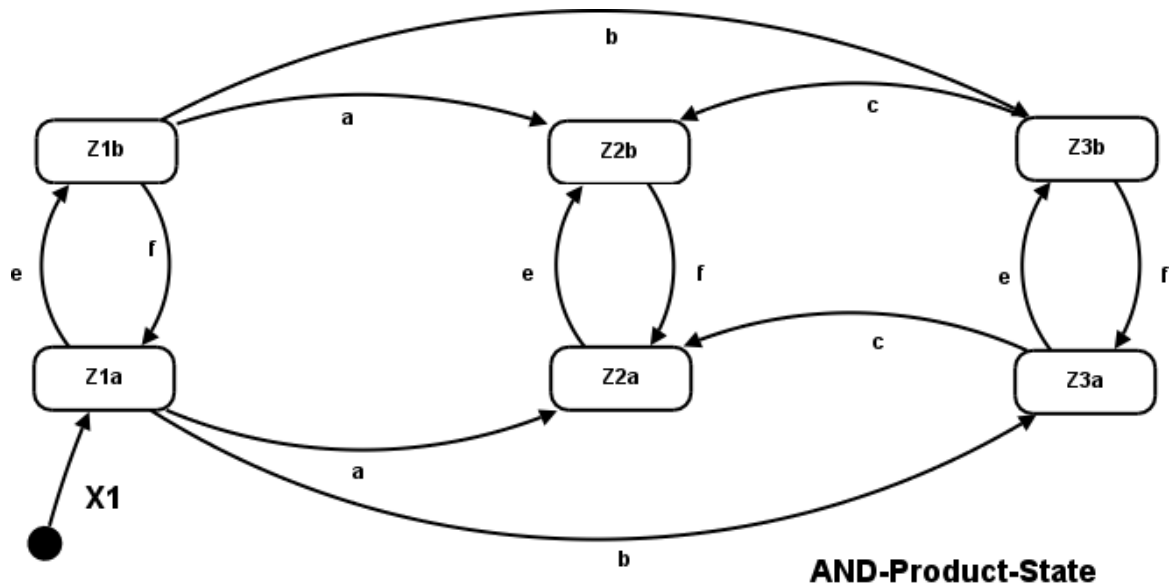


Figure 2.2.7: Decomposition of a AND-Super-State

- large number of commercial simulation tools available (**StateMate**, **StateFlow**, **BetterState**, ...)
- some "back-ends" translate StateCharts into C or VHDL, thus enabling software or hardware implementations
- **Disadvantages**
 - generated C programs frequently inefficient
 - not useful for distributed applications
 - no program constructs
 - no description of non-functional behavior
 - no object-orientation
 - no description of structural hierarchy.
- **Implementation of a Statemachine in C**
 - As an example we take a coffee machine
 - State changes are caused by inputs (left/right/ready/!ready)
 - Output is not State Change -> Moore Machine
- **State Transition Table**
- **C Code - Definition of States**
 - Always use ENUM to define states
 - Define an initial State

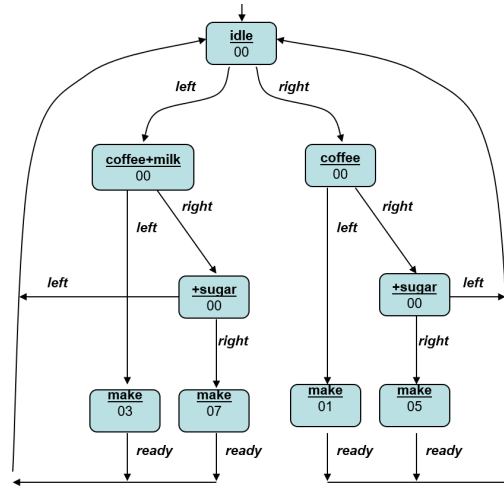


Figure 2.2.8: Coffee Machine FMS

state S	input	state S+	output
00	left	03: coffee+milk	0
idle (initial)	right	01: coffee	0
01	left	05: make coffee	0
coffee	right	03: coffee+sugar	0
02	left	06: make coffee+milk	0
coffee+milk	right	04: coffee+milk+sugar	0
03	left	00: idle	0
coffee+sugar	right	07: make coffee+sugar	0
04	left	00: idle	0
coffee+milk+sugar	right	08: make coffee+milk+sugar	0
05	!ready	05: make coffee	01
make coffee	ready	00: idle	03
06	!ready	06: make coffee+milk	05
make coffee+milk	ready	00: idle	07
07	!ready	07: make coffee+sugar	07
make coffee+sugar	ready	00: idle	
08	!ready	08: make coffee+milk+sugar	
make coffee+milk+sugar	ready	00: idle	

Figure 2.2.9: Coffee Machine State Transition Table

```

1 // -----
2 // the FSM states for the Coffee Automata
3 // -----
4 enum { FSM_STATE_IDLE, FSM_STATE_COFFEE,
5        FSM_STATE_COFFEE_MILK, FSM_STATE_COFFEE_SUGAR,
6        FSM_STATE_COFFEE_MILK_SUGAR, FSM_STATE_MAKE_COFFEE,
7        FSM_STATE_MAKE_COFFEE_MILK, FSM_STATE_MAKE_COFFEE_SUGAR,
8        FSM_STATE_MAKE_COFFEE_MILK_SUGAR,
9 };
10 // -----
11 int fsm_state = FSM_STATE_IDLE;
12 // -----
13 // -----
14 // the FSM outputs one for each state (Moore Automata model)
15 // -----
16 const int fsm_output[NUM_FSM_STATES] = {
17     // output is fsm_output[fsm_state]
18     0, 0, 0, 0, 0, 1, 3, 5, 7
19     // bit0: add coffee, bit1: add milk, bit2: add sugar
20 };

```

• C Code - Inputs and Outputs

- If possible use ENUM again

```

1 // -----
2 // the FSM outputs one for each state (Moore Automata model)
3 // -----
4 const int fsm_output[NUM_FSM_STATES] = {
5     // output is fsm_output[fsm_state]
6     0, 0, 0, 0, 0, 1, 3, 5, 7
7     // bit0: add coffee, bit1: add milk, bit2: add sugar
8 };
9 // -----
10 // the FSM inputs
11 // -----
12 enum {FSM_INP_NONE=0, FSM_INP_LEFT, FSM_INP_RIGHT, FSM_INP_READY };

```

• C Code - State Transition

- This function is called periodically
- The period defines the response time of the system to input

```

1 int fsm_tic(int fsm_inp) // called periodically e.g. each 50 ms
2 {
3     // the FSM transients realizing the state transient table
4     // -----
5     switch (fsm_state) {
6     case FSM_STATE_IDLE:
7         if (fsm_inp == FSM_INP_LEFT) fsm_state = FSM_STATE_COFFEE;
8         if (fsm_inp == FSM_INP_RIGHT) fsm_state = FSM_STATE_COFFEE_MILK;
9         break;
10    case FSM_STATE_COFFEE:

```

```

11     if (fsm_inp == FSM_INP_LEFT) fsm_state = FSM_STATE_MAKE_COFFEE;
12     if (fsm_inp == FSM_INP_RIGHT) fsm_state = FSM_STATE_COFFEE_SUGAR;
13     break;
14 case FSM_STATE_COFFEE_MILK:
15     if (fsm_inp == FSM_INP_LEFT) fsm_state = FSM_STATE_MAKE_COFFEE_MILK;
16     if (fsm_inp == FSM_INP_RIGHT) fsm_state = FSM_STATE_COFFEE_MILK_SUGAR;
17     break;
18 case FSM_STATE_COFFEE_SUGAR:
19     if (fsm_inp == FSM_INP_LEFT) fsm_state = FSM_STATE_IDLE;
20     if (fsm_inp == FSM_INP_RIGHT) fsm_state = FSM_STATE_MAKE_COFFEE_SUGAR;
21     break;
22     case FSM_STATE_COFFEE_MILK_SUGAR:
23     if (fsm_inp == FSM_INP_LEFT) fsm_state = FSM_STATE_IDLE;
24     if (fsm_inp == FSM_INP_RIGHT) fsm_state =
        FSM_STATE_MAKE_COFFEE_MILK_SUGAR;
25     break;
26 case FSM_STATE_MAKE_COFFEE:    //here we wait for the coffeemaking to
        finish (ready!)
27 case FSM_STATE_MAKE_COFFEE_MILK:
28 case FSM_STATE_MAKE_COFFEE_SUGAR:
29 case FSM_STATE_MAKE_COFFEE_MILK_SUGAR:
30     if (fsm_inp == FSM_INP_READY) fsm_state = FSM_STATE_IDLE;
31     break;
32 default:
33     ;    // no state change
34 }
35 OUTPUT(fsm_output[fsm_state]);    // Moore FSM: one output per state
36 return fsm_state;
37 }

```

- **Recipe for realizing a Moore Statemachine**

- Analyze the problem before coding plot a state diagram define the states
- Use enums to encode states in a well readable form
- Define outputs as enums with suitable encoding e.g. as bit-fields
- Define input conditions causing the state transitions as enums as well
- Realize a periodically called fsm-tic function realize transients and outputs

- **Some Tools for creating and testing State Machines**

- The purpose of this list is not to give an exhaustive list of tools
- The first part contains browserbased and free to use (at least with restrictions)
- The second part contains free but installed tools
- The final part commercial tools
-
- Browser based tools:
 - Online Visual Paradigm - Commercial, with free option here:
 - <https://online.visual-paradigm.com/app/diagrams/#diagram:proj=0&type=StateMachine>
 - Nice editor
 -
 - Draw.IO - Free- Now located at
 - <https://app.diagrams.net/>

- Many different options
-
- State Machine Simulator
- http://ivanzuzak.info/noam/webapps/fsm_simulator/
- Powerful, complex editor and simulator/tester for FSM
- Free tools:
 - Open Modelica available as part of the language and extensions in StateGraph
 - <https://build.openmodelica.org/Documentation/Modelica.StateGraph.html>
 - Powerful simulation, more complex, good integration for co-simulation
- Commercial tools:
 - Matlab / Simulink Stateflow library
 - <https://de.mathworks.com/products/stateflow.html>
 - Powerful simulation, more complex, good integration into other libraries

2.3 Activity Oriented Models

- modeling of the flow of datas in an embedded system → **DFG** (data flow graph)
- node contains the activity
- special nodes are input/output nodes and memory-nodes
- shows the dependencies of data on all abstraction levels
- ideally for the specification phase and as input for scheduling, because there don't exits sequential defaults
- hierachical structure of DFG possible
- extension of a DFG with control flow → **CFG**
- introduction of “control”-elements
- edges transfers the control flow
- CFG → Flow charts

Message Sequence Charts (MSC)

- modeling of timing and scheduling behaviour
- graphical means for representing schedules
 - time is a separate axis (timing aspects are explicitly shown)
 - time axis vertical
 - activity axis horizontal
- **Advantages of activity oriented modeling**
 - appropriate for visualizing schedules
 - proven method for representing schedules in transportation
 - hierarchy allowed in DFG and CFG states.

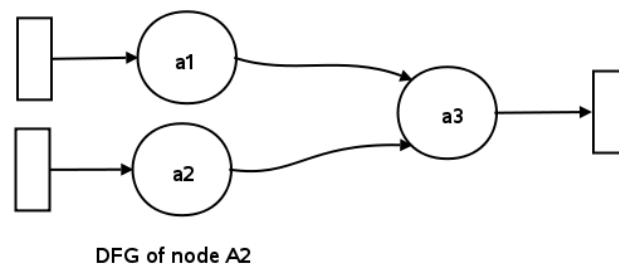
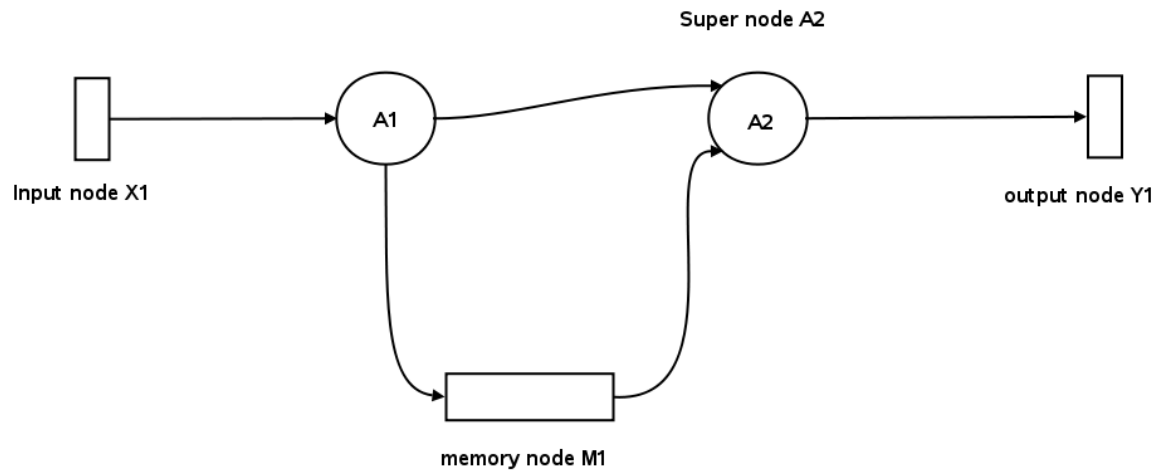


Figure 2.3.1: Flow Graph

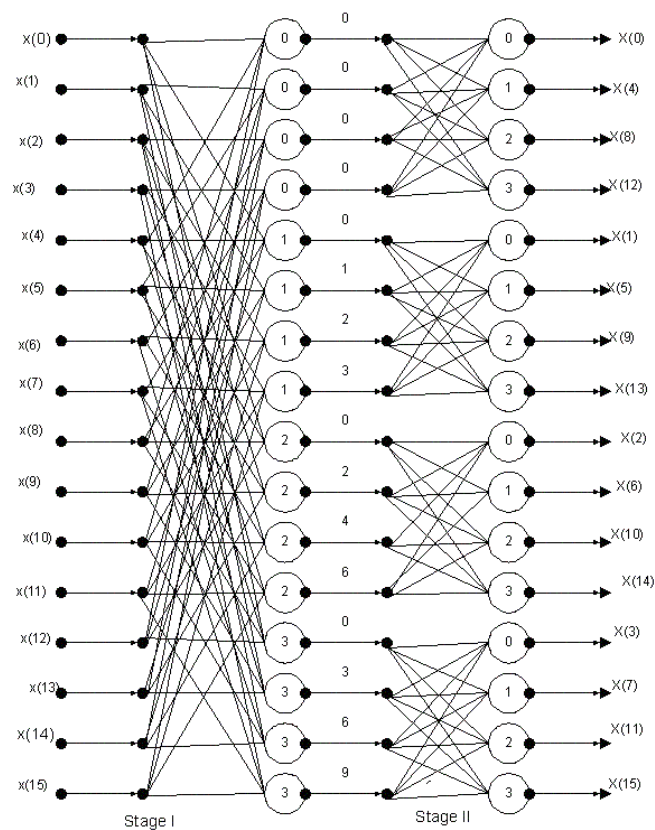


Figure 2.3.2: DFG of a FFT-algorithm

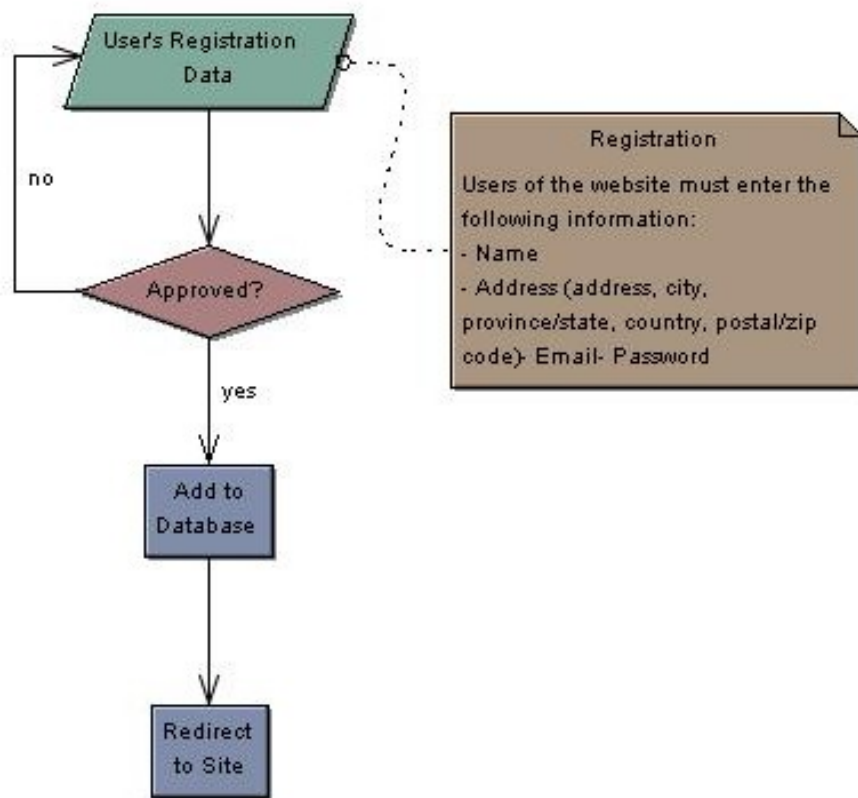


Figure 2.3.3: Flow Chart of an Algorithm

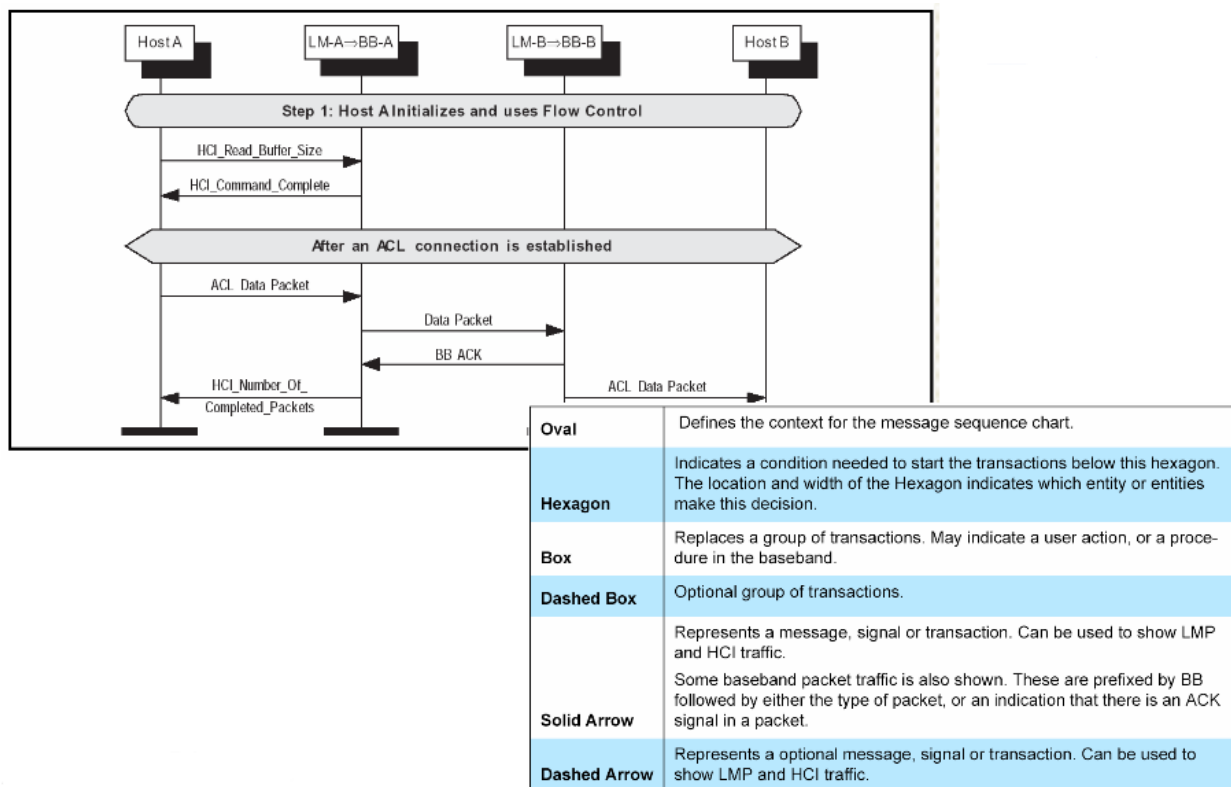


Figure 2.3.4: Message Sequence Chart

- **Disadvantages**
 - focused on one case, no timing tolerances or concurrency (MSC)
 - no description of non-functional behavior
 - no object-orientation
 - no description of structural hierarchy (MSC)

Chapter 3

Cyber-Physical Systems

Cyber-Physical Systems (CPS) are typical mechatronics systems because they are based on mechanical systems augmented with communication capabilities and connectivity controlled by IT-components. The basic idea is to take physical systems, described by a set of physical entities and equations and control or monitor these systems by computer based algorithms. All the components have first to be described and the behavior simulated and during the development process an exchange between the simulated and the real parts takes place.

Today we find cyber-physical systems especially in bigger projects which deal with aerospace, automotive, chemical, energy, manufacturing applications. CPS involves interdisciplinary approaches like system engineering, control theory but also mechanical design and design in general. An interesting link exists between CPS and the new technology Internet of Things (IoT).

In this chapter we'll first try to get a shorter look at cyber-physical systems. Then we focus on a tool which gives us an opportunity to modelling different physical systems like mechanical and electrical systems but also question about the thermal requirements we'll try to answer. The modelling tool (Modelica) offers methods to write physical equations or to use a graphical user interface to describe systems. The systems themselves can have a simple input output behaviour but also complex relationships.

3.1 Overview, Delimitation of the Topic

Topics of Cyber-Physical Systems

- Combination between:
 - mechanical components
 - electrical components
 - design aspects
 - communication capabilities
 - IT-components
- simulation of all different components
- study of the dependencies between the components
- mix between simulated components and real hardware (Hardware in the Loop, HIL)

Definition 3.1 An **Cyber-Physical Systems** are integrations of computation with physical processes. Embedded computers and networks monitor and control the physical processes, usually with feedback loops where physical processes affect computations and vice versa. [Edward A. Lee, 2008]

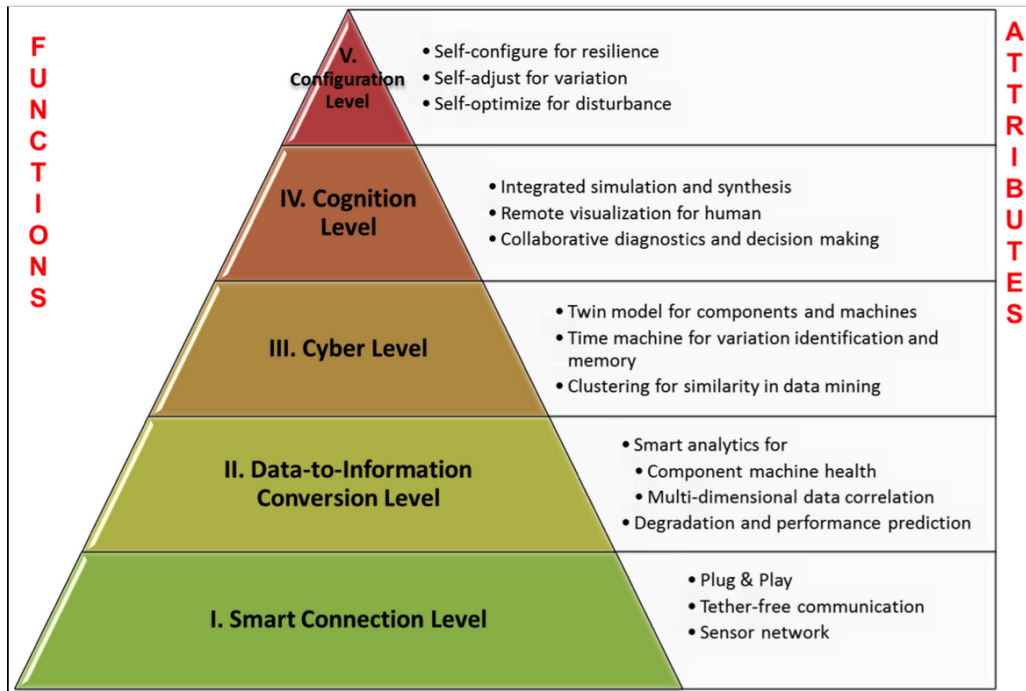


Figure 3.1.1: Architecture of a Cyber-Physical System

- Study of the behaviour of Cyber-Physical Systems
 - Simulationtools
 - use of linear and nonlinear equations systems
 - modeling time continuing and time discret systems
 - models with concentrated parameters
 - blockoriented systems
 - objectoriented systems
- typical tools for the simulations
 - MatLab (Simulink)
 - Mathematica
 - Modelica (Dymola)
 - OpenModelica
- standard methods in technical applications → blockoriented modells (systems)
- fixed input/output relations (causal dependency, assignments)
- often with the use of graphical input/output support
- in physical systems objectoriented methods are more useful
- focus on physical behavior (equations)
- input/output relations more in the background (assignments not fixed)

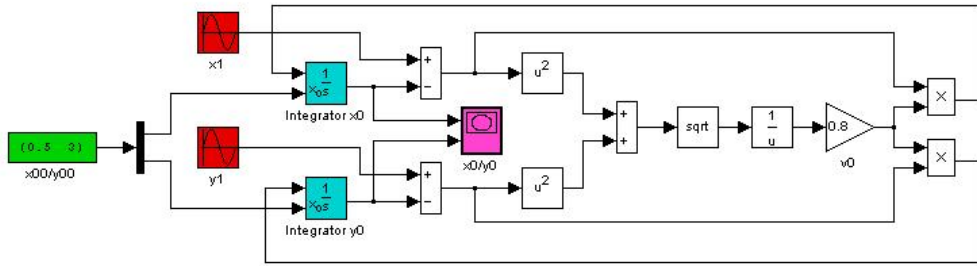


Figure 3.1.2: Blockoriented Modell (Simulink)

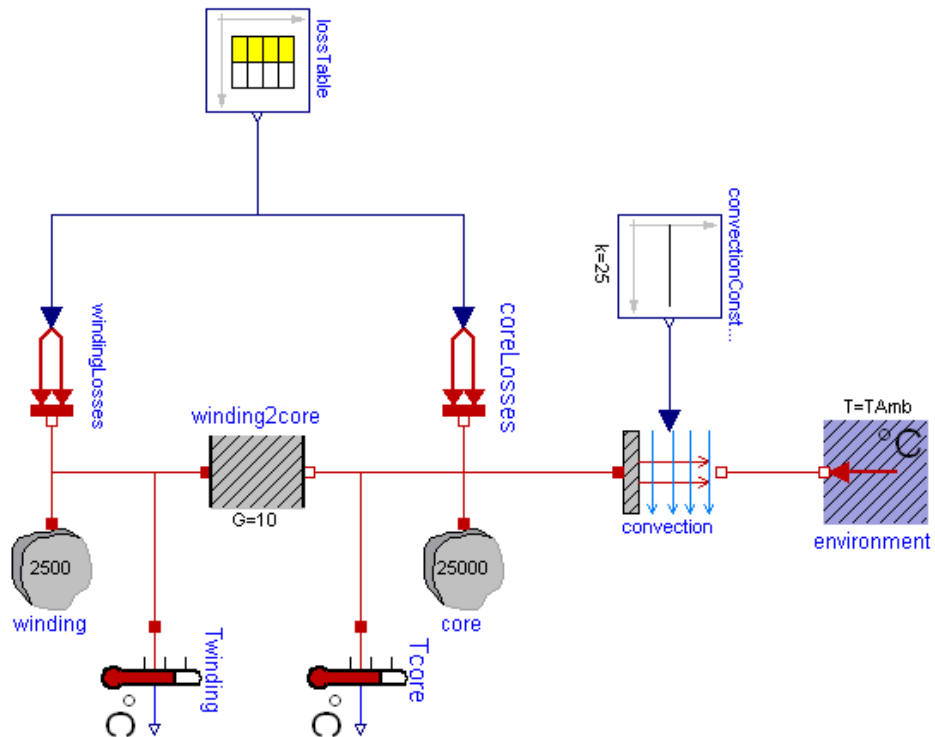


Figure 3.1.3: OpenModelica Modell of a Heattransfer

3.2 OpenModelica

Features of OpenModelica

- objectoriented modeling and simulation
- integration of hardware components in the simulation (HIL, Hardware in the loop)
- standardisation of interfaces for simulation (FMI, Functional mookup interface)
- description of a physikal system by a set of equations
- no specific description of of input/output relations (equation systems has no assignments)
- focus on physical dimension system (SI dimensions)

Features as an objectoriented modelling and simulation tool

- data structure with objects (dada objects) and methods (functions)
- features and methods are linked to objects
- features of objects are grouped in classes
- classes can be parts of other classes (hierachical dada structures, modells)
- group of classes in packages
- separation between declaration and quantity (e.g. declaration of a resistor and specification of values)
- inhertance of classes and subclasses

Features as a multi-domain modelling an simulation tool

- declarative language (equation set, acausal modelling, high level specification)
- combine modelling of electrical, mechanical, thermodynamic, hydraulic biological .. systems)
- visual component modelling and simulation
- code generation opportunity (control application)

First Elements of OpenModelica

- Data types
 - Boolean: true, false
 - Integer: 32-bit Integer-number (same as in C)
 - Real: 64-bit floating point number (same as double in C)
 - String: char-string
 - enumeration: (same as in C)
- acausal equation (no assignments, but description of relation between the Potential and the Flow)
- parameter
- example: simple differential equation $\dot{x} = -a \cdot x$

```
1  class HDE
2  Real x(start = 1);
3  parameter Real a = 1;
4  equation
5  der(x) = - a*x;
6  end HDE;
```

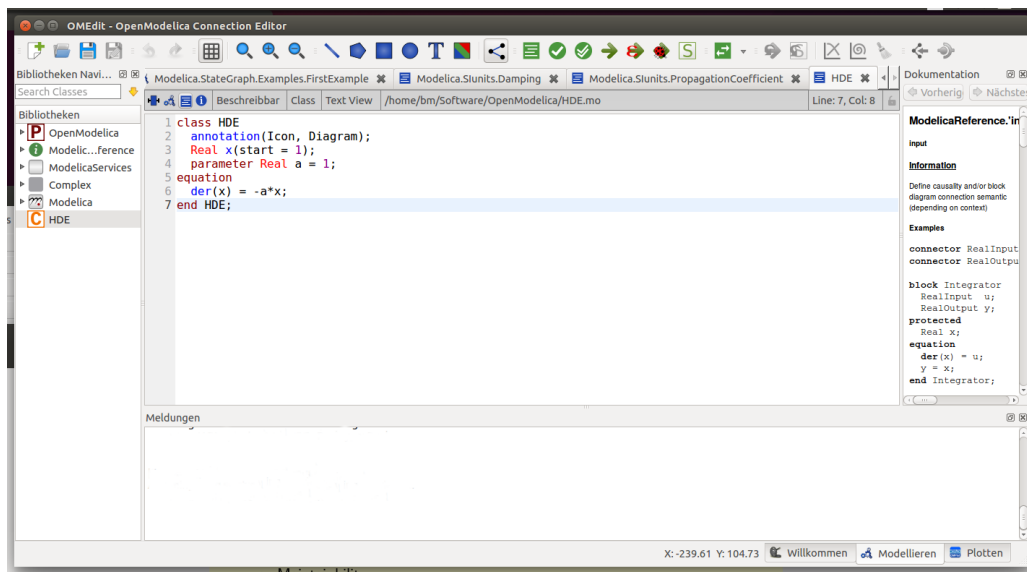


Figure 3.2.1: Modell HDE in OMEdit

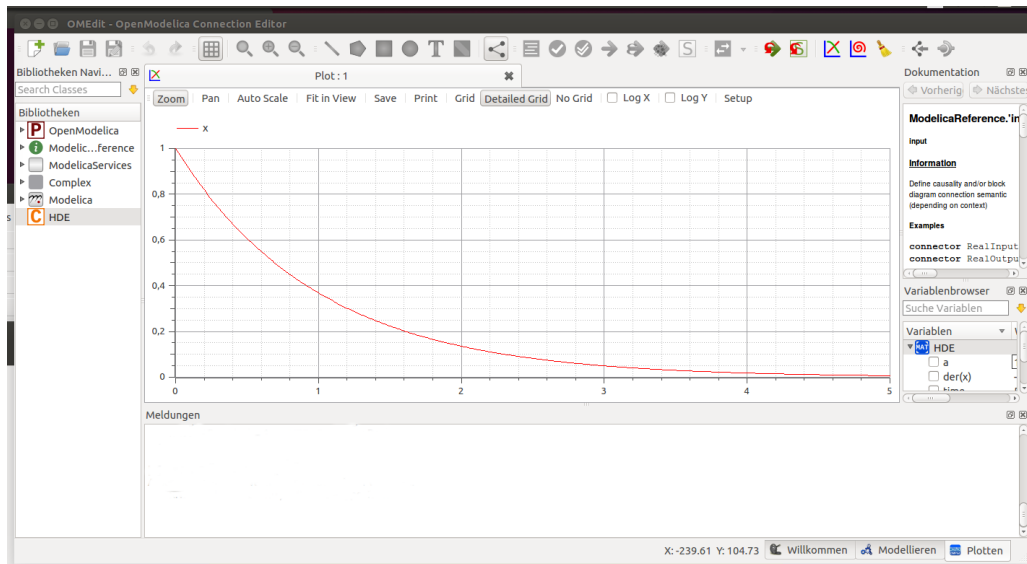


Figure 3.2.2: Plot of the Modell HDE

- number equations which describes of relation between the Potential and the Flow indicates the number of variables (state variables)
- additionally a modell can contain algebraic equation (no derivatives)
- example: Planar Pendulum

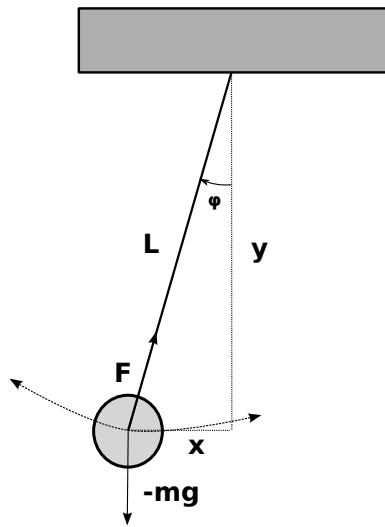


Figure 3.2.3: Planar Pendulum

$$m\dot{v}_x = -\frac{x}{L}F$$

$$m\dot{v}_y = -\frac{y}{L}F - mg$$

$$\dot{x} = v_x$$

$$\dot{y} = v_y$$

$$x^2 + y^2 = L^2$$

```

1  class Pendulum
2  constant Real PI;
3  parameter Real m = 1, g = 9,81, L = 0.5;
4  Real F;
5  output Real x(start = 0.5), y(start = 0);
6  output Real vx, vy;
7  equation
8  m*der(vx)=- (x/L)*F;
9  m*der(vy)=- (y/L)*F - m*g;
10 der(x)=vy;
11 der(y)=vy;
12 x^2 + y^2 = L^2; /* algebraic equation */
13 end Pendulum;
```

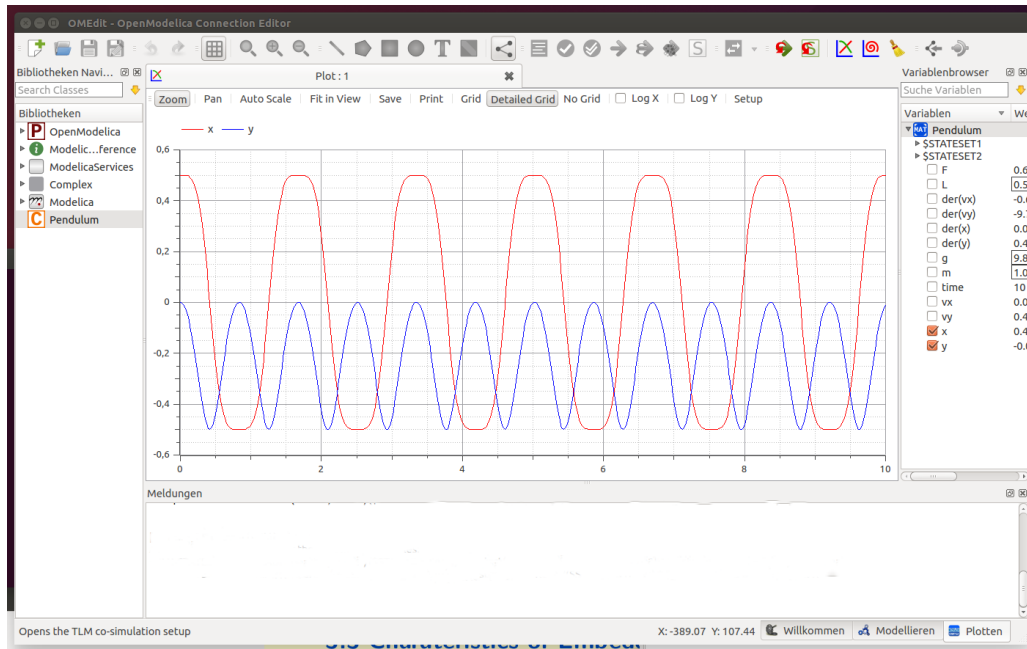


Figure 3.2.4: Horizontal and Vertical Position of a Planar Pendulum

Classes and Instances of OpenModelica

- Classes
 - every object has a class that defines the data and its behavior
 - a class has to kind of members:
 - data variables (constants, parameter, result of computations)
 - equations specifying the behaviour of the data (sometimes interact with other other equations, program execution)
 - classes can be members of other classes
 - example of a class

```

1      class Point    /* Point in a three-dimensional space */
2      Real x;
3      Real y;
4      Real z;
5      end Point;;

```

- Instances
 - instances are generated by the use of classes
 - example of instances of the class point (and define a new class triangle)

```

1      class Triangle
2      Point point1;
3      Point point;
4      Point point;
5      end Triangle;;

```

- with the instantiation an initialization can take place

```

1  class Triangle
2  Point point1(start={1,2,3});
3  Point point;
4  Point point;
5  end Triangle;;

```

- electrical example with different classes and initializations

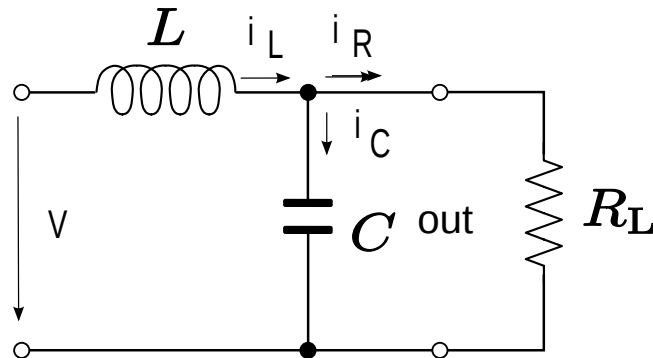


Figure 3.2.5: RLC-Circuit

```

1  Model RLC1 \* A resistor-inductor-capacitor circuit model \*/
2      type Voltage=Real(unit="V");
3      type Current=Real(unit="A");
4      type Resistance=Real(unit="Ohm");
5      type Capacitance=Real(unit="F");
6      type Inductance=Real(unit="H");
7      parameter Voltage Vb=24 "Battery Voltage";
8      parameter Inductance L = 1;
9      parameter Resistance R = 100;
10     parameter Capacitance C = 1e-3;
11     Voltage V;
12     Current i_L;
13     Current i_R;
14     Current i_C;
15 equation
16     V = i_R*R;
17     C*der(V) = i_C;
18     L*der(i_L) = (Vb-V);
19     i_L=i_R+i_C;
20 end RLC1;

```

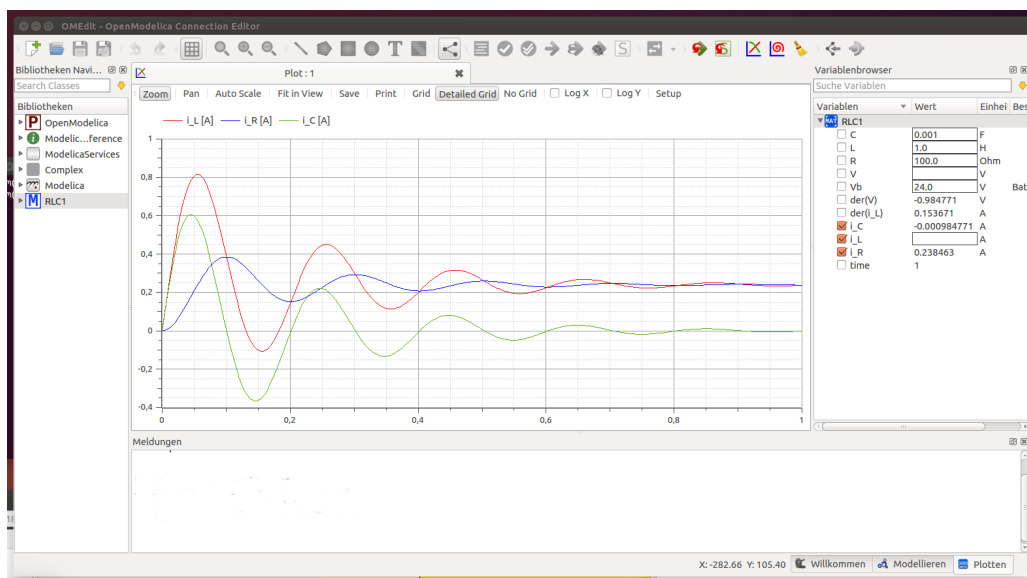


Figure 3.2.6: Behaviour of the different Currents in a RLC-Circuit

Chapter 4

Processors

4.1 Processors for Embedded Systems

Embedded systems need special processors. A standard PC processor isn't suitable, as the microprocessor used doesn't operate independently. It has to be implanted on a motherboard, which serves as the communication interface between the processor and other input/output devices. This involves increased costs and such a system occupies a large space.

Shifting our focus to microcontrollers, we observe that an 8-bit microcontroller possesses small computing power and has too many restrictions in building an appropriate internal or external memory. So these types of processors are also not suitable for a sophisticated embedded system. The above mentioned restrictions are not evident when working with 16- and 32-bit microcontrollers. A big advantage of these types of microcontrollers is that nearly every type of interface is available which we might have to use in an embedded system.

If an Embedded application/project allows us to use a small motherboard (no hard discs and usually no GUI 'graphical user interfaces' etc.) then it makes sense to deploy an "Embedded PC" or a PC/104 system. Such systems contain a special, so called "Mobile microprocessor" like "Pentium M" or "Geode" or "XScale". These processors are variants of standard 8086-type microprocessors (Pentium M and Geode). If we are working with these Embedded PC's, then you have access to support in the form of well known standard PC tools like compilers, linkers and operating systems.

Another type of processor which could be present in an embedded system is a signal processor. A signal processor has much more computing power than a microcontroller. But on the contrary to a microprocessor, a signal processor has nearly the same interfaces available in a microcontroller. The disadvantage of using a signal processor is that there aren't much development tools available.

Microprocessors for Embedded Systems

- use a special version of an 8086-type of microprocessors (Pentium M or Geode)
- these microprocessor are integrated in an "embedded PC" or a "PC/104-system"
- these type of PCs' work without any hard disc (and without any cooling system)
- instead of a hard disc, a "FLASH Memory" is used (e.g. SD-Card)
- if a graphical user interface is available, then the "graphical processing unit" is a part of the microprocessor.

Signal processors for Embedded Systems

- same structure as microcontrollers



Figure 4.1.1: Embedded PC as Single Board Computer

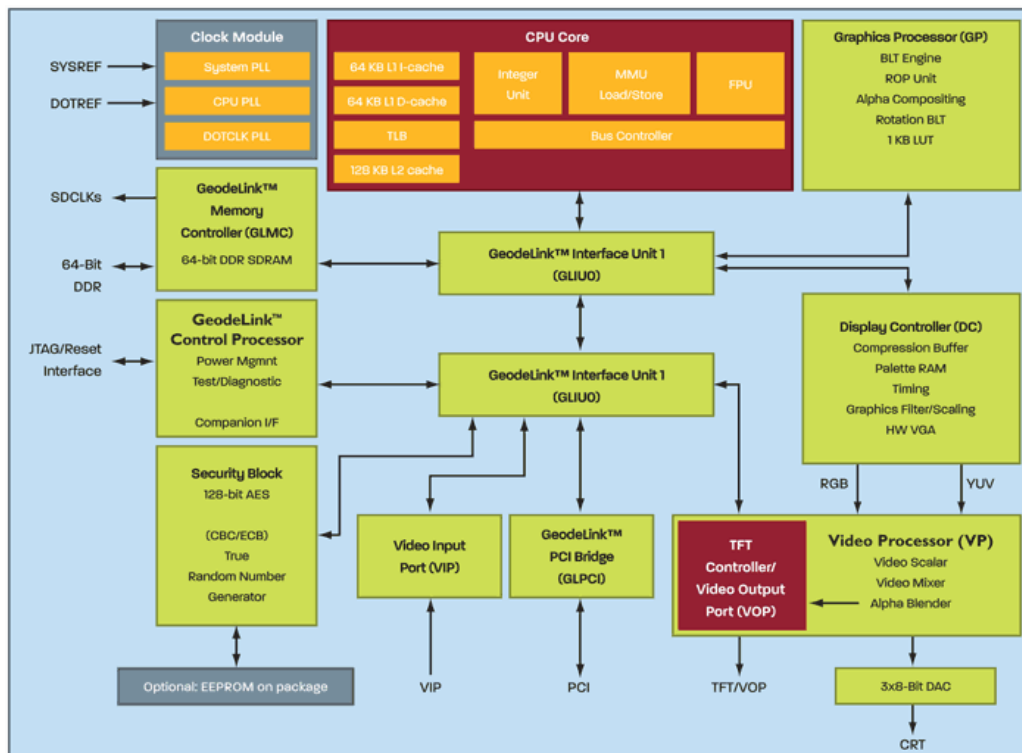


Figure 4.1.2: Blockdiagram of a Geode-Microprocessor

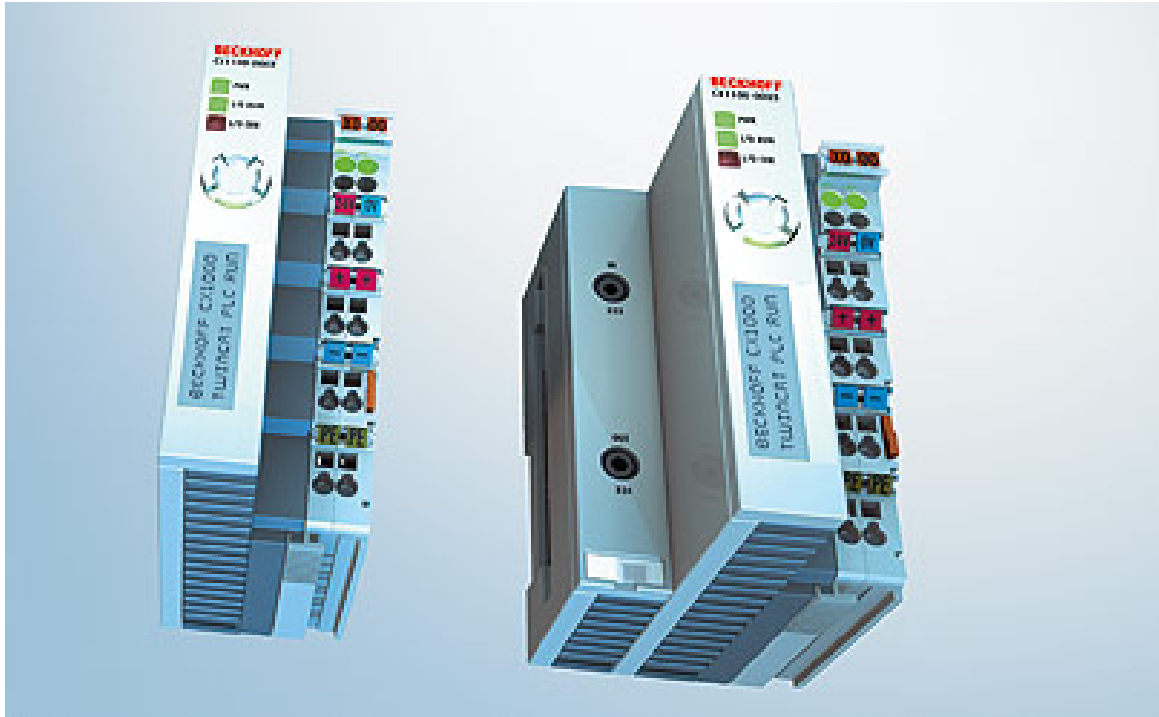


Figure 4.1.3: A different type of Embedded PC

- a lot of interfaces
- external memory available as instruction and data memory (Harvard architecture)
- building of programs using cross-development technique
- much higher computing power than microcontrollers
- specially built for signal processing applications (e.g. video compression, audio processing etc.)

Microcontrollers for Embedded Systems

- most embedded systems employ a microcontroller
- differences in the use of internal memory and/or external memory
- human interface almost nonexistent or really small
- according to the reduction of hardware costs more functions implemented in a processor
- a combination of microcontroller and signal processor are build (OMAP-Processor, part of the Beagle-Board)

Components of the microcontroller/signal processor based Beagle Board

- microcontroller ARM Cortex-A8 (wordlength 32 Bit)
- signalprocessor TMS 320DM641 (fix point digital signal processor, wordlenth 32 bit, multi media control and interfaces)
- TPS 65950 Reset and Power Management
- TFP 410 Interface to external LCD panels
- different digital interfaces (I/O)

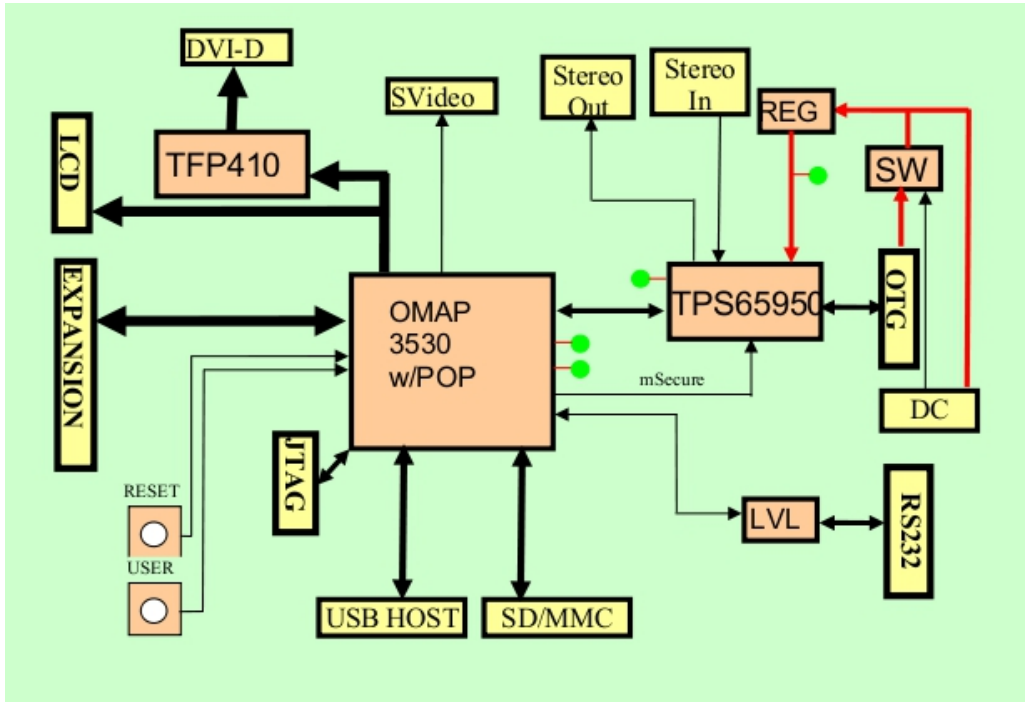


Figure 4.1.6: Blockdiagram of a Beagle Board with different Processors

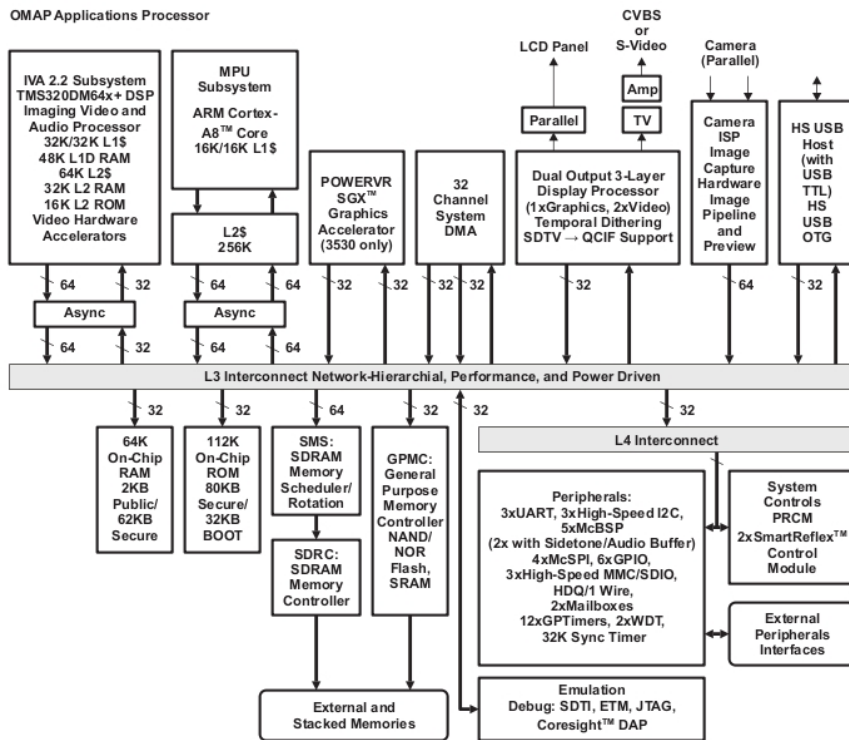


Figure 4.1.7: Features of a OMAP-Processor

4.2 Overview about ARM Processors

Background

- ARM-Processors mostly part of a SoC (System on chip)
- ARM-Processor itself is a IP-Core (intellectual property core), means a licensed part which contains the processor functionality
- ARM-IP-Cores placed in a lot of different boards (like smart phones, embedded systems) and in the beagle board
 - same register model for address- and data register.
 - implementation of a 'load/store' architecture
 - use super user- and a user mode. (Priority of execution)
- implementation of superscalar architecture (Processing unit focused on integer processing).
- integration of FPU coprocessor (NEON Processing unit focused on Floating Point processing).
- in some processors a MMU (memory management unit) is implemented (important for operation systems)
- supports SIMD-applications.
- integration of a lot of interfaces

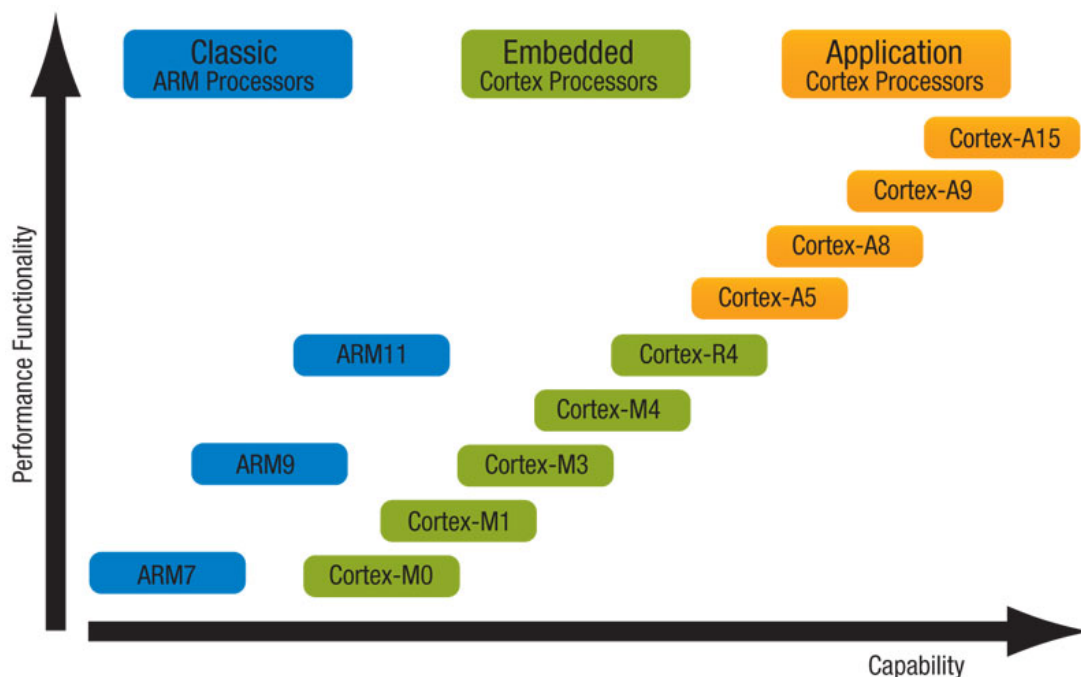


Figure 4.2.1: Overview about different ARM-Processors

4.3 Functions of the ARM-Cortex-A8-Processor

General Functions

- ARM-architecture is a RISC-architecture (reduced instruction set computer)
- according to the RISC-architecture more register banks are available (for every processor unit)
- wordlength of the integer unit (and address unit) 32 Bit (wordlength of general purpose register 32 bit)
- implements a superscalar architecture
- internal Harvard architecture based on two different caches- Instruction and Data (I-Cache and D-Cache respectively).
- memory management unit (MMU).
- different instruction set for:
 - Thumb-instruction set (16 bit integer unit)
 - ARM-instruction set (32 bit integer unit)
 - ThumbEE-instruction set (32 bit integer unit without MMU)
 - NEON-, VFP-instruction set (for SIMD applications in integer (NEON) or floating point format (VFP))
 - FP-instruction set (for single floating point numbers)

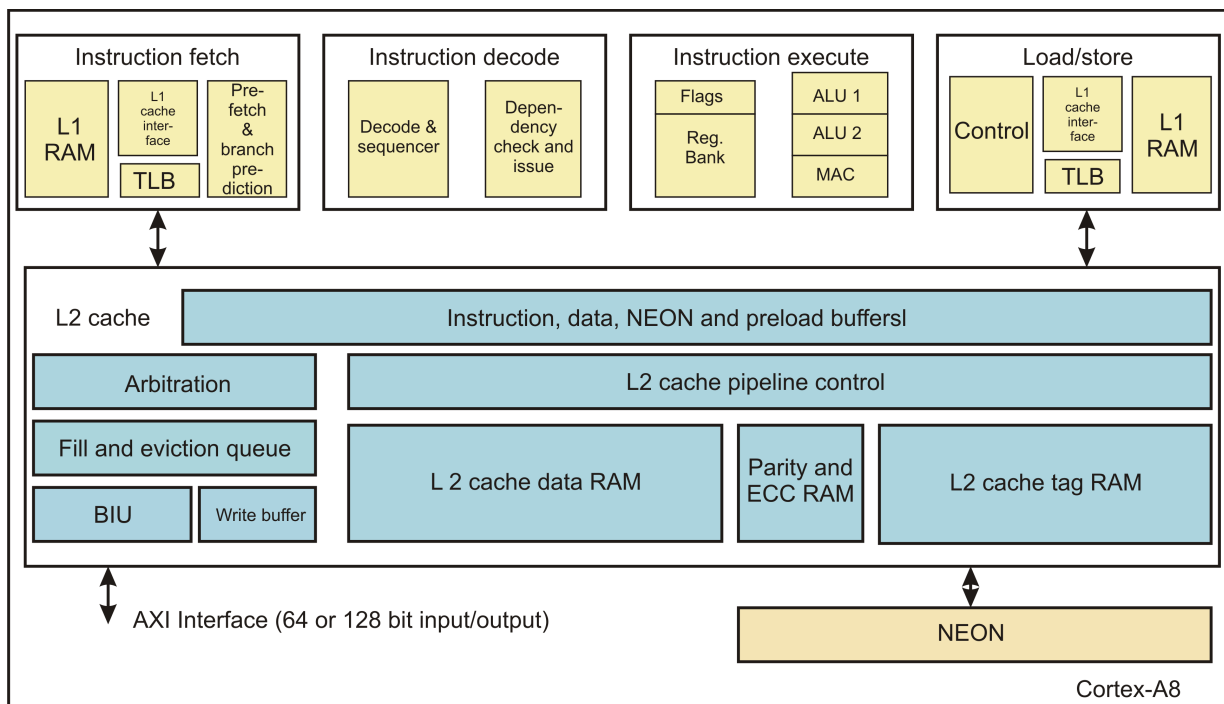


Figure 4.3.1: Blockdiagram Cortex-A8-Processor

ALU1,2

- both ALU (ALU1 and ALU2) contains 32-bit integer units (could be used for signed, unsigned and fix point operations).
- standard wordlength 32 bit
 - word 32 bit (data manipulation, address operations)
 - halfword 16 bit (data)
 - byte 8 bit (data)

- doubleword 64 bit (data)
- works with RISC-operation on fixed length instructions (32 bit).
 - load/store technique used
 - in every instruction a register is involved (as a source or destination register)
 - the order of operands are allways the same
 - special data transfer instruction implemented for a read access (load) and a write access (store)
 - function calls work register based (not memory based as for CISC-processors)
 - conditions could be used for transfer and data manipulation instructions
- pipeline not visible for the user (only in some special instructions an insert of NOPs are necessary to synchronize the pipeline).
- to support the pipeline operation for branches (especially conditional branches) a branch cache is involved.
- three different instruction sets are available for the ALU
 - Thumb instruction set for 16 bit applications
 - ThumbEE instruction set for 32 bit applications (typical embedded applications)
 - ARM instruction set for 32 bit application and the support of virtual memory management systems

		User Mode				
		Privileged Modes				
		Exception Modes				
User	System	Supervisor	Abort	Undefined	Interrupt	Fast Interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

Stack Link

CPSR Current Processor Status Word
SPSR Saved Processor Status Word

Figure 4.3.2: Register Set of the ALU

- ALU (and the processor) runs in seven different modes
- register set from r0 - r12 in every mode visible (without fast interrupt mode)
- in exception modes the contents of the current processor status register (CPSR) are automatically saved (SPSR) (before the exception procedure starts)

- modes
 - user → standard mode
 - system → mode for operating systems
 - supervisor → mode for operating systems (mode after a reset)
 - abort → illegal instruction, illegal data operation
 - undefined → undefined instruction detected
 - interrupt → external interrupt
 - fast interrupt → internal interrupt (trap)

MAC (multiply and accumulate)

- part of the standard multiply function
- supports a multiplication and a add-operation in one instruction
- different wordlength and signed/unsigned numbers are supported
- typical examples (wordlength)
 - $32 = 32 +/\!- 16 \times 16.$
 - $32 = 32 + 16 \times 16 + 16 \times 16$
 - $32 = 16 \times 16$
 - $64 = 64 +/\!- 32 \times 32$
 - $64 = 64 +/\!- 16 \times 16$
 - $64 = 32 \times 32$

NEON / VFP (SIMD and vector processing of integer and floating point numbers)

- supports integer (fix point) and floating point packed calculations (NEON) of 32 bit numbers
- vector floating point numbers with 32 bit or 64 bit are processed by the VFP-unit
- every floating point number is:
 - normalized
 - there exists no carry bit or a doubling of the word format after a multiplication (very useful in combined arithmetical expressions)
 - every floating point number could be characterized (zero, normalized, denormalized, exist, free)
- operations on NEON- and VFP-unit works as
 - scalar operation on single (s = 32 bit), double (d = 64 bit) or quad q = 128 bit) precision, the involved registers have not be placed at the beginning of a bank
 - vector operations on 4 single precision elements, the involved registers starts at the beginning ore in the middle of a bank
 - vector operations on 2 double precision elements, in this case the registers starts at the beginning ore in the middle of a bank
 - scalar operation on 1 quad precision element
- Examples for scalar and vector operations
 - scalar operation
 - FADDS S12, S21, S22

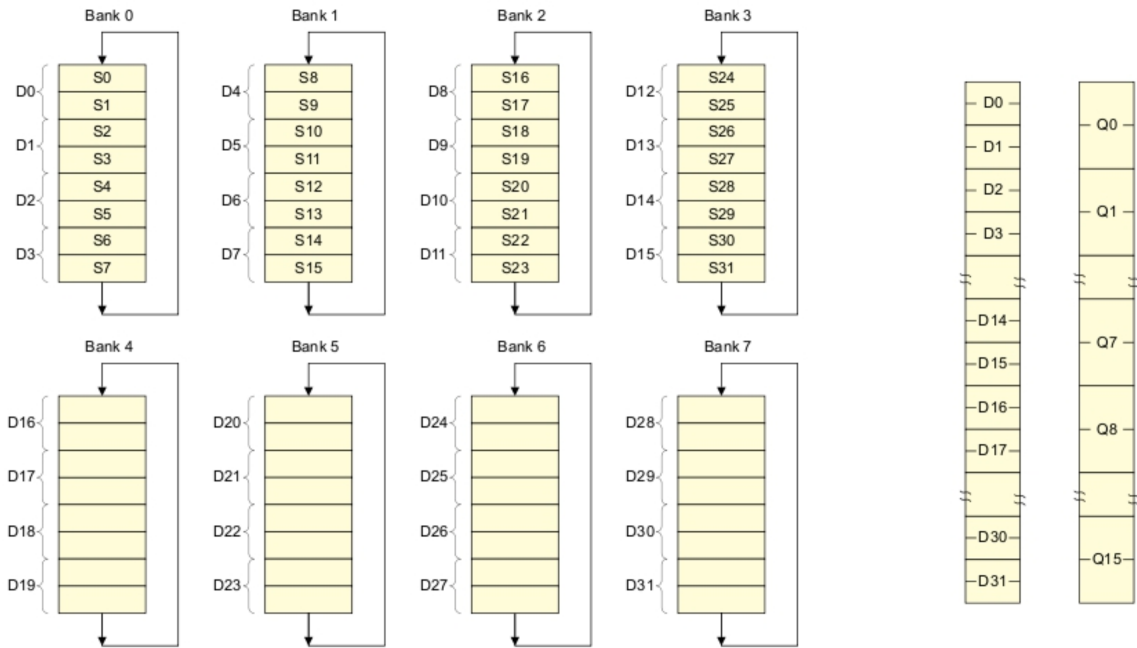


Figure 4.3.3: Register set of NEON/VFP Coprocessor

- vector operation on single precision with 4 iterations
- FMACS S16, S0, S8 (instruction)

FMACS S16, S0, S8 ; 1st iteration

FMACS S17, S1, S9 ; 2nd iteration

FMACS S18, S2, S10 ; 3rd iteration

FMACS S19, S3, S11 ; 4rd and last iteration

- vector operation on double precision with 2 iterations
- FMULD D12, D8, D2 (instruction)

FMULD D12, D8, D2 ; 1st iteration

FMULD D13, D9, D3 ; 2nd and last iteration

MMU (memory management unit)

- supports a flexible, software-defined virtual environment.
- in the case of virtual memory, the MMU supports 4-, 8-Kbyte, 1-Mbyte or 16-Mbyte page sizes.
- the pages addressed with separate, 32-entry, fully associative instruction and data TLBs (TLBs translation lookaside buffer).
 - the separation of instruction- and data-TLB allows a parallel access of both instructions and data (Harvard-architecture).
 - the TLB works as registers (faster than memory based entries)
- the MMU itself is memory-mapped (control, status, and fault registers)
- the access to the MMU starts with a control-register
- MMU supports programmable address space prioritisation steps (ASID address space identifier).

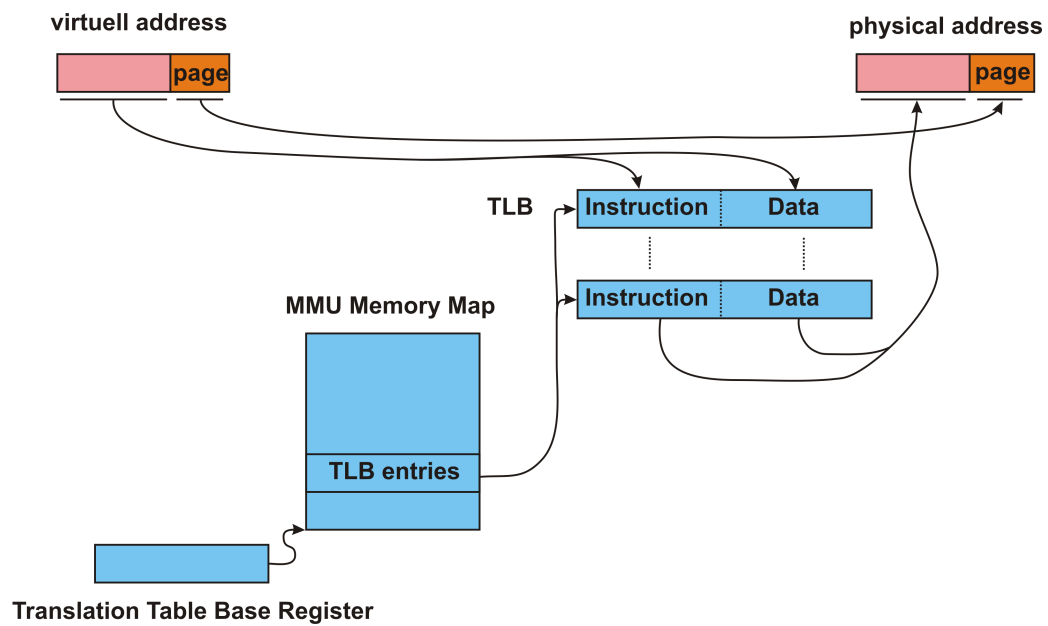


Figure 4.3.4: Principal functions of the MMU

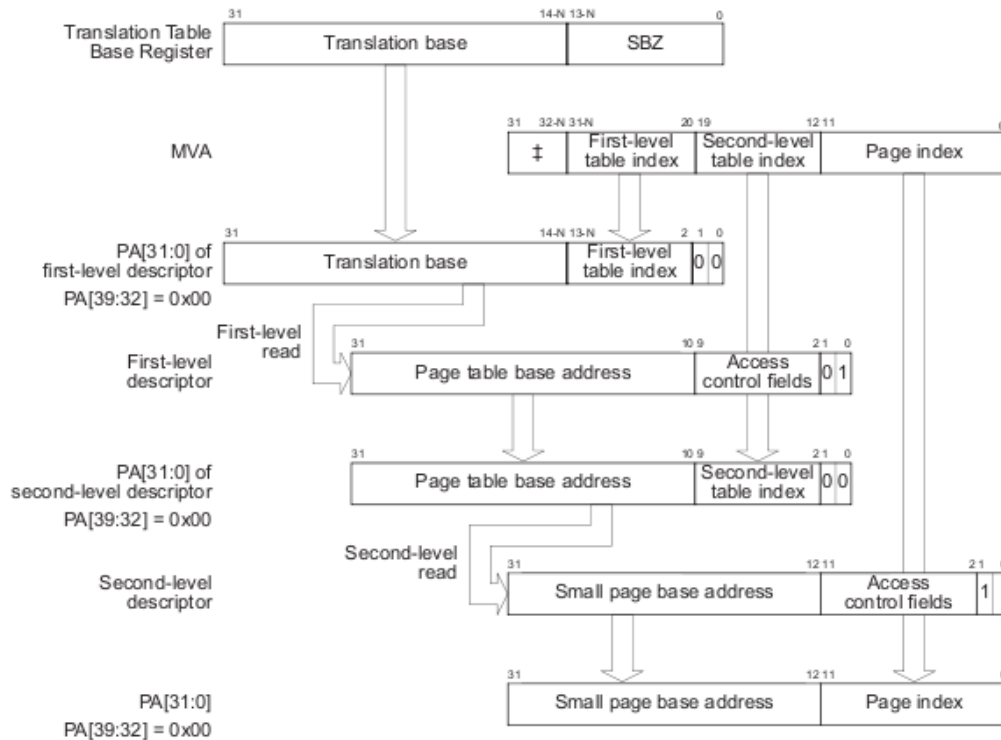


Figure 4.3.5: Page Address Translation

4.4 Overview about the DSP

DSP Functions and additional Features

- DSP supports the OMAP-Processors with
 - processor functionality to work on digital signal algorithm
 - internal RAM and ROM (data memory, Boot-ROM)
 - external RAM (SDRAM for instruction and data memory)
 - a lot of digital interfaces
- DSP is a 32 bit fix point processor

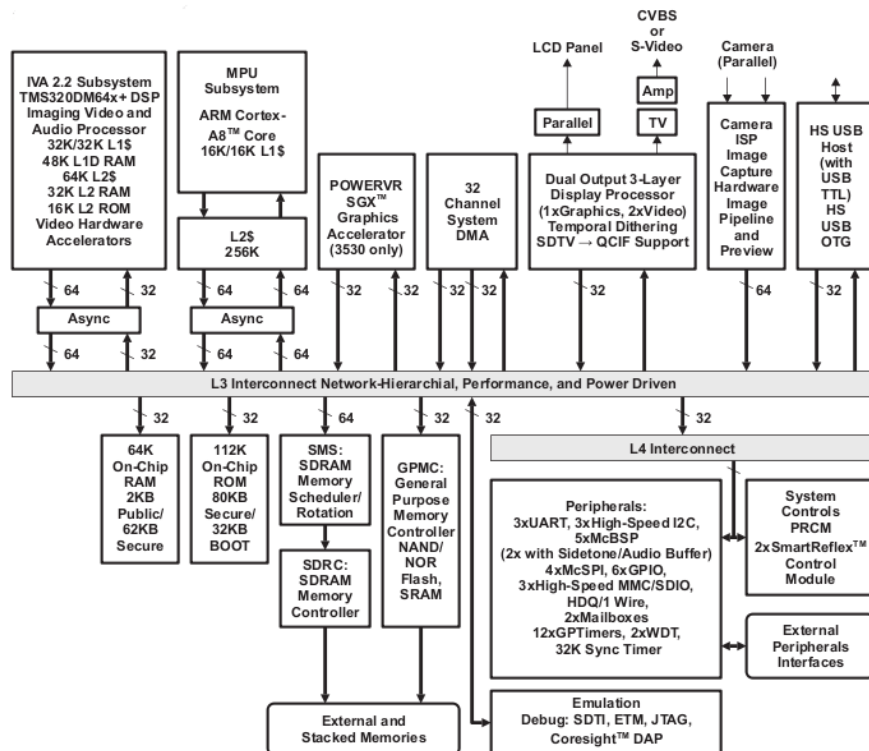


Figure 4.4.1: Overview about the Units in a OMAP-Processors

DSP (digital signal processing)

- DSP processor with 64 x 32 bit registers grouped in two banks (bank A and bank B)
- multiply add operational unit (MAC) (with a internal wordlength of 40 bits)
- barrel shifter to scale the result
- use internal memories (on the DSP-chip) to store up to 3 operands in 1 cycle
- MAC-operation and barrel shifting-operation execute in 1 cycle
- different interfaces for
 - connection to an external SDRAM
 - L2 cache system to coordinate parallel data transfers (for the transfer of both values in a multiplikation instruction)

- internal timers
- interrupt sources
- DSP is a VLIW-processor (very long instruction word)
 - wordlength 256 bit
 - more instructions are placed in one instruction word (e.g. 2 transfers 4 arithmetical instructions)
 - one VLIW-instruction are excuted in one cycle

Figure 4.4.2: Blockdiagram of the DSP-Part of the OMAP-Processor

I/O-Units

- additional to the processor (ARM-Cortex-A8) and the signal processor (DSP) many digital I/O-units implemented
 - all units are connected by two interprocessor busses (L3 and L4)
 - the communication are controlled by specific cache controllers and the according cache memories
 - all processors (internal memories) and the registers or memories of the I/O-units and the external memory are visible in the Global Memory Map
-
- L3 interprocessor communication bus connects the processors and memory subsystems
 - L4 interprocessor communication bus connects the processors with the I/O-systems
 - interprocessor communication and the communication between processors and the I/O-units are controlled by mailboxes

QUARTER	Device Name	Start Address (HEX)	End Address (HEX)	Size	Description
Q0 (1GB)	Boot space GPMC			1MB 1GB or 1GB-1MB	
	GPMC	0x0000 0000	0x3FFF FFFF	1GB	8/16 Ex RW
Q1 (1GB)	On-Chip memory			128MB	ROM/DRAM address space
	Boot ROM internal	0x4000 0000	0x4001 3FFF	80KB	Reserved for boot code Not accessible after boot
		0x4001 4000	0x4001 BFFF	32KB	32-bit Ex R
	Reserved	0x4001 C000	0x400F FFFF	912KB	Reserved
	Reserved	0x4010 0000	0x401F FFFF	1MB	Reserved
	SRAM internal	0x4020 0000	0x4020 FFFF	64KB	32-bit Ex RW
	Reserved	0x4021 0000	0x4024 FFFF	256KB	Reserved
	Reserved	0x4025 0000	0x47FF FFFF	128,704KB	Reserved
	L4 interconnects			128MB	All system peripherals
	L4-Core (L4 Wakeup)	0x4800 0000	0x48FF FFFF	16MB	
		(0x4830 0000)	(0x4833 FFFF)	(256KB)	
	L4-Per	0x4900 0000	0x490F FFFF	1MB	
	Reserved	0x4910 0000	0x4FFF FFFF	1,111MB	Reserved
	SGX			64MB	Graphic accelerator slave port
	SGX	0x5000 0000	0x5000 FFFF	64KB	Graphic accelerator slave port
	Reserved	0x5001 0000	0x53FF FFFF	65,472KB	Reserved
	L4 Emulation			64MB	Emulation
	L4-Emu	0x5400 0000	0x547F FFFF	8MB	
	Reserved	0x5480 0000	0x57FF FFFF	56MB	Reserved
	Reserved			64MB	Reserved
	Reserved	0x5800 0000	0x5BFF 0FFF	64MB	Reserved
	IVA2.2 Subsystem			64MB	IVA2.2 subsystem
	IVA2.2 Subsystem	0x5C00 0000	0x5EFF FFFF	48MB	IVA2.2 subsystem. See
	Reserved	0x5F00 0000	0x5FFF FFFF	16MB	Reserved
	Reserved			128MB	Reserved
	Reserved	0x6000 0000	0x67FF FFFF	128MB	Reserved
	L3 Interconnect			128MB	Control registers
	L3 control registers	0x6800 0000	0x68FF FFFF	16MB	
	Reserved	0x6900 0000	0x6BFF FFFF	48MB	Reserved
	SMS registers	0x6C00 0000	0x6CFF FFFF	16MB	Configuration registers SMS address space 2
	SDRC registers	0x6D00 0000	0x6DFF FFFF	16MB	Configuration registers SMS address space 3
	GPMC registers	0x6E00 0000	0x6EFF FFFF	16MB	Configuration registers GPMC address space 1
	Reserved	0x6F00 0000	0x6FFF FFFF	16MB	Reserved
	SDRC/SMS			256MB	SDRC/SMS
	SDRC/SMS virtual Address space #0	0x7000 0000	0x7FFF FFFF	256MB	SDRC-SMS virtual address space 0
Q2 (1GB)	SDRC/SMS			1GB	SDRAM main address space (SMS)
	CS0 – SDRAM [®]	0x8000 0000	0x9FFF FFFF	512MB	SDRC/SMS
Q3 (1GB)	CS1 – SDRAM [®]	0xA000 0000	0xBFFF FFFF	512MB	SDRC/SMS
	Reserved			512MB	Reserved
	Reserved	0xC000 0000	0xDFFF FFFF	512MB	Reserved for future use
	SDRC/SMS			512MB	SDRC/SMS
	SDRC/SMS virtual Address space 1	0xE000 0000	0xFFFF FFFF	512MB	SDRC-SMS virtual address space 1

Figure 4.4.3: Global Memory Map of the OMAP-Processor

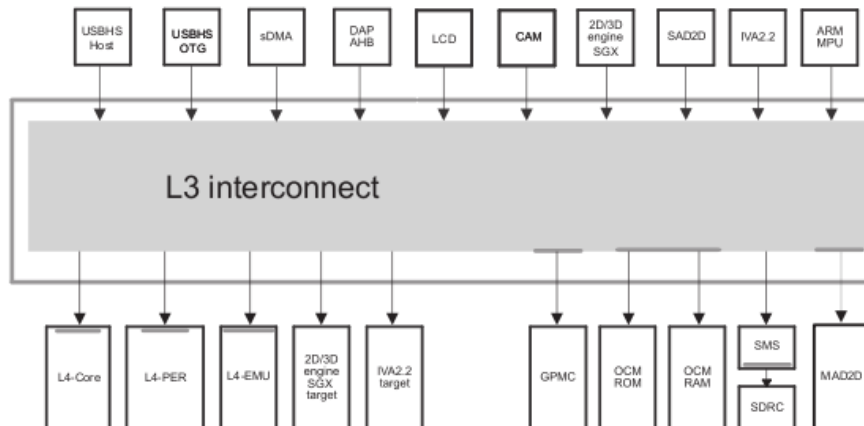


Figure 4.4.4: L3 Interprocessor Communication Bus

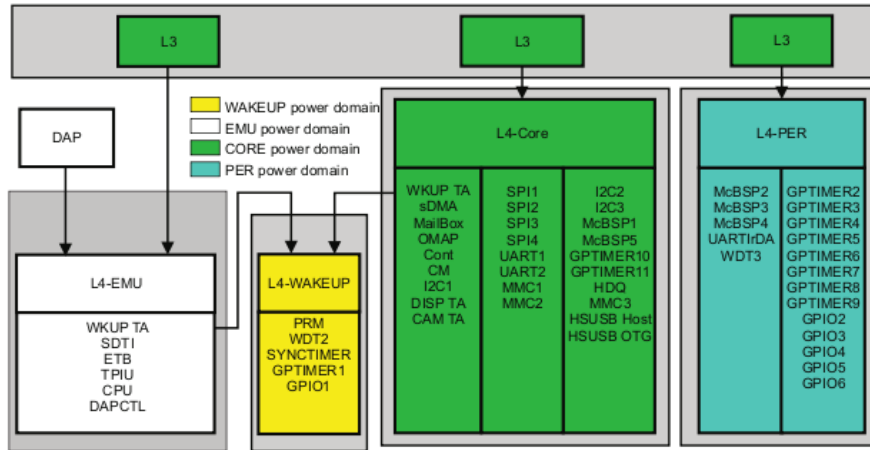


Figure 4.4.5: L4 Interprocessor Communication Bus

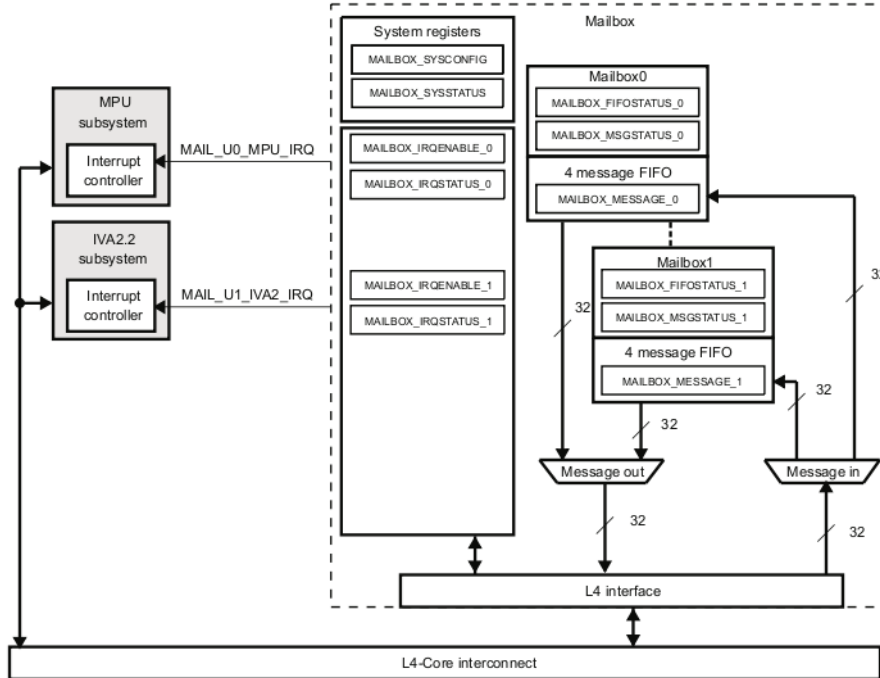


Figure 4.4.6: Communication Mailbox

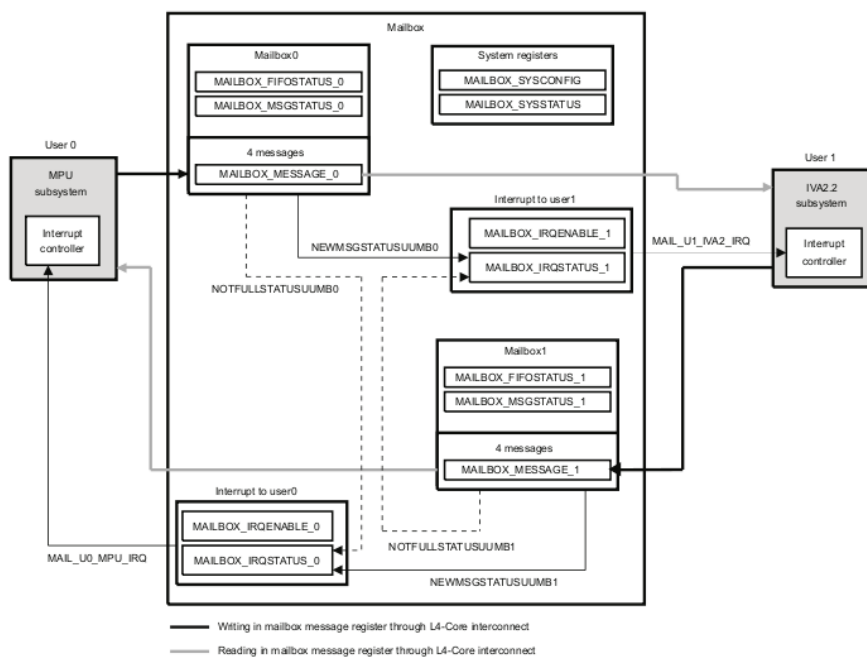


Figure 4.4.7: Example for Communications with Mailboxes

Chapter 5

Programming

5.1 Programming in Assembler

The method of programming a microcontroller is little bit different from that used for programming of microprocessors. In the case of microcontrollers, the programming is usually done using the “cross compilation” technique. In this method, all the necessary tools for compiling are installed on a PC (i.e. editor, compiler, linker etc.) and the executable file generated is then downloaded to the target system. Here, the target system (e.g. ARM-Cortex-A8) has a completely different processing core than the host PC (which employs e.g. an Intel Processor). Another method involves development on the target directly and doesn’t have a download mechanism. The target processor and the processor for the development system is the same and a loader of an available operating system loads the executable file and starts the program. A next important difference between the development of programs for microcontroller and microprocessor is the opportunity to use an operating system or not. In nearly every application which uses a microprocessor there is support from an operating system (OS). Simple input/output-operations, memory allocation and mathematical libraries would be offered through the operating system. In the case of a microcontroller you have to program all these functions.

The microcontroller ARM-Cortex-A8 is a processor just on the border between microprocessors and microcontrollers. From the point of a programmer it is a typical microprocessor, from the point of a hardware designer it is a highly sophisticated microcontroller. So in this chapter we don’t find the specific restrictions which are typical for other microcontrollers. Starting with an overview about all the registers (register model) from the different parts of this controller we will progress to assembler programming aspects. The focus is to understand the specific aspects of this controller and not to write big assembler programs. These could be better done through C-programs.

Register model

- the register model of all ARM processors first introduce the ALU register set
- the ALU registers are used for data operations and for address calculations
- a few register are only visible in the different processor mode (like user mode, system mode, supervisor mode ..)
- other processor units (like FPU, NEON) use there one register sets
- register models are important for the development of assembler programs (a C-compiler tries to hide the most aspects of registers).

- registers are part of the core (not mapped in the address space of the memory).
- register model for integer operations (core register).
 - register r0 - r12 used for data- and address-operation
 - register r13 → stack (address register for the stack)
 - register r14 → link (address register for the return address of a subroutine)
 - register r15 → pc (address register for the next instruction)
 - the access to a data-register could be done as a Byte (b), Halfword (h) (16-bit) or Word (32-bit) (default).
 - the access to an address-register only a Word is possible.

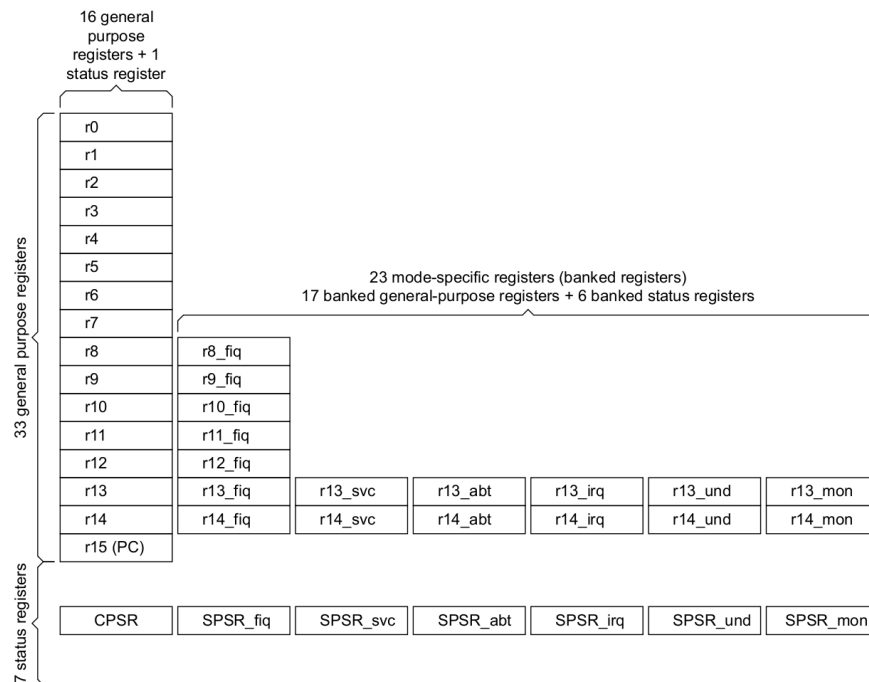


Figure 5.1.1: Integer Registers

- CPSR → Common Processor Status Register
- 32 bit register (with 6 different copies in SPSR, depends on the different modes of the processor).
- N → Negative flag (set if the most significant bit of the result is set; otherwise cleared)
- Z → Zero flag (set if the result equals zero; otherwise cleared)..
- C → Carry flag (set if a carry out of the most significant bit of the operand occurs for an addition, or if a borrow occurs in a subtraction; otherwise cleared); for unsigned numbers.
- V → Overflow flag (set if an arithmetic overflow occurs implying that the result cannot be represented in the operand size; otherwise cleared); for signed numbers.
- Q → Sticky Overflow (not changed after every arithmetic operations; explicitly changed with a MSR instruction)
- flags could be changed after all instructions (including transfer instructions).

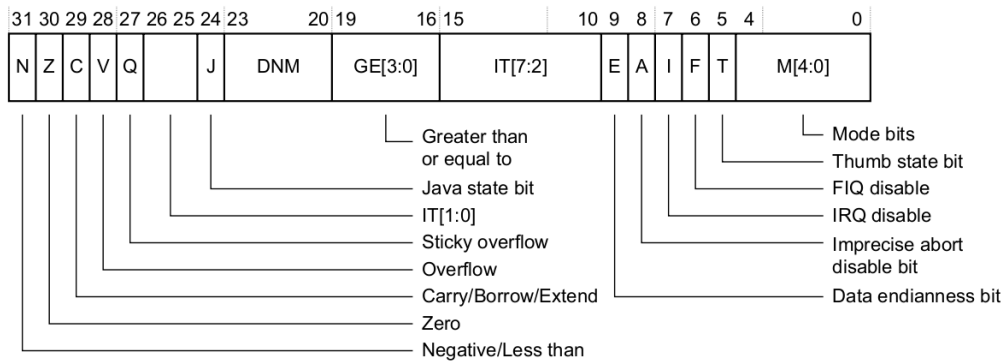


Figure 5.1.2: Processor Status Word

- flags are used in conditional branch instructions (the instruction which affected a flag and the appropriate branch instruction should follow each other).
- flags could also be used in conditional transfer instructions (typical feature of RISC-Processors)

Addressing modes

- addressing modes could be separated in five groups
- **Register addressing**
 - name of the register coded in the instruction
 - LDR r0,r1 ;load the contents from r1 to r0 (32 bit, default)
 - LDRB r2,r0 ;load the least significant byte from r0 to r2
;(least significant position)
 - LDRH r3,r2 ;load the least significant halfword from r2 to r3
;(same position in r3)
 - SDR r0,r1 ;store the contents from r0 to r1 (32 bit, default)
 - SDRB r2,r0 ;store the least significant byte from r2 to r0
;(least significant position)
 - SDRH r3,r2 ;store the least significant halfword from r3 to r2 (same position in r2)
 - MOV r0,r1 ;transfer contents from r0 to r1 (always 32 bit)
 - MOVS r0,r1 ;transfer contents from r0 to r1 (always 32 bit)
;and set the Z- and N-flag
 - RISC-like register instructions
 - LDR r15,r14 ;load PC with the contents of the link register
;→ same as RET-instruction
 - MSR r4,CPSR ;move the contents of CPSR to r4 (32 bit)
 - MRS CPSR_f,r4 ;move the flag-field
;(5 most significant bits → N-, Z-, C-, V-, Q-flag)
- **Immediate addressing**
 - instruction contains the data
 - the immediate value is signed (-4095 to +4096 for LDR-instruction)

- this type of addressing is useful for source addressing only


```
LDR r0,#0x100    ;transfer the value 100 (hex)to r0 (32 bit)
LDR r0,#0x-100   ;transfer the value -100 (hex)to r0 (32 bit)

LDR r0,=#0x2000  ;transfer the value 2000 (hex)
                  ;to r0 (pseudo-instruction)
                  ;LDR r0,=#0x2000 → LDR r0,[pc,offset_to_litpool]
                  ;...
                  ;litpool DCD 0x2000
```

- **Direct addressing**

- instruction contains the address of a data
- not directly implemented, an direct or absolut address could be given with a pseudo-instruction


```
LDR r0,=label    ;load the register r0 with address of label
                  ;LDR r0=label → LDR r0,[pc,offset_to_litpool]
                  ;...
                  ;litpool DCD label;
```

- **Indirect addressing**

- use one of the registers r0 - r12.
- address registers r13 - r15 are special address registers.
- address registers are always 32-bit registers.
- indirect addressing modes operate in different versions.

- **Indirect addressing standard**

```
LDR r1,[r0]    ;load the register r1 with contents
                ;of address where r0 points to
SDR r2,[r0]    ;store the register r2 to the address
                ;where r0 points to
```

- **Indirect addressing with index and offset**

- address calculation with an address register and an additional register and a offset
- the additional register and the offset are signed.


```
LDR r0,[r1,r2,0x20]    ;load register r0 from the address given by
                        ;r1 + r2 + 0x20
SDR r0,[r1,r2,-0x20]   ;store register r0 at the address given by
                        ;r1 - r2 - 0x20
```

- **Indirect addressing with index,offset and update**

- address calculation with an address register and an additional register and a offset
- the additional register and the offset are signed.
- after the transfer the first mentioned address register are updated


```
LDR r0,[r1,r2,0x20]!   ;load register r0 from the address given by
                        ;r1 + r2 + 0x20, r1 updated
SDR r0,[r1,r2,-0x20]! ;store register r0 at the address given by
                        ;r1 - r2 - 0x20, r1 updated
```

- **post increment / pre decrement**

- load and store registers from stack

LDMFA r13!,{r0 - r12} ;load stackframe (r13) to registers r0 - r12
 ;full ascending stack
 ;increment register r13 and store
 ;the last position of the stack in r13
 ;(pop-operation on Intel processors)

STMFD r13!,{r0 - r12}, ;generate a stackframe r0 - r12
 ;full descending stack
 ;save registers r0 - r12 on the stack
 (push-operation on Intel processors)

STMFD r13!,{r4 - r12,LR} ;push registers r4 - r12 to the stack
 ;and place the return address (LR)
 ;on the stack

LDMFA r13!,{r4 - r12,PC} pop registers r 4 - r12 from the stack
 ;and jump back to the calling routine

- post increment and predecrement addressing mode work which other register as the stack also

LDMIA / STMIA... ;increment after transfer
 LDMIB / STMIB... ;increment before transfer
 LMDMA / STMDA... ;decrement after transfer
 LDMDB / STMDB... ;decrement before transfer

- **Relative addressing**

- the program counter (PC) works as an address register.
- this addressing mode has to be selected for position independent programs.
- only useful with an offset.
- relative addressing modes operate in different versions (index register).

- **Relative addressing with an offset**

- the offset is signed value.

LDR r0,[PC,0x20] ;load the register r0 with contents
 ;of address where (PC + 20) points to

- easier use of relative the relative addressing mode with the pseudo instruction ADR

ADR r0,label ;load the register r0 with the
 ;relative address of label

- Classification of the instruction set

- transfer instructions
- integer arithmetic instructions
- floating-point arithmetic instructions
- logical instructions
- bit manipulation instructions

- control instructions
- other

5.2 Programming in C

Specification

- Cross compilation:
- use of a special toolchain to build programs for ARM-Cortex-A8
 - compiler arm-none-linux-gnueabi-gcc.
 - version for the “standard”-gcc (generates output for the ARM-Cortex-processor)
 - in the case of cross compilation there is mostly support from an operating system. This means it makes no sense to use the MMU features of the processor.
 - the output of the compiler depends on the position which has to be specified in the linker.
 - if the linker doesn’t generate a position depend output a additional tool necessary
 - objdump (mostly part of a build package)
 - this output format is indicated by the endings “.bin”.
 - to download these programs in the target convert the “.bin”-programs to a “.srec”-format or a “.ihex”-format if using RS-232-Interface.
- Development in the target system
 - use of the same compiler (arm-none-linux-gnueabi-gcc) as in the case with cross compilation.
 - the development has to be done in the target system (longer development cycles).
 - the executable file has to be generated position independent (controlled by a special linker command).
 - in the case ARM-Cortex-processor the MMU is active.
 - the output of the linker is build as .elf-file (elf - executable and linkable format) and could directly loaded from the operating system.
- Development with the same operating system in the target and in the cross-system
 - use of Linux as operating system on the target.
 - implementation of an NFS-server (network file system) on the cross-system.
 - mount the “working directory” of the cross-system in the target and direct load and execution is possible.
 - shorter development cycles as in the cross-compilation.
 - uses the advantages of an operating system.

Special features

- no restriction in the use of data types.
- the type float and double also supported (the basic arithmetic steps like add, sub, mul, and div directly supported from the FPU, more complex floating point arithmetic expressions like $\sin(x)$, $\cos(x)$, x^y supported from a special library).
- access to registers (and memory) could be done in two different methods:

- cross-compilation without the use on the MMU (independent of the OS, the memory here is managed by the compiler).
- set a pointer to the register (type of pointer depends on the register).

```
#define I/O_Peripherals 0xF2000000          /* Base address of */
                                           /* all I/O Address */

(*unsigned int *) (I/O_Peripherals) = 0x1234; /* Set Register */
```
- program runs in the “user-mode” of the processor.
- development in the target with the use of the MMU.
- program have to be developed in the “kernel-mode”.

Parameter Passing Conventions

- C-Programs and Assembler-Programs could be combined without any restrictions.
- this feature is based on a similar concept about passing parameters (C-parameter passing conventions, AAPCS ARM Architecture Procedure Call Standard).
- specification of the return value of a function (always passed in registers r0, f0, s0, d0 as follows)
 - Byte (char, unsigned char) → r0[7..0]
 - Halfword (short, unsigned short) → r0[15..0]
 - Word (int, unsigned int, pointers) → r0
 - Float → f0, s0
 - Double → d0
- the calling parameter are passed on the registers r0 - r3 and the stack (from left to right)
- Register names and there use

Register	Synonym	Use
r15	PC	Program counter
r14	LR	Link register
r13	SP	Stack pointer
r12	IP	Intra-procedure-call
r11	v8	Variable register
r10	v7	Variable register
r9	v6	Variable register
r8	v5	Variable register
r7	v4	Variable register
r6	v3	Variable register
r5	v2	Variable register
r4	v1	Variable Register
r3	a4	Argument register 4
r2	a3	Argument register 3
r1	a2	Argument register 2
r0	a1	Argument register 1

- example `int myfunc(int i, int j, int k, int l, int m ); /* myfunc = i + j + k + l + m */`

Preparation of the Stack

SP →

int m

Transfer of the parameters with registers

r3	int l
r2	int k
r1	int j
r0	int i

- Program code
- ```
;int myfunc(int i, int j, int k, int l, int m)
GLOBAL myfunc
myfunc: ADD r0,r0,r1 ;r0 = r0 + r1
 ADD r0,r0,r2 ;r0 = r0 + r2
 ADD r0,r0,r3 ;r0 = r0 + r3
 LDR r3, [sp] ;load int m from stack
 ADD r0,r0,r3 ;result in r0 (return value)
 ADD sp,sp,#4 ;remove int m from stack
 MOV pc,lr ;back to the calling function

END
```
- if there are additional local variable necessary, they could placed on the stack.
- before leave every reservation on the stack (local variable) have to be removed

## 5.3 Python for Embedded Systems

### Python for Embedded

- A very short introduction

### Agenda

- Introduction
- Running Python
- Micropython
- Python Programming
- Data types
- Control flows
- Classes, functions, modules
- Hands-on Exercise

### What is python?

- Object oriented language

- Interpreted language
- Supports dynamic data type
- Independent from platforms
- Focused on development time
- Simple and easy grammar
- High-level internal object data types
- Automatic memory management (Garbage Collected)
- It is free (open source)!

## Timeline

- Python born, name picked - Dec 1989
  - By Guido van Rossum, now at GOOGLE
- First public release (USENET) - Feb 1991
- python.org website - 1996 or 1997
- 2.0 released - 2000
- Python Software Foundation - 2001
- Current version:
  - 2.7 (frozen 2008)
  - 3.10 (parallel versions)

## Language properties

- Everything is an object
- Modules, classes, functions
- Exception handling
- Dynamic typing, polymorphism
- Static scoping
- Operator overloading
- Indentation for block structure

## High-level data types

- Numbers: int, long, float, complex
- Strings: immutable
- Lists and dictionaries: containers
- Other types for e.g. binary data, regular expressions, introspection
- Extension modules can define new "built-in" data types

## Why learn python?

- Fun-to-use "Scripting language"
- Object-oriented
  - Highly educational
- Very easy to learn
- Powerful, scalable, easy to maintain
  - high productivity
  - Lots of libraries
- Glue language
  - Interactive front-end for FORTRAN/C/C++ code

### Why learn python? (cont.)

- Reduce development time
- Reduce code length
- Easy to learn and use as developers
- Easy to understand codes
- Easy to do team projects
- Easy to extend to other languages

### Where to use python?

- System management (i.e., scripting)
- Graphic User Interface (GUI)
- Internet programming
- Database (DB) programming
- Text data processing
- Distributed processing
- Numerical operations
- Graphics
- And so on...

### Python vs. Perl

- Easier to learn
  - important for occasional users
- More readable code
  - improved code maintenance
- Fewer "magical" side effects
- More "safety" guarantees

- Better Java integration
- Less Unix bias

## Python vs. Java

- Code 5-10 times more concise
- Dynamic typing
- Much quicker development
  - no compilation phase
  - less typing
- Yes, it runs slower
  - but development is so much faster!
- Similar (but more so) for C/C++
- Use Python with Java: JPython!

## Running Python - The Interpreter

- Python is an interpreted language and not a compiled one, although compilation is a step.
- Python code, written in a .py file is first compiled to what is called bytecode.
- Stored with a .pyc or .pyo format
- bytecode is a low-level set of instructions that can be executed by an interpreter.
- mostly at /usr/local/bin/pythonx.x.
- Instead of executing the instructions on CPU, bytecode instructions are executed on a Virtual Machine.

## Why to interpret?

- Platform-independent
  - As long as the Python bytecode and the Virtual Machine have the same version, Python bytecode can be executed on any platform (Windows, MacOS, etc).
- Dynamic typing
  - In static-typed languages like C++, you have to declare the variable type and any discrepancy like adding a string and an integer is checked during compile time.
  - In strongly typed languages like Python, it is the job of the interpreter to check the validity of the variable types and operations performed.

## Why not to interpret?

- Dynamic typing provides a lot of freedom, but simultaneously it makes your code risky and sometimes difficult to debug.
- Slow because the interpreter has to do extra work to have the bytecode instruction translated into a form that can be executed on the machine.
- A StackOverflow post makes it easier to understand using an analogy
  - If you can talk in your native language to someone, that would generally work faster than having an interpreter having to translate your language into some other language for the listener to understand.

## Garbage Collection

- Many Languages use manual memory management. When variables are no longer in use or referenced, they need to be cleaned from the memory.
- Garbage Collector automatically frees up space without you doing anything.
- The memory management works in two ways -
  - In a simplified way, it keeps track of the number of references to an object. When that number goes down to zero, it deletes that object. This is called reference counting. This cannot be disabled in Python.
  - In cases where object references itself or two objects refer each other, a process called generation garbage collection helps. This is something traditional reference counting cannot take care of.

## PyCache

- Folder named `__pycache__` : created automatically.
  - `/folder`
  - `- __pycache__`
  - `- preprocess.cpython-36.pyc`
  - `- preprocess.py`
- Filename is the same as the one outside `__pycache__` folder.
  - The `.pyc` extension tells us that the file contains bytecode for `preprocess.py`.
  - The names `cpython` denotes the type of interpreter. CPython means that the interpreter was implemented in C language. JPython is a Python interpreter implemented in Java.

## PyCache

- This slightly increases the speed of the Python program.
- Unless you change your Python code, recompilation to bytecode is avoided, thereby saving time.

## Running Python Interactively

- Start python by typing "python"
  - /usr/bin/python
- $\hat{D}$  (control-D) exits
  - % python
  - $\hat{D}$
  - %
- Comments start with #

```

1 \>\>\> 2+2 #Comment on the same line as text
2 4
3 \>\>\> 7/3 #Numbers are integers by default
4 2
5 \>\>\> x = y = z = 0 #Multiple assigns at once
6 \>\>\> z
7 0

```

## Running Python Programs

- In general
  - % python ./myprogram.py
- Can also create executable scripts
  - Compose the code in an editor like vi/emacs
    - % vi ./myprogram.py# Python scripts with the suffix .py.
  - Then you can just type the script name to execute
    - % python ./myprogram.py
  - The first line of the program tells the OS how to execute it:
    - #! /afs/isis/pkg/isis/bin/python
  - Make the file executable:
    - % chmod +x ./myprogram.py
  - Then you can just type the script name to execute
    - % ./myprogram.py

## Running Python Programs Interactively

- Suppose the file script.py contains the following lines:
  - print 'Hello world'
  - x = [0,1,2]
- Let's run this script in each of the ways described on the last slide:
  - python -i script.py
  - Hello world
  - x

- `[0,1,2]`
- 
- `$ python`
- `execfile('script.py')`
- `x`
- `[0,1,2]`

## Running Python Programs Interactively

- Suppose the file `script.py` contains the following lines:
- `print 'Hello world'`
- `x = [0,1,2]`
- Let's run this script in each of the ways described on the last slide:
- `python`
- `import script# DO NOT add the .py suffix. Script is a module here`
- `x`
- Traceback (most recent call last):
- File "`stdin`", line 1, in ?
- `NameError: name 'x' is not defined`
- `script.x# to make use of x, we need to let Python know which #module it came from, i.e. give Python its context`
- `[0,1,2]`

## Running Python Programs Interactively

- `# Pretend that script.py contains multiple stored quantities. To promote x(and only x) to the top level context, type the following:`
- `$ python`
- `from script import x`
- `Hello world`
- `x`
- `[0,1,2]`
- 
- `# To promote all quantities in script.py to the top level context, type`
- `from script import *` into the interpreter. Of course, if that's what you want, you might as well type `python -i script.py` into the terminal.
- `from script import *`

## File naming conventions

- python files usually end with the suffix `.py`

- but executable files usually don't have the .py extension
- modules (later) should always have the .py extension

## Comments

- Start with# and go to end of line
- What about C, C++ style comments?
  - NOT supported!

## Micropython

### What is?

- "Micro Python was successfully funded via a Kickstarter campaign. The software is available to the public under the MIT open source license. "
- "Micro Python is a lean and fast implementation of the Python 3 programming language that is optimised to run on a microcontroller."
- "The Micro Python board is a small electronic circuit board that runs Micro Python on the bare metal, and gives you a low-level Python operating system that can be used to control all kinds of electronic projects." [micropython.org](http://micropython.org)

### What is exactly?

- Full implementation of Python 3 grammar. Cpython 3.4 (no batteries)
- Implements lexer, parse, compiler, vm and runtime.
- Execute files or REPL (Read-Evaluate-Print-Loop).
- Python code is compiled to byte code that run in vm.
- Storing objects in efficient ways.
- Decorator can be compiled to native code. 2 time faster than byte code.
- Memory is managed using a simple and fast mark-sweep garbage collector.

### What is more?

- Micro Python as a "mini Python operating system"
  - Checks for a local filesystem on the flash.
  - Checks for /boot.py.
  - Runs /boot.py to configure USB and other low-level parameters that can't be changed without a reset.
  - The main Python script is run if it exists. Default is /src/main.py.

- If the main script doesn't exist, or exits, REPL (read-evaluate-print-loop) mode is entered.

## Batteries included?

- I am the terror that flaps in the night, I am the batteries that are not included. I, am Darkwing Duck
- Python standard libraries
  - cmath - mathematical functions for complex numbers
  - gc - control the garbage collector
  - math - mathematical functions
  - os - basic "operating system" services
  - select - wait for events on a set of streams
  - struct - pack and unpack primitive data types
  - sys - system specific functions
  - time - time related functions
    - additional libs in <https://github.com/micropython/micropython-lib>

## More batteries

- Python standard libraries "micro-ified"
  - ubinascii - binary/ASCII conversions
  - uhashlib - hashing algorithm
  - uheapq - heap queue algorithm
  - ujson - JSON encoding and decoding
  - ure - regular expressions
  - usocket - socket module
  - uzlib - zlib decompression

## Jupyter

- The Jupyter Notebook can also be used for both regular and micropython.
- Circuitpython is an adafruit specific version to be used for physical computing.

## Programming Python

## Python Syntax

- Much of it is similar to C syntax
- Exceptions:

- missing operators: ++, −
- no curly brackets, {}, for blocks; uses whitespace!!!
- different keywords
- lots of extra features
- no type declarations!

## Simple data types

- Numbers
  - Integer, floating-point, complex!
  -
- Strings
  - characters are strings of length 1
  -
- Booleans are False or True

## Numbers

- The usual notations and operators

```

1 12, 3.14, 0xFF, 0377, (-1+2)*3/4**5, abs(x), 0<x<=5
2
3 C-style shifting & masking
4
5 1<<16, x&&0xff, x|1, ~x, x^y
6
7 Integer division truncates :-(
8
9 1/2 -> 0 # float(1)/2 -> 0.5
10
11 Long (arbitrary precision), complex
12
13 2L**100 -> 1267650600228229401496703205376L
14 1j**2 -> (-1+0j)

```

## Strings and formatting

```

1 i = 10
2 d = 3.1415926
3 s = "I am a string!"
4 print "%d\t%f\t%s" % (i, d, s)
5 print "newline\n"
6 print "no newline"

```

## Variables

- No need to declare
- Need to assign (initialize)
  - use of uninitialized variable raises exception
- Not typed
  - if friendly: greeting = "hello world"
  - else: greeting = 12\*\*2
  - print greeting
- Everything is a variable:
  - functions, modules, classes

## Reference semantics

- Assignment manipulates references
  - `x = y` does not make a copy of `y`
  - `x = y` makes `x` reference the object `y` references
- Very useful; but beware!
- Example:

```
1 \>\>\> a = [1, 2, 3]; b = a
2 \>\>\> a.append(4); print b
3 [1, 2, 3, 4]
```

## Simple data types: operators

```
1 + - * / % (like C)
2 += -= etc. (no ++ or --)
3 Assignment using =
4
5 but semantics are different!
6 a = 1
7 a = "foo" # OK
8
9 Can also use + to concatenate strings
```

## Strings

```

1 "hello"+"world" "helloworld" # concatenation
2 "hello"*3 "hellohellohello" # repetition
3 "hello"[0] "h" # indexing
4 "hello"[-1] "o" # (from end)
5 "hello"[1:4] "ell" # slicing
6 len("hello") 5 # size
7 "hello" < "jello" 1 # comparison
8 "e" in "hello" 1 # search
9 New line: "escapes: \n "
10 Line continuation: triple quotes '''
11 Quotes: 'single quotes ', "raw strings "

```

## Simple Data Types

- Triple quotes useful for multi-line strings
- `s = """ a long... string with "quotes" or anything else """`  
`s' a long string with "quotes" or anything else'`  
`len(s)`45

## Methods in string

- `upper()`
- `lower()`
- `capitalize()`
- `count(s)`
- `find(s)`
- `rfind(s)`
- `index(s)`
- `strip()`, `lstrip()`, `rstrip()`
- `replace(a, b)`
- `expandtabs()`
- `split()`
- `join()`
- `center()`, `ljust()`, `rjust()`

## Compound Data Type: List

- List:
  - A container that holds a number of other objects, in a given order
  - Defined in square brackets

```

1 a = [1, 2, 3, 4, 5]
2 print a[1] # number 2
3 some_list = []
4 some_list.append("foo")
5 some_list.append(12)
6 print len(some_list) # 2

```

## List

```

1 a = [99, "bottles of beer", ["on", "the", "wall"]]
2
3 Flexible arrays, not Lisp-like linked lists
4 Same operators as for strings
5
6 a+b, a*3, a[0], a[-1], a[1:], len(a)
7
8 Item and slice assignment
9
10 a[0] = 98
11 a[1:2] = ["bottles", "of", "beer"]
12
13 -> [98, "bottles", "of", "beer", ["on", "the", "wall"]]
14
15 del a[-1] # -> [98, "bottles", "of", "beer"]

```

## More list operations

```

1 \>\>\> a = range(5) # [0,1,2,3,4]
2 \>\>\> a.append(5) # [0,1,2,3,4,5]
3 \>\>\> a.pop() # [0,1,2,3,4]
4 5
5 \>\>\> a.insert(0, 5.5) # [5.5,0,1,2,3,4]
6 \>\>\> a.pop(0) # [0,1,2,3,4]
7 5.5
8 \>\>\> a.reverse() # [4,3,2,1,0]
9 \>\>\> a.sort() # [0,1,2,3,4]

```

## Operations in List

- Indexing e.g., `L[i]`
- Slicing e.g., `L[1:5]`
- Concatenation e.g., `L + L`
- Repetition e.g., `L * 5`
- Membership test e.g., `'a' in L`

- Length e.g., len(L)
- append
- insert
- index
- count
- sort
- reverse
- remove
- pop
- extend

## Nested List

- List in a list
- E.g.,
  - `s = [1,2,3]`
  - `t = ['begin', s, 'end']`
  - `t`
  - `['begin', [1, 2, 3], 'end']`
  - `t[1][1]`
  - `2`

## Dictionaries

- Dictionaries: curly brackets
- What is dictionary?
- Refer value through key; "associative arrays"
- Like an array indexed by a string
- An unordered set of key: value pairs
- Values of any type; keys of almost any type

```

1 {"name": "Guido", "age": 43, ("hello", "world"): 1, 42: "yes", "flag": ["
 red", "white", "blue"]}
2 d = { "foo" : 1, "bar" : 2 }
3 print d["bar"] # 2
4 some_dict = {}
5 some_dict["foo"] = "yow!"
6 print some_dict.keys() # ["foo"]

```

## Methods in Dictionary

- `keys()`
- `values()`
- `items()`
- `has_key(key)`
- `clear()`
- `copy()`
- `get(key[,x])`
- `setdefault(key[,x])`
- `update(D)`
- `popitem()`

## Dictionary details

- Keys must be immutable:
  - numbers, strings, tuples of immutables
    - these cannot be changed after creation
  - reason is hashing (fast lookup technique)
  - not lists or other dictionaries
    - these types of objects can be changed "in place"
  - no restrictions on values
- Keys will be listed in arbitrary order
  - again, because of hashing

## Tuples

- What is a tuple?
  - A tuple is an ordered collection which cannot be modified once it has been created.
  - In other words, it's a special array, a read-only array.
- How to make a tuple? In round brackets
  - E.g.,
  - `t = ()`
  - `t = (1, 2, 3)`
  - `t = (1, )`
  - `t = 1,`
  - `a = (1, 2, 3, 4, 5)`
  - `print a[1] # 2`

## Operations in Tuple

- Indexing e.g., `T[i]`

- Slicing e.g., `T[1:5]`
- Concatenation e.g., `T + T`
- Repetition e.g., `T * 5`
- Membership test e.g., `'a' in T`
- Length e.g., `len(T)`

## List vs. Tuple

- What are common characteristics?
  - Both store arbitrary data objects
  - Both are of sequence data type
- What are differences?
  - Tuple doesn't allow modification
  - Tuple doesn't have methods
  - Tuple supports format strings
  - Tuple supports variable length parameter in function call.
  - Tuples slightly faster

## Data Type Wrap Up

- Integers: `2323`, `3234L`
- Floating Point: `32.3`, `3.1E2`
- Complex: `3 + 2j`, `1j`
- Lists: `l = [ 1,2,3]`
- Tuples: `t = (1,2,3)`
- Dictionaries: `d = 'hello' : 'there', 2 : 15`

## Data Type Wrap Up

- Lists, Tuples, and Dictionaries can store any type (including other lists, tuples, and dictionaries!)
- Only lists and dictionaries are mutable
- All variables are references

## Input

- The `raw_input(string)` method returns a line of user input as a string
- The parameter is used as a prompt
- The string can be converted by using the conversion methods `int(string)`, `float(string)`, etc.

## File I/O

- `f = file("foo", "r")`
- `line = f.readline()`
- `print line,`
- `f.close()`
- `# Can use sys.stdin as input;`
- `# Can use sys.stdout as output.`

## Files: Input

|                                        |                                 |
|----------------------------------------|---------------------------------|
| <code>input = open('data', 'r')</code> | Open the file for input         |
| <code>S = input.read()</code>          | Read whole file into one String |
| <code>S = input.read(N)</code>         | Reads N bytes (N $\geq$ 1)      |
| <code>L = input.readlines()</code>     | Returns a list of line strings  |

## Files: Output

|                                         |                                              |
|-----------------------------------------|----------------------------------------------|
| <code>output = open('data', 'w')</code> | Open the file for writing                    |
| <code>output.write(S)</code>            | Writes the string S to file                  |
| <code>output.writelines(L)</code>       | Writes each of the strings in list L to file |
| <code>output.close()</code>             | Manual close                                 |

## `open()` and `file()`

- These are identical:
- `f = open(filename, "r")`
- `f = file(filename, "r")`
- The `open()` version is older
- The `file()` version is the recommended way to open a file now
  - uses object constructor syntax (next lecture)

## OOP Terminology

- class – a template for building objects
- instance – an object created from the template (an instance of the class)
- method – a function that is part of the object and acts on instances directly
- constructor – special "method" that creates new instances

## Objects

- Objects:
  - What is an object?
    - data structure, and
    - functions (methods) that operate on it
  - class thingy:
    - # Definition of the class here, next slide
    - t = thingy()
    - t.method()
    - print t.field
  - Built-in data structures (lists, dictionaries) are also objects
    - though internal representation is different

## Defining a class

- class Thingy:
  - """This class stores an arbitrary object."""
  - def \_\_init\_\_(self, value):
    - """Initialize a Thingy."""
    - self.value = value
  - def showme(self):
    - """Print this object to stdout."""
    - print "value = %s" % self.value
- constructor
- method

## Using a class (1)

- t = Thingy(10) # calls \_\_init\_\_ method
- t.showme() # prints "value = 10"
- t is an instance of class Thingy
- showme is a method of class Thingy
- \_\_init\_\_ is the constructor method of class Thingy
  - when a Thingy is created, the \_\_init\_\_ method is called
- Methods starting and ending with \_\_ are "special" methods

## Using a class (2)

- print t.value # prints "10"
- value is a field of class Thingy
- t.value = 20 # change the field value
- print t.value # prints "20"

## "Special" methods

- All start and end with `--` (two underscores)
- Most are used to emulate functionality of built-in types in user-defined classes
- e.g. operator overloading
  - `--add--`, `--sub--`, `--mult--`, ...
  - see python docs for more information

## Control flow (1)

```

1 if, if/else, if/elif/else
2 if a == 0:
3 print "zero!"
4 elif a < 0:
5 print "negative!"
6 else:
7 print "positive!"
8
9 Notes:
10
11 blocks delimited by indentation!
12 colon (:) used at end of lines containing control flow keywords

```

## Control flow (3)

```

1 while loops
2
3 a = 10
4 while a > 0:
5 print a
6 a -= 1

```

## Control flow (4)

```

1 for loops
2
3 for a in range(10):
4 print a
5
6 really a "foreach" loop

```

## Control flow (5)

```

1 Common for loop idiom:
2

```

```
3 a = [3, 1, 4, 1, 5, 9]
4 for i in range(len(a)):
5 print a[i]
```

## Control flow (6)

```
1 Common while loop idiom:
2 f = open(filename, "r")
3 while True:
4 line = f.readline()
5 if not line:
6 break
7 # do something with line
```

## Control flow (7): odds & ends

```
1 continue statement like in C
2 pass keyword:
3 if a == 0:
4 pass # do nothing
5 else:
6 # whatever
```

## Defining functions

```
1 def foo(x):
2 y = 10 * x + 2
3 return y
4 All variables are local unless specified as global
5 Arguments passed by value
```

## Executing functions

```
1 def foo(x):
2 y = 10 * x + 2
3 return y
4
5 print foo(10) # 102
```

## Why use modules?

- Code reuse
  - Routines can be called multiple times within a program
  - Routines can be used from multiple programs
- Namespace partitioning
  - Group data together with functions used for that data
- Implementing shared services or data
  - Can provide global data structure that is accessed by multiple subprograms

## Modules

- Modules are functions and variables defined in separate files
- Items are imported using from or import
  - from module import function
  - function()
  - 
  - import module
  - module.function()
- Modules are namespaces
  - Can be used to organize variable names, i.e.
    - `atom.position = atom.position - molecule.position`

## Modules

- Access other code by importing modules
  - `import math`
  - `print math.sqrt(2.0)`
- or:
  - `from math import sqrt`
  - `print sqrt(2.0)`

## Modules

- or:
  - `from math import *`
  - `print sqrt(2.0)`
- Can import multiple modules on one line:

- import sys, string, math
- Only one "from x import y" per line

## Example: NumPy Modules

- <http://numpy.scipy.org/>
- NumPy has many of the features of Matlab, in a free, multiplatform program. It also allows you to do intensive computing operations in a simple way
- Numeric Module: Array Constructors
  - ones, zeros, identity
  - arange
- LinearAlgebra Module: Solvers
  - Singular Value Decomposition
  - Eigenvalue, Eigenvector
  - Inverse
  - Determinant
  - Linear System Solver

## Arrays and Constructors

```

1 \>\>\> a = ones((3,3),float)
2 \>\>\> print a
3 [[1., 1., 1.],
4 [1., 1., 1.],
5 [1., 1., 1.]]
6 \>\>\> b = zeros((3,3),float)
7 \>\>\> b = b + 2.*identity(3) # "+" is overloaded
8 \>\>\> c = a + b
9 \>\>\> print c
10 [[3., 1., 1.],
11 [1., 3., 1.],
12 [1., 1., 3.]]

```

## Overloaded operators

```

1 \>\>\> b = 2.*ones((2,2),float) #overloaded
2 \>\>\> print b
3 [[2.,2.],
4 [2.,2.]]
5 \>\>\> b = b+1 # Addition of a scalar is
6 \>\>\> print b # element-by-element
7 [[3.,3.],
8 [3.,3.]]
9 \>\>\> c = 2.*b # Multiplication by a scalar is
10 \>\>\> print c # element-by-element
11 [[6.,6.],

```

## Array functions

```

1 \>\>\> from LinearAlgebra import *
2 \>\>\> a = zeros((3,3),float) + 2.*identity(3)
3 \>\>\> print inverse(a)
4 [[0.5, 0., 0.],
5 [0., 0.5, 0.],
6 [0., 0., 0.5]]
7 \>\>\> print determinant(inverse(a))
8 0.125
9 \>\>\> print diagonal(a)
10 [0.5,0.5,0.5]
11 \>\>\> print diagonal(a,1)
12 [0.,0.]
13
14 transpose(a), argsort(), dot()

```

## Eigenvalues

```

1 \>\>\> from LinearAlgebra import *
2 \>\>\> val = eigenvalues(c)
3 \>\>\> val, vec = eigenvectors(c)
4 \>\>\> print val
5 [1., 4., 1.]
6 \>\>\> print vec
7 [[0.816, -0.408, -0.408],
8 [0.575, 0.577, 0.577],
9 [-0.324, -0.487, 0.811]]
10 also solve_linear_equations, singular_value_decomposition, etc.

```

## Least Squares Fitting

```

1 Part of Hinsens's Scientific Python module
2
3 \>\>\> from LeastSquares import *
4 \>\>\> def func(params,x): # y=ax^2+bx+c
5 return params[0]*x*x + params[1]*x +
6 params[2]
7 \>\>\> data = []
8 \>\>\> for i in range(10):
9 data.append((i,i*i))
10 \>\>\> guess = (3,2,1)
11 \>\>\> fit_params, fit_error =
12 leastSquaresFit(func,guess,data)
13 \>\>\> print fit_params

```

```
14 [1.00,0.000,0.00]
```

## FFT

```
1 \>\>\> from FFT import *
2 \>\>\> data = array((1,0,1,0,1,0,1,0))
3 \>\>\> print fft(data).real
4 [4., 0., 0., 0., 4., 0., 0., 0.]]
5
6 Also note that the FFTW package ("fastest Fourier transform in the West
7 ") has a python wrapper. See notes at the end
8 Python Standard Libraries/Modules:
9
10 http://docs.python.org/library/
11 http://its2.unc.edu/dci/dci_components/shared_apps/packages/
 python_packages.html
12 http://pypi.python.org/pypi/
```

## Command-line arguments

```
1 import sys
2 print len(sys.argv) # NOT argc
3 # Print all arguments:
4 print sys.argv
5 # Print all arguments but the program
6 # or module name:
7 print sys.argv[1:] # "array slice"
```

## Catching Exceptions

```
1 #python code a.py
2 x = 0
3 try:
4 print 1/x
5 except ZeroDivisionError, message:
6 print "Can't divide by zero:"
7 print message
8 \>\>\>python a.py
9 Can't divide by zero:
10 integer division or modulo by zero
```

## Try-Finally: Cleanup

```
1 f = open(file)
2 try:
3 process_file(f)
4 finally:
5 f.close() # always executed
6 print "OK" # executed on success only
```

## Raising Exceptions

- raise IndexError
- raise IndexError("k out of range")
- raise IndexError, "k out of range"
- try: somethingexcept: # catch everything print "Oops" raise # reraise

## Python: Pros & Cons

- Pros
  - Free availability (like Perl, Python is open source).
  - Stability (Python is in release 2.6 at this point and, as I noted earlier, is older than Java).
  - Very easy to learn and use
  - Good support for objects, modules, and other reusability mechanisms.
  - Easy integration with and extensibility using C and Java.
- Cons
  - Smaller pool of Python developers compared to other languages, such as Java
  - Lack of true multiprocessor support
  - Absence of a commercial support point, even for an Open Source project (though this situation is changing)
  - Software performance slow, not suitable for high performance applications

## References

- Python Homepage
- <http://www.python.org>
- Python Tutorial
- <http://docs.python.org/tutorial/>
- Python Documentation
- <http://www.python.org/doc>
- Python Library References
- <http://docs.python.org/release/2.5.2/lib/lib.html>
- Python Add-on Packages:

- <http://pypi.python.org/pypi>

## Raspberry Pi Simulator

- There is a simulator for the RasPi and some Hats here:
  - <https://trinket.io/sense-hat>
- This is obviously tailored to help their sales, but it is useful.
- You can interact with it using the keyboard and the mouse.
- In the Rock-Paper-Scissors example, the Hat Joystick is mapped to the arrow keys
- Tilting the unit can be done by the mouse

## Exercise

- Understand the workings of the Rock-Paper-Scissors program
- Modify the program so that you can play Rock-Paper-Scissors-Spock-Lizard
- R-P-S is a state based game
  - Is the game implemented as a state machine?
- 



## 5.4 Node.JS in embedded Systems

### Node.js

- For Embedded Systems

### Node.js and Java Script

- Language Requirements for IoT
- Support Web Protocols
- Single Language for all tasks
  - Client Application
  - Cloud Engine
  - Gateways
  - Embedded Code
- JS is a possible solution

## Popular Languages on github

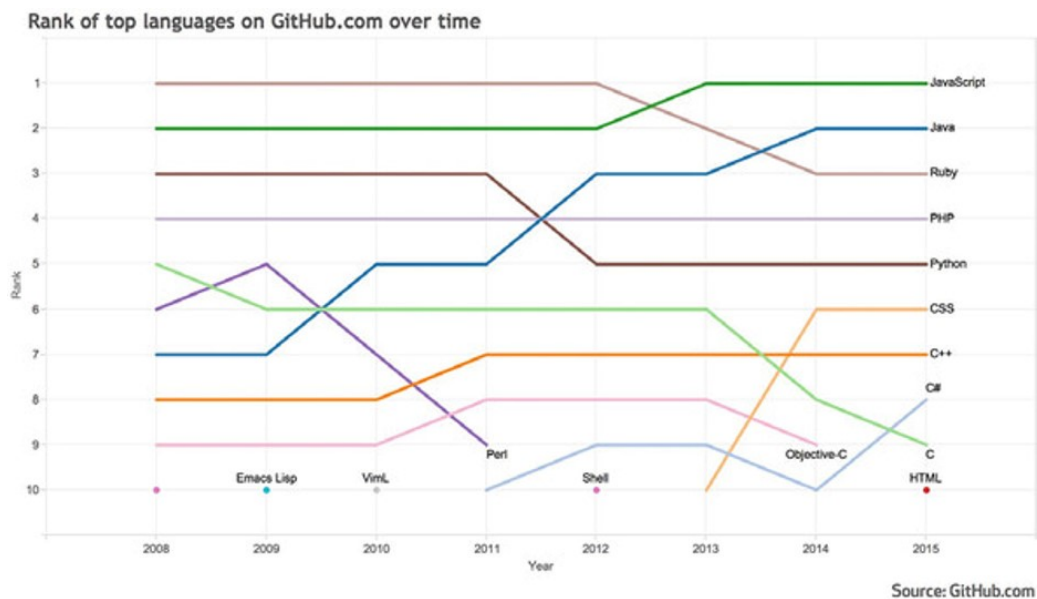


Figure 5.4.1: Common Languages

## The rise of JS

- JS is powerful when running on web-like structures
- IoT and networked embedded is becoming that
- Server side is often in PHP, Ruby, Python or Java
- Node.JS is serverside JS
- <https://nodejs.org/en/foundation/>

## Node.js

- event-driven architecture
- non-blocking I/O API
-

- framework for high-performance server-side web applications
- Normal Server: deploy your application in a running server instance
- Node : your application is the server.
- Builds on the highly efficient Google V8 JavaScript engine
- Heart of the Chrome browser

## Language

- Node isn't JavaScript
- JavaScript is the language you use to build Node applications
- Others, like CoffeeScript are possible.

## Installation

- Node.JS needs to be installed on any machine to run it
- <http://nodejs.org/en/download/>.
- Entering `node -version` will tell you if you have it.

## A first blink sketch

```

1
2 var http = require("http"); //\#A
3 http.createServer(function(req,res){ //\#B
4 res.writeHead(200, {'Content-Type': 'text/plain'}); //\#C
5 res.end('Hello World');
6 }).listen(8585); //\#D
7 console.log('Server started!');
8
9 //\#A "require" is used to "import" libraries
10 //\#B creates a new HTTP server and pass it a function to be called
 whenever a client sends a request
11 //\#C we start writing a response, beginning with the HTTP headers
12 //\#D starts the HTTP server on port 8585

```

## What, a server?

- First, require the http module. This basically loads the HTTP module and makes it available to your application
- Then use the HTTP object to create a new server.
- Pass to this server a function that will be called whenever a client connects to your server.
- When a client actually connects, the function is called with two parameters:

- req represents the client request and offers a number of functions to retrieve information about it, such as the requested URL or the payload that was sent.
- res represents the response you'd like to send back to the client.

## Calling Sensors

```

1
2 var http = require("http"); var port = 8686;
3 function randomInt (low, high) {
4 return Math.floor(Math.random() * (high - low) + low);
5 }
6 http.createServer(function(req,res){ console.log('New incoming client
 request for ' + req.url); res.writeHead(200, {'Content-Type': '
 application/json'}); //\#A
7 switch(req.url) { //\#B
8 case '/temperature': // And return the corresponding JSON
9 res.write('{"temperature" : ' + randomInt(1, 40) + '}'); //\#C
10 break;
11 case '/light': res.write('{"light" : ' + randomInt(1, 100) + '}
12 '); break;
13 default: res.write('{"hello" : "world"}');
14 }
15 res.end(); //\#D
16 }).listen(port);
17 console.log('Server listening on http://localhost:' + port);
18
19 //\#A Setting the header to announce we return JSON representations
20 //\#B Read the request URL and provide responses accordingly
21 //\#C Write the temperature result as JSON
22 //\#D Causes to return the results to the client

```

## Node.js Event Loop

- Building server-side web applications that needs to deal with more than a single client at a time requires the ability to handle a large number of connections in a concurrent manner.
- If the connections couldn't be accepted concurrently, each new web client would have to wait until the clients that arrived earlier were served.
- Imagine a single gas station on a busy intersection, with a single pump used by a single attendant.

## Multithreaded Approach

- The first pattern is to create one process - or better, one thread - per request.
- A thread is essentially a lightweight process because it's able to share some of the process's resources (for example, some of the allocated memory) while executing largely independently.

- `var result = database.query("SELECT things FROM deviceTable");console.log(result);`
- This works fine in the PHP and Java worlds because while one client waits to be served, the underlying server (for example, Apache) serves the next clients.
- It does this by creating one thread for each incoming client.

### Threaded Multi-Request-Response

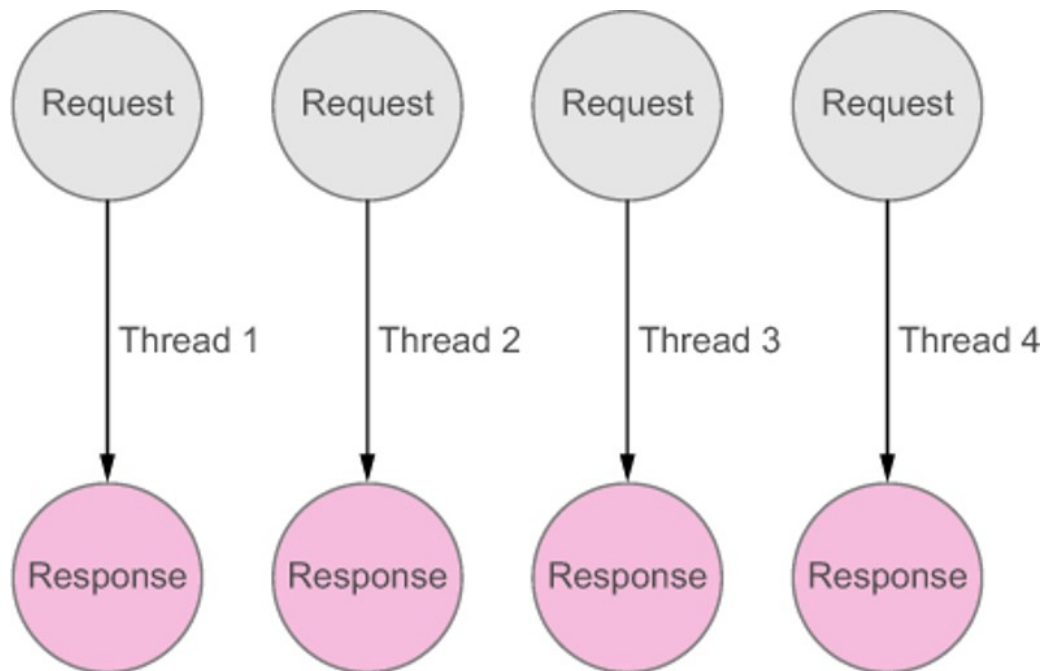


Figure 5.4.2: Threaded request response

### Single-threaded, non-blocking web servers

- Imagine each thread is a pump with an attendant, each web client is a customer, and the database is the central tank.
- We're better off than with a single pump and attendant, but each attendant is still idle some of the time, waiting for our tank to get filled.
- What if we had only one attendant managing several pumps in an efficient manner?
- Wouldn't this be more economical in terms of costs?

### Single Thread - Non Blocking

- This second pattern is an event-driven system often based on the idea of a single thread (or a limited number of threads) with an event loop and non-blocking or asynchronous I/O.
- The Node.js runtime is built on these principles. It runs a single thread with an event loop and strongly favors asynchronous I/O operations.

- When the Node server accepts a client, it puts it on hold until the I/O operations it requested (for example, reading from the database, reading a value from a sensor, or uploading a file on a remote server) have returned and goes on serving other clients in the meanwhile.

### Non Blocking Event Loop

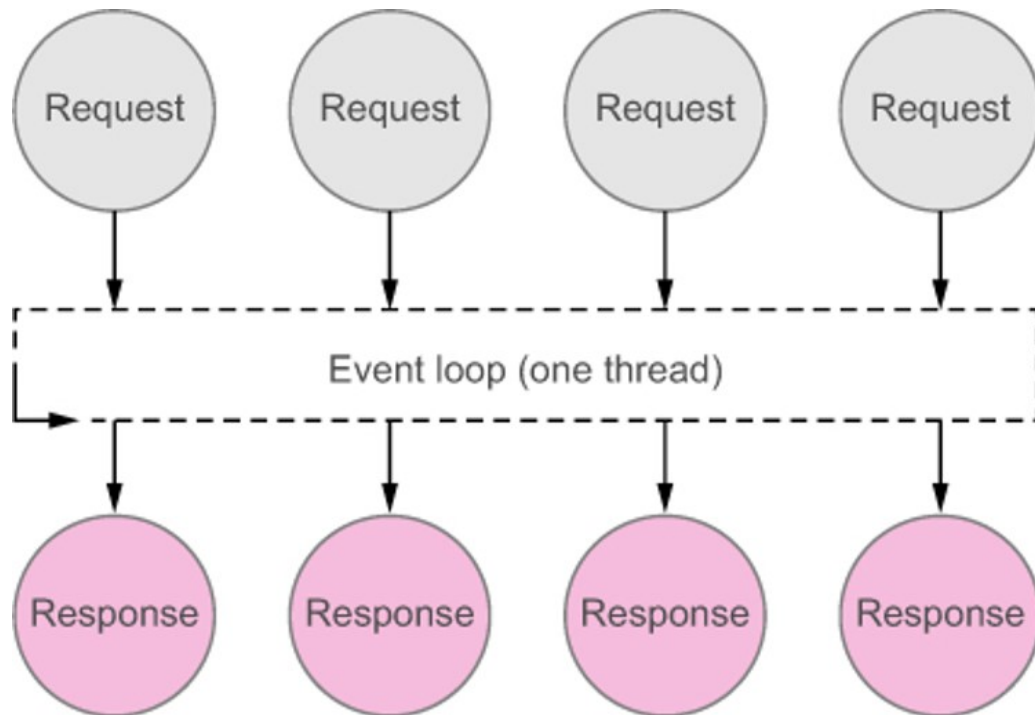


Figure 5.4.3: Non-blocking Event Loop

### Synchronous vs Asynchronous

- Imagine that we run the synchronous code shown previously in a Node server.
- The Node runtime will fetch data from the database, wait until it gets it, and then execute the `console.log()` instruction.
- If there's a lot of data to fetch, you might have to wait for quite some time, during which the Node server wouldn't be able to serve or accept any other incoming request - not a good idea to scale your system!

```
1 Database.query("SELECT things FROM deviceTable",
2 function(results) {
3 //do something with those results
4 }
5);
6 Console.log(results);
```

---

## Callback

- The major difference with this version is that you pass a function as a parameter when calling the `database.query` function (callback function).
- The Node event loop will then put this call on hold until a response arrives from the database.
- This time, while waiting for the response, other instructions will be executed until the results have been fetched from the database and are ready to be consumed.
- When the results are retrieved, the event loop will call the `callback function(results)` with the results.

## Callback 2

- This means that the `database.query` instruction will return directly after it's called.
- As a consequence, the `console.log(results)` will execute right away, before the results are actually ready!
- Moreover, the results themselves will be available only to the anonymous function (injected in the callback by the event loop)
- The variable will be undefined for the `console.log` instruction.

## Synchronous?

- As a convention, to make sure you keep track of your synchronous calls, Node functions that are synchronous end with the `Sync` suffix and do not usually take a function as a parameter.
- For example,
  - `fs.readFileSync(filename)` reads a file synchronously (that is, blocking the single thread until the file is fully read) as opposed to
  - `fs.readFile(filename,callback)`, which reads a file asynchronously (that is, releasing the thread to do other work while waiting for the file to be read).

## DIRT

- Single-threaded servers with asynchronous I/O are not
- But they've been shown to yield better performance and scaling in a number of situations, particularly for data-intensive real-time (DIRT) applications.
- Interestingly, the nature of real-world Things like sensors does perfectly fit into the DIRT type of applications, which partially explains the growing interest in these types of servers in the context of the Internet of Things.

- Reading data from the real world requires gathering a lot of sensor data (through I/O operations on physical sensors), and clients expect to be informed about changes of these sensors' state in a timely, event-driven, and almost real-time manner!

## Anonymous Callback

- Consider a function  $F$  that performs some asynchronous operation such as fetching some data from the network,
- We need a way to express what to do next when  $F$  returns so we give  $F$  another function to call whenever  $F$  finishes with the message-sending operation.
- This is an example of a callback - that is, a function passed as a parameter to an asynchronous function describing what to do after the async function has completed.

## Callback Code

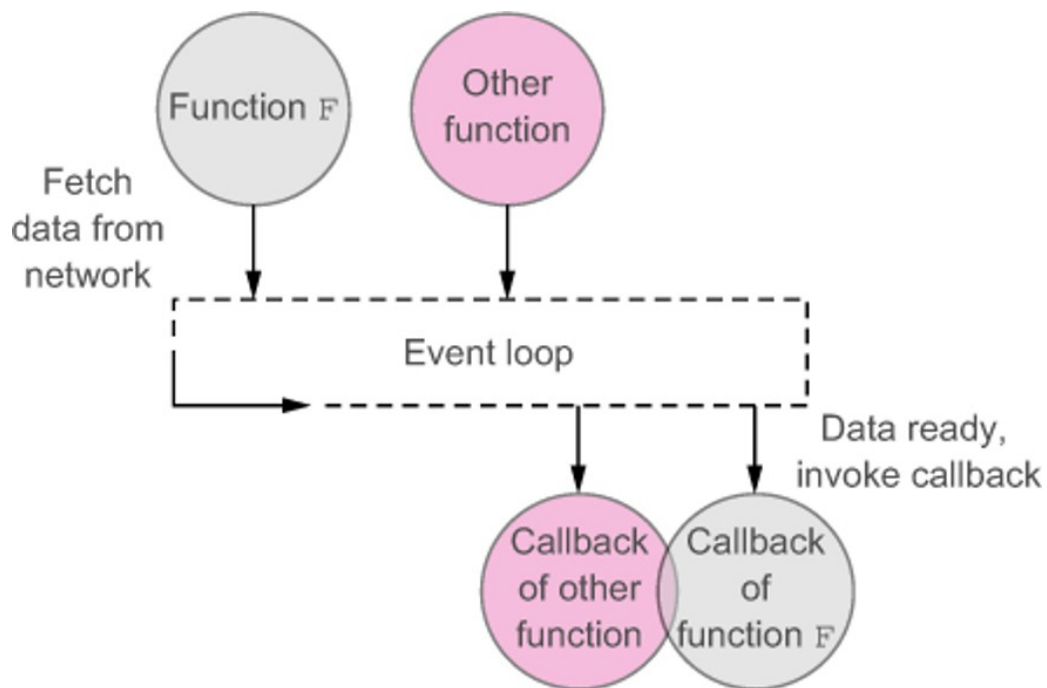


Figure 5.4.4: Callback

## Callback Code

```

1 var http = require("http"), request = require('request'), fs = require('
 fs');
2 var serviceRootUrl = 'http://localhost:8686';
3 http.createServer(function (req, res) { console.log('New incoming
 client request...');
4 if (req.url === '/log') { getTemperature(res); //\#A
5 } else { res.writeHead(200, {"Content-Type": "text/plain"});
 res.write('Please use /log'); res.end(); }

```

```

6 }).listen(8787);
7 function getTemperature(res) { //\#B
8 request({url: serviceRootUrl + '/temperature', json: true}, function
9 (err, resp, body) {
10 if (err) throw err;
11 if (resp.statusCode === 200) { console.log(body); var
12 temp = body.temperature; getLight(res, temp); //\#C } });}
13
14 function getLight(res, temp) {
15 request({url: serviceRootUrl + '/light', json: true}, function (err,
16 resp, body) {
17 if (err) throw err;
18 if (resp.statusCode === 200) {
19 console.log(body);
20 var light = body.light;
21
22 logValuesReply(res, temp, light); //\#D
23 }
24 });
25 }
26
27 function logValuesReply(res, temp, light) {
28 var logEntry = 'Temperature: ' + temp + ' Light: ' + light;
29 fs.appendFile('log.txt', logEntry + '\n', encoding = 'utf8', function
30 (err) {
31 if (err) throw err;
32 res.writeHead(200, {"Content-Type": "text/plain"}); //\#E
33 res.write(logEntry);
34 res.end();
35 });
36 }
37
38 //\#A Get the temperature and start the chain of calls
39 //\#B A named temperature function
40 //\#C Once the callback for temperature has been called we proceed with
41 //\#D We then call the named function to log values
42 //\#E Return to the client

```

## Named Callbacks

- This code is essentially the same as listing before, but this time the workflow is split across several functions. In the callback of the server you invoke `getTemperature()`; this function asynchronously gets the temperature and calls the `getLight()` function as soon as the temperature arrives. `getLight()` fetches the light from the server and invokes `logValuesReply()` as soon as the light value is ready. Finally, `logValuesReply()` asynchronously logs the value to a file and replies to the client.

## 5.5 A systematic approach to debugging (especially Linux)

### Attribution

- This part of the Lecture is based on a talk at the
- Embedded Linux Conference
- June 30, 2020
- A Disciplined Approach to Debugging
- Lev Iserovich
- D.E. Shaw Research, LLC

## Debugging Statistics

- ca. 50% of developer time spent on admin tasks <sup>1</sup>
- ca. 50% of programming time is spent debugging (same source)
  - ca. 25% of your time is spent debugging
- About 1.5M SW developers in US, total wages roughly USD 150B <sup>2</sup>
- roughly USD 37B spent debugging in the US
- Little methodology, very few university courses deal with debugging
  - MIT course catalog: grep 'debugging' ca. 3/10,299 words <sup>3</sup>

## Step 0 - Testing

- Always cheaper than debugging
- Unit tests
  - For each subcomponent / class / function
- Integration test
  - Entire system
- More expensive bugs -> More testing
  - ASIC design
  - ca. \$10M, months of time for each mistake
  - Entire teams devoted to design verification and test
- "Everyone knows that debugging is twice as hard as writing a program in the first place. So if you're as clever as you can be when you write it, how will you ever debug it?" -Brian Kernighan

## First: Reproduce

---

<sup>1</sup> Britton, T., Jeng, L., Carver, G., Cheak, P., Katzenellenbogen, T. 2013. Reversible debugging software. Cambridge Judge Business School; <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.444.9094&rep=rep1&type=pdf>.

<sup>2</sup> [https://www.bls.gov/oes/current/oes\\_nat.htm](https://www.bls.gov/oes/current/oes_nat.htm)

<sup>3</sup> <http://student.mit.edu/catalog/m6a.html>

- User / bug reports often are missing information
  - Or missing vital information
  - Low signal / noise ratio
- Needs to be reproducible to properly instrument / observe
- Confirmed fix only possible when reproduced
  - Intermittent problems
  - Load or environmental related bugs can come and go
- Often bugs found 'by inspection' don't actually fix the problem
  - There are lots of bugs in software, did you find the right one?
- •

## Second: Observe

- Visibility into program is vital
- Logging
  - printf
  - Logging frameworks (single machine)
    - C++: Google's glog (<https://github.com/google/glog>)
    - Python: logging module (<https://docs.python.org/3.5/library/logging.html>)
    - Shell: logger (man logger), file redirect
    - Many languages: fluentd ([www.fluentd.org](http://www.fluentd.org)) / fluentbit ([www.fluentbit.io](http://www.fluentbit.io))
  - Logging aggregators (many machines)
    - Splunk (<https://www.splunk.com/>)
    - ElasticStack (ElasticSearch / Kibana / Logstash) (<https://www.elastic.co/elastic-stack>)
    - Grafana (visualization) (<https://grafana.com/>)
  - Check and print all return codes
    - `assert()` is your friend

9/2

9/9

0800 Action started  
 1000 " stopped - action ✓ { 1.2700 9.027 847 025  
 1300 (032) MP-MC ~~1.50776415~~ 9.027 846 895 convert  
 032 PRO 2 2.130476415  
 convert 2.130676415  
 Relays 6-2 in 032 failed special speed test  
 in relay 11.000 test.  
 Relays changed  
 1100 Started Cosine Tapc (Sine check)  
 1525 Started Multi Adder Test.  
 1545  Relay #70 Panel F  
 (moth) in relay.  
 First actual case of bug being found.  
 1630 Action started.  
 1700 closed down.

### Third: Observe

- strace - trace system calls
- gdb
  - Attach to running process
  - Get core with gcore / kill -SEGV (set ulimit -c unlimited)
  - Don't forget debug symbols (gcc -g)
  - Recompile with -O0 for straightforward flow
  - Always use -Wall -Werror
- Reverse debugging
  - Sounds cool, not super useful
- Wireshark / tcpdump
- Hardware analyzers
  - i2c/SPI, PCI-Express, etc

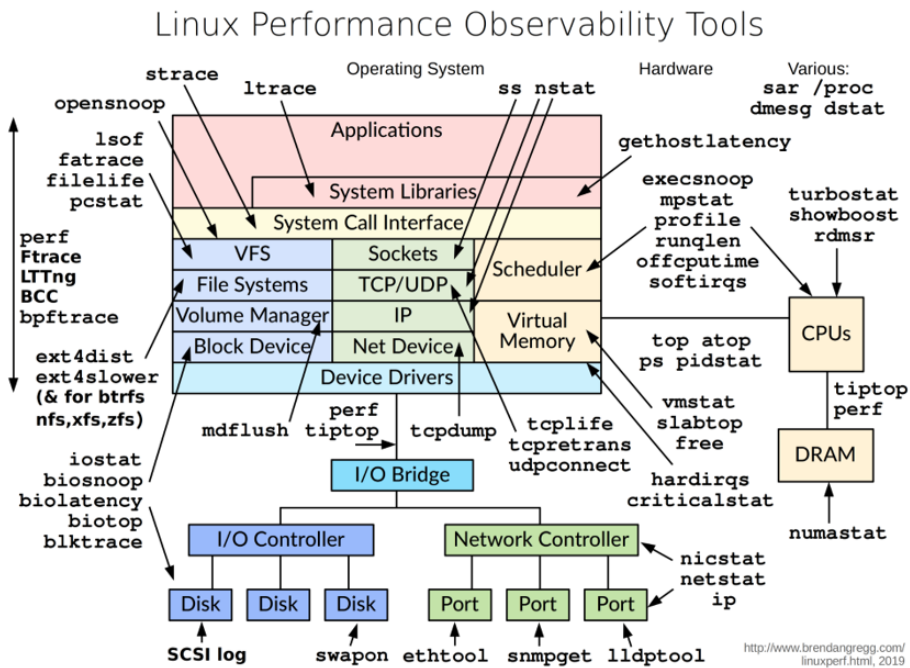
### Observe

- Performance monitoring
  - gperftools (Google perf tools)
  - iostat (man iostat), sar, etc.
  - htop
  - oprofile, other kernel profiling tools
- Memory / access debugging
  - gperftools memory profile

- valgrind (10-100x slowdown)
- Dr. Memory (similar to valgrind)
- 
- 

## No Title

- From Brendan Gregg
- <http://www.brendangregg.com/linuxperf.html>



## Bisect

- With or without hypothesis
  - Educated or uneducated guess
- Disable parts of program / workflow
- Switch compiler optimization levels
- Switch tool version
  - One at a time!
- Always move up the trust stack
  - Your code ; library ; compiler ; OS ; hardware
  - Trust but verify

## Classifying Bugs

- Hard error (Easy)

- Crash
- Unrecoverable error
- Wrong (but consistent) data output
- Resource exhaustion (eg. out of memory)
- Soft error (Difficult)
  - Intermittent failure / crash
  - Inconsistent data output
  - Recovered error
  - Performance degradation
- Difference: reproducibility!
  - Take the easy way
  -
- 

## Turning Soft Errors into Hard Errors

- Tracing and logging
  - Turn up debug level
  - Catch first instance of error
  - Trace external events - logs, network, CPU load, I/O load
  - Try to replicate error message
- Stress test application
  - Load testing
- Debugger
  - Breakpoints
  - Watchpoints
  -
- 

## Example: Disk I/O Error

I/O error log in program

```
2019-06-25 11:48:37.353: Uncategorized and unmanaged std::runtime_error exception:
2019-06-25 11:48:37.353: boost::filesystem::status: Input/output error: "/f/a/job/
2019-06-25 11:48:37.353: details: boost::filesystem::status: Input/output error: "
2019-06-25 11:48:37.355: Recording error code BAD_ERROR 1001 (priority 1)
```

"Soft error" - intermittent - EIO is unpredictable

Trying other errors (ENOENT, EPERM) does not result in the same failure

Look at /usr/include/asm-generic/errno.h and errno-base.h

How about ELOOP

```
$ ln -s a b
```

```
$ ln -s b a
$ cat a
cat: a: Too many levels of symbolic links
```

## Found!

- Bingo!

```
int main() {
 printf("exists foo: %d\n", boost::filesystem::exists("foo"));
 printf("exists a: %d\n", boost::filesystem::exists("a"));
 return 0;
}
```

```
$./test
```

```
exists foo: 0
```

```
terminate called after throwing an instance of 'boost::filesystem::filesystem\
```

```
what(): boost::filesystem::status: Too many levels of symbolic links: "a"
```

```
Aborted
```

```
boost::filesystem::exists throws on certain errors
```

- Solution: use noexcept variant

```
bool exists(const path& p); bool exists(const path& p, system::error_code& ec) n
Returns:Â exists(status(p))Â orÂ exists(status(p, ec)), respectively. If ec != 0 an
Throws:Â As specified inÂ Error reporting.
```

## Start-up Problem

- Program fails to start

```
Somewhere in log file:
```

```
Intel MKL FATAL ERROR: Cannot load /opt/intel/mkl/lib/intel64_lin/libmkl_rt.so
```

```
$ ls /opt/intel/mkl/lib/intel64_lin/libmkl_rt.so
```

```
/opt/intel/mkl/lib/intel64_lin/libmkl_rt.so
```

- 
- Step 1: Reproduce
  - When running the same command from command line, works fine!
  - Library is not on NFS, present on the machine where command fails

## No Title

- Step 2: Observe
- Use strace when starting program, look for failing trace

- Use '-f' to follow forked processes

```
\ite Look for errors different than working trace
\ite 22293 mprotect(0x7f7cddb4000, 2097152, PROT_NONE) = 0
\ite 22293 mmap(0x7f7cdddf4000, 32768, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|
\ite 22293 mmap(0x7f7cdddfc000, 73280, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|
\ite 22293 close(3) = 0
\ite 22293 mprotect(0x7f7cdddf4000, 24576, PROT_READ) = 0
\ite 22293 mprotect(0x7f7cde184000, 8192, PROT_READ) = 0
\ite 22293 getcwd(0x7f7cdddfef1a0, 4096) = -1 ENOENT (No such file or directory)
\ite 22293 open("/usr/share/locale/C/mkl_msg.cat", O_RDONLY) = -1 ENOENT (No such f
\ite 22293 open("/usr/share/locale/C/LC_MESSAGES/mkl_msg.cat", O_RDONLY) = -1 ENOEN
\ite 22293 open("/usr/share/locale/C/mkl_msg.cat", O_RDONLY) = -1 ENOENT (No such f
\ite 22293 open("/usr/share/locale/C/LC_MESSAGES/mkl_msg.cat", O_RDONLY) = -1 ENOEN
\ite 22293 write(2, "Intel MKL FATAL ERROR: Cannot lo...", 132) = 132
\ite 22293 write(2, "\n", 1) = 1
\ite 22293 exit_group(2) = ?
\ite 22293 +++ exited with 2 +++
\ite 22290 <... wait4 resumed> [{WIFEXITED(s) &\& WEXITSTATUS(s) == 2}], 0, NULL)
\ite 22290 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
```

### Step 3: Bisect

- getcwd() called on removed directory
- Reproduce with shell script, observe behavior

```
#!/bin/bash
mkdir test
cd test
(sleep 1; prog) \&
cd ..
rmdir test
wait
```

```
$./mkl-test.sh
```

```
...
```

```
Intel MKL FATAL ERROR: Cannot load /opt/intel/mkl/lib/intel64_lin/libmkl_rt.so
```

- Conclusion: MKL is very particular where you run it, don't remove underlying directory

### Observe with GDB

- For hangs, or to debug live process: `gdb jexe pid`
- Stops process, so be careful!

- gcore will generate a core from running process without stopping it
- Make sure to set ulimit -c unlimited to get core dumps
- Not sure which executable? Gdb will tell you:

```
$ gdb /bin/ls core.1234
...
Core was generated by '/path/to/my/exe'
Thread support:
Show all threads: info threads
Switch to thread: thread 1
Command on all threads: thread apply all backtrace
```

## More GDB Features

- Remote debugging for embedded platforms (gdb-server and target command)
- Watchpoints - triggers on memory access - useful to watch for corruption
  - watch *jexpression<sub>i</sub>* (write), rwatch (read), awatch (r/w)
  - HW watchpoints are free, but limited in number - 4 on x86
  - SW watchpoints - unlimited, but expensive
- Conditional breakpoints
  - break *jlocation<sub>i</sub>* [*thread jn<sub>i</sub>*] if *jcondition<sub>i</sub>*
- Observe values at each break
  - command *jbreakpoint<sub>i</sub>*
    - print val
    - continue
  - catch [exception — fork — signal — ...]

## Debugging with GDB

- 
- What can debuggers do?
- 
- Run programs
- Make the program stops on specified places or on specified conditions
- Give information about current variables' values, the memory and the stack
- Let you examine the program execution step by step - stepping
- Let you examine the change of program variables' values - tracing
- 
- ! To be able to debug your program, you must compile it with the -g option (creates the symbol table) !
- CC -g my\_prog

## GDB - Freeware GNU Debugger

- Starting GDB: `gdb`
- 
- Loading the symbol table:
- 
- `file my_prog`
- 
- `/my_prog` is executable file name!/  
•
- 
- Exit GDB: `quit`
- 
- Executing shell commands: `shell command args`
- `make` is a special case: `make args`
- Start GDB and load the symbol table in one step:
- 
- `gdb my_prog`

## GDB - Commands

- Command names may be truncated if the abbreviation is unambiguous: `s` (for `step`), `r` (for `run`)
- 
- UNIX style TAB completion for the command names.
- Alternative way: complete chars.
- 
- Ex.: complete `h` results in: `handle`
- `hbreak`
- `help`
- Getting help:
- `help` (or `h`) - lists all classes of commands.
- `h` command - displays a short description of the command

## GDB - Running Programs

- Running a program:
- `run` (or `r`) – creates an inferior process that runs your program.
- 
- if there are no execution errors the program will finish and results will be displayed

- in case of error, the GDB will show:
- - the line the program has stopped on and
- - a short description of what it believes has caused the error
- 
- There is a certain information that affects the execution of a program:
  - program's arguments
  - program's environment
  - program's working directory
  - the standard input and output

## **GDB - Program's arguments**

- Specifying arguments for your program:
- As arguments to run: `run arg1 arg2 ...`
- With set args command: `set args arg1 arg2 ...`
- 
- `! run` without arguments uses the same arguments used by the previous run.
- 
- `! set args` without arguments - removes all arguments.
- 
- `! show args` command shows the arguments your program has been started with.

## **GDB - Program's environment**

- Changing the PATH environment variable:
- `path dir` - add the directory `dir` at the beginning of the PATH variable. You may specify several directory names separated by `':'` or white space.
- 
- `show paths` - displays the search paths for executables.
- 
- Changing the working directory:
- `cd dir` - to change the working directory
- 
- Redirecting output:
- `run & outfile` direct the output to the file `outfile`.

## **GDB - Debugging an already-running process**

- From inside GDB:
- `attach process-id`
- `/` To get the process ID use the UNIX command `ps /`

- 
- From outside GDB:
- `gdb my_prog process-id`
- 
- ! The first thing GDB does after arranging to debug the specified process is to stop it.
- 
- `detach` - detaches the currently attached process from the GDB control. A detached process continues its own execution.

## GDB - Breakpoints and watchpoints

- Breakpoints and watchpoints allow you to specify the places or the conditions where you want your program to stop.
- 
- `break arg` - stops when the execution reaches the specified line
- `/ arg` - function name, line number, +/- offset /
- `watch expr` - stops whenever the value of the expression changes
- `clear [arg]`
- Without arguments deletes any breakpoint at the next instruction to be executed in the current stack frame
- `delete [bnum]`
- Without arguments deletes all breakpoints.

## GDB - Examining variables

- ! Global variables can be examined from every point in the source file.
- ! Local variables - can be examined only in their scope or using:
- `file::variable` or `function::variable`
- 
- The variable type: `ptype var`
- 
- Current value: `print var`
- 
- Automatic display: `display var` - adds var to the automatic display list.
- `undisplay dnum`
- Specifying the output format (x, o, d, u, t, a, f, and c) :
- `print /t var` - prints the value of var in binary format

## GDB - Value history

- The value history keeps the values printed by the print command.
- 
- Previously printed values can be accessed by typing \$ followed by their history number.
- \$ - refers to the most recent value and
- \$\$n - refers to the n-th value from the end.
- 
- show values [n—+]
- Without argument - the last 10 values.
- n - 10 values centered around n
- + - 10 values after the last printed

## **GDB - Stepping through the program**

- step [count] - program execution continue to next source line going into function calls.
- 
- next [count] - program execution continue to the next source line omitting function calls.
- 
- continue - resume program execution
- 
- until - continue until the next source line in the current stack frame is reached.  
/useful to exit from loops/

## **GDB - Altering execution**

- Returning from a function
- finish - forced return
- return [ret\_value] - pops the current stack frame
- Continuing at different address
- jump line\_num—\*address
- Altering the value of a variable
- set i=256
- Proceeding to a specified point:
- until [line\_num—\*address —function\_name]

## **GDB - The stack frame**

- Stack frames are identified by their addresses, which are kept in the frame pointer register.

- 0
- 1
- 2
- Selecting a frame:
- frame n—addr
- up n
- down n
- Information about the current frame
- frame - brief description
- info args - shows function arguments
- info locals - shows local variables

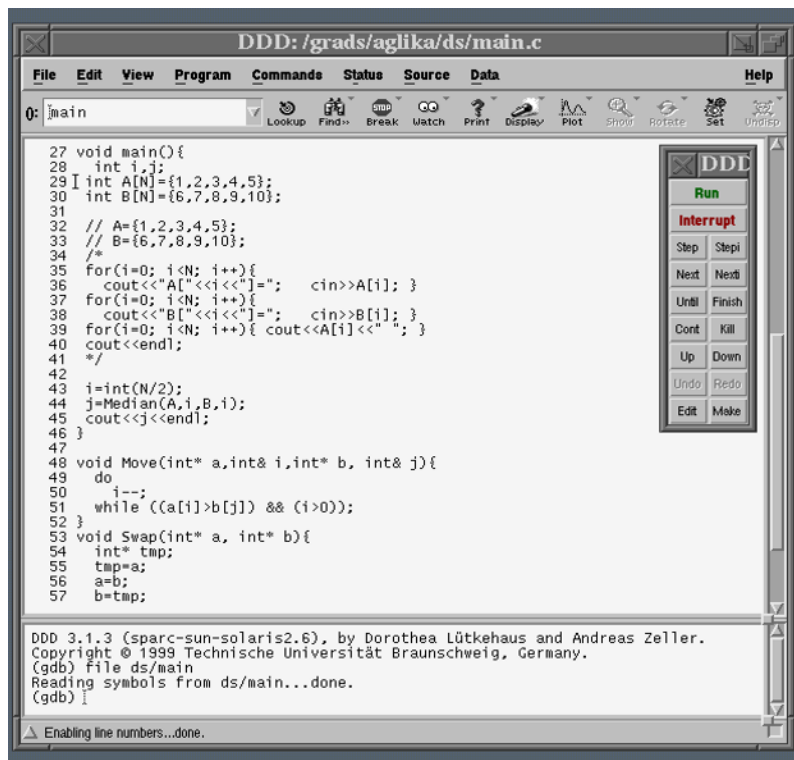
## GDB - Convenience variables

- Convenience variables are used to store values that you may want to refer later. Any string preceded by \$ is regarded as a convenience variable.
- Ex.: \$table = \*table\_ptr
- 
- There are several automatically created convenience variables:
  - \$pc - program counter
  - \$sp - stack pointer
  - \$fp - frame pointer
  - \$ps - processor status
  - \$\_ - contains the last examined address
  - \$\_\_ - the value in the last examined address
  - \$\_exitcode - the exit code of the debugged program

## DDD - The Data Display Debugger

- DDD is a GUI debugger that can work with several inferior debuggers including GDB.
- 
- GDB commands can be typed in the debugger console window.
- 
- DDD sets some GDB settings automatically and will not work correctly if you change them. These are:
- 
- set height 0
- set width 0
- set verbose off
- set prompt (gdb) !

## DDD - View



## DDD - GUI Advantages

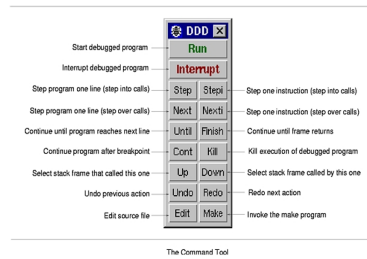
- Frequently used commands are on the toolbars, have assigned shortcut keys or can be done just with a mouse click.
- 
- Easy browsing through the source !
- 
- Examining current variables values directly - by placing the mouse pointer over them.
- 
- Possibility to graphically display the program data.
- 
- Help menu - What now? - very helpful !

## DDD - Command menu

- 

## GDB - Examining memory

- The x command (for "examine"):



- x/nfu addr - specify the number of units (n), the display format (f) and the unit size (u) of the memory you want to examine, starting from the address addr. Unit size can be - b, h (half), w and g (giant).
- x addr - start printing from the address addr, others default
- x - all default
- Registers
- Registers names are different for each machine. Use info registers to see the names used on your machine.
- GDB has four "standard" registers names that are available on most machines: program counter, stack pointer, frame pointer and processor status.

## GDB - Additional process information

- info proc - summarize available information about the current process.
- info proc mappings - address range accessible in the program.
- info proc times - starting time, user CPU time and system CPU time for your program and its children.
- help info !
- info signals - information about the system signals and how GDB handles them.

## Debug Hang with GDB

- Application hanging periodically (once every 10,000 runs approximately)
- Job scheduler kills application
- Step 1: Reproduce
  - Rerun of the same inputs doesn't lead to hang
  - Difficult to reproduce
- Step 2: Observe
  - Add code to scheduler that detects hung application to request coredump (gcore)

- (Tip: core files are large but gzip really well)
- Analyze core dump
- gdb /my/app core.1234

## No Title

```
(gdb) info threads
```

| Id  | Target Id | Frame                                                     |
|-----|-----------|-----------------------------------------------------------|
| * 1 | LWP 5088  | 0x00007efc1b74d1ad in glob64 () from /lib64/libc.so.6     |
| 2   | LWP 5263  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 3   | LWP 5265  | 0x00007efc1ff5370d in read () from /lib64/libpthread.so.0 |
| 4   | LWP 5732  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 5   | LWP 5733  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 6   | LWP 5734  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 7   | LWP 5735  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 8   | LWP 5736  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 9   | LWP 5737  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 10  | LWP 5738  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 11  | LWP 5739  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 12  | LWP 5740  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |
| 13  | LWP 5741  | 0x00007efc1ff50945 in pthread_cond_wait@@GLIBC_2.3.2 () f |

```
...
```

```
(gdb) thread 3
```

```
[Switching to thread 3 (LWP 5265)]
```

```
#0 0x00007efc1ff5370d in read () from /lib64/libpthread.so.0
```

Stuck in read(), but why?

libpthread.so is not compiled with debug info

## No Title

```
(gdb) info registers
```

|     |                    |                 |
|-----|--------------------|-----------------|
| rax | 0xfffffffffffffe00 | -512            |
| rbx | 0x0                | 0               |
| rcx | 0xffffffffffffffff | -1              |
| rdx | 0x4                | 4               |
| rsi | 0x7efc1aa620cc     | 139621243953356 |
| rdi | 0x0                | 0               |
| rbp | 0x4                | 0x4             |
| rsp | 0x7efc1aa62080     | 0x7efc1aa62080  |
| r8  | 0x1                | 1               |
| r9  | 0x4b0              | 1200            |

```
...
```

[https://en.wikipedia.org/wiki/X86\\_calling\\_conventions](https://en.wikipedia.org/wiki/X86_calling_conventions)

```
$ man 2 read
```

```
...
ssize_t read(int fd, void *buf, size_t count);
read(fd = 0, buf, 4); <- fd 0 is STDIN
```

|                        |                                                     |   |
|------------------------|-----------------------------------------------------|---|
| System V AMD64 ABI[22] | Solaris, Linux, BSD, OS X (GCC, Intel C++ Compiler) | R |
|------------------------|-----------------------------------------------------|---|

## Bug in Hanging Server

```
void MyServer::main_loop() {
 while (!m_thread_exit) {
 ret = read(m_socket, &m_buf, 4);
 if (ret <= 0) return; // returns on EOF or error
 // Process data
 }
}

void MyServer::cleanup()
{
 log("Cleaning up server...\n");
 close(m_socket); // tear down connection
 m_thread_exit = true; // signal exit to main loop thread
 wait(); // wait for it to finish
}
Race between close() and read()
Object is cleaned up, and memory used for something else that sets m_socket = 0
```

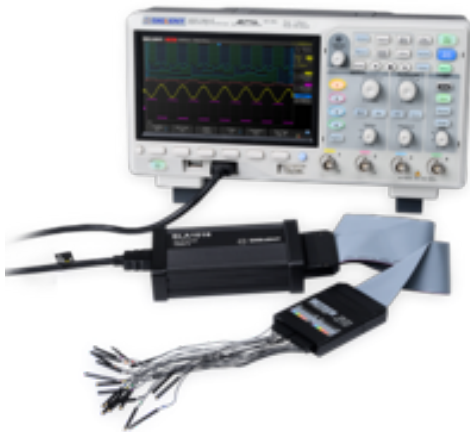
## Observe Network

- Wireshark is the best GUI tool for network capture/debug
- tcpdump to capture output to file, then visualize with wireshark
- Note: capture specification language and visual selection language are different
- man 7 pcap-filter for capture
- tcpdump -i eth0 tcp port 80 and host 142.250.64.78
- man 4 wireshark-filter
  - tcp.port == 80 and ip.addr == 142.250.64.78
  - Try to capture as much as possible without missed packets, filter in wireshark
  - For fast streams kernel cannot keep up - look for 'dropped by kernel' statistic
  - Wireshark can follow TCP streams

## TCP Packet Loss

## Hardware Debugging

- Observability is key!
- Get an analyzer for any interface which you are building hardware for
- I2C / SPI
- Saleae Logic analyzer ([www.saleae.com](http://www.saleae.com)) - Linux support
- Beagle analyzer ([www.totalphase.com](http://www.totalphase.com)) - Linux support
- Easyi2c ([www.easyi2c.com](http://www.easyi2c.com)) - Windows only
- USB
- Tcpdump / wireshark supports USB!
- modprobe usbmon
- tcpdump -i usbmon1
- PCIe / NVMe / SATA, other high speed protocols
- LeCroy protocol analyzers ([www.teledynelecroy.com](http://www.teledynelecroy.com))
- Keysight protocol analyzers ([www.keysight.com](http://www.keysight.com))



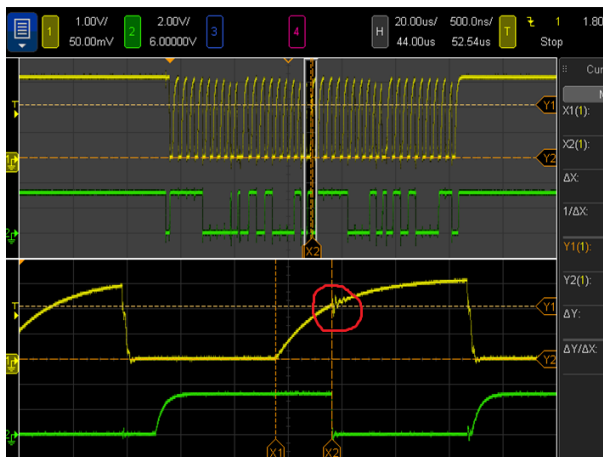
## I2C Example

- Periodic loss of i2c bus connected to Xilinx FPGA
  - Once every few days in normal operation
  - Devices stop responding, board reset required
- Step 1 - Reproduce
  - Increase activity on i2c bus
  - Poll temp / volt sensors every .1 second instead of 10 seconds
  - Hangs now happen every few minutes
- Step 2 - Observe
  - Xilinx FPGA block IP was encrypted - cannot instrument
  - Use logic analyzer to capture i2c waveforms

## No Title

- I2C Analyzer Example

- Yellow: SCL (clock)
- Green: SDA (data)
- Zoomed region
  - Note glitches on rise
- Causes 'extra' clock
  - Everything is off now
- More devices = slower rise
- Solution
  - Stronger pull-up
  - Faster rise-time
- Debugging I2C from TI:
- <https://www.ti.com/lit/an/slyt770/slyt770.pdf>



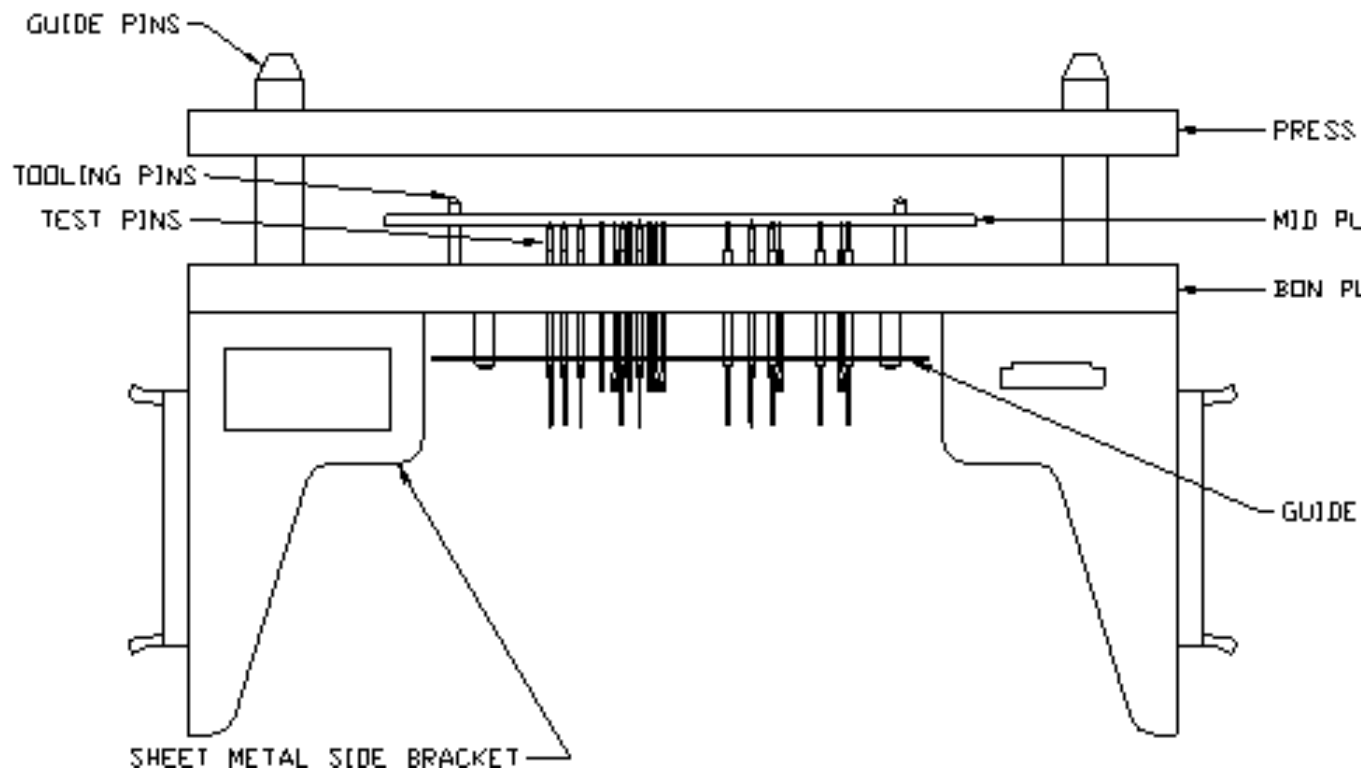
## 5.6 Debugging using JTAG

### Topics Discussed in this section

- Bed of Nails(Bon Tester)
- What is JTAG ???
- Advantages
- Boundary Scan logic and Architecture
- Real JTAG Applications

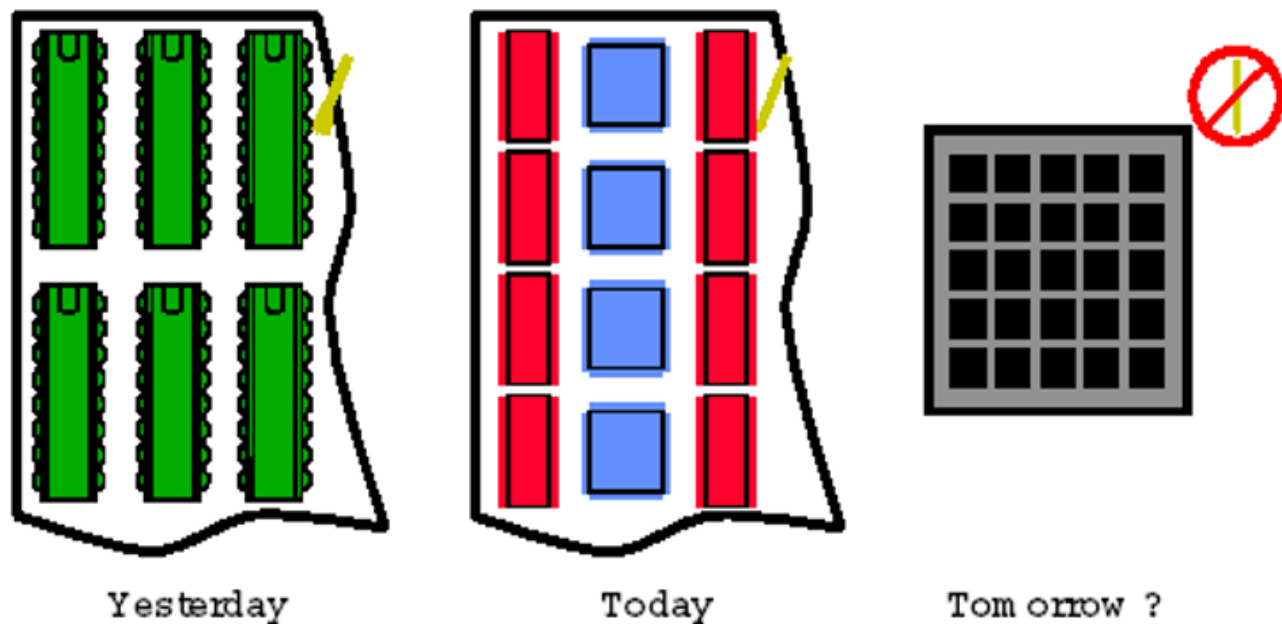
### Bed of Nails Fixture

- PCB Testing was done using Bed of Nails in circuit testing
- Every pad had to be contacted by a spring loaded pin.



Increased problem of test

■ Miniaturization results in loss of test access



Circuit Complexity

- Chips are becoming more and more complex
- Where there was a single logic unit in a chip there is now a complete microcontroller inside
- The ratio between functions and accessible pins has changed dramatically.

## Development Cost

- Cost will increase by a factor of ten as fault finding moves from one level of complexity to the next. The results are:
  - Reduced profit margins
  - Longer time to product introduction
  - Dissatisfied Customers
- 1. Device Level: 1 Unit of Cost
- 2. Board Level: 10 Units of Cost
- 3. System Level: 100 Units of Cost
- 4. Field Level: 1000 Units of Cost

## JTAG (1149.1 STANDARD)

- IEEE 1149.1, a standard 4 wire serial protocol protocol that established the details of access to any chip with a JTAG port .
- 
- Boundary scan testing of Integrated Circuits(ICs) and boards.
- Debug Embedded devices.
- System level debug capability
- 
- 

## What can it be used for?

- The standard defines instructions that can be used to perform :
- Functional Tests
- Interconnect tests
- Built-in self test procedures.

## Advantages

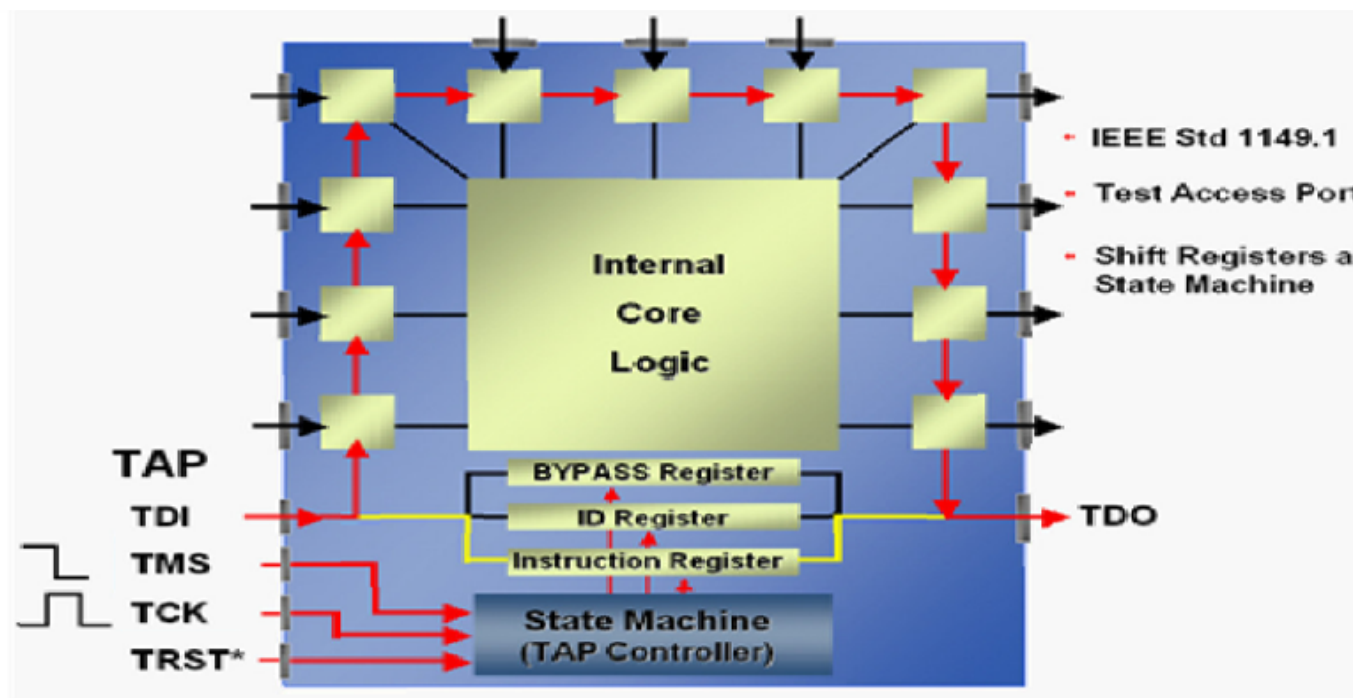
- The need for physical test points on the board is eliminated leading to significant savings as a result of simpler board layouts.
- Less costly test fixtures.

- reduced time on in-circuit test systems.
- increased use of standard interfaces.
- faster time-to-market.

## Boundary Scan

- What is Boundary Scan?
- Boundary scan is a methodology allowing complete controllability and observability of the boundary pins of a JTAG compatible device via software control.

## Boundary Scan Architecture



## BOUNDARY SCAN PRINCIPLE

- How does it work?
- The top level schematic of the test logic defined by IEEE Std 1149.1 includes three key blocks:
  - The TAP Controller
  - This responds to the control sequences supplied through the test access port (TAP) and generates the clock and control signals required for correct operation of the other circuit blocks.
  - The Instruction Register

- This shift register-based circuit is serially loaded with the instruction that selects an operation to be performed.
- The Data Registers
- These are a bank of shift register based circuits. The stimuli required by an operation are serially loaded into the data registers selected by the current instruction. Following execution of the operation, results can be shifted out for examination.

### **JTAG Test Access Port**

- The JTAG Test Access Port (TAP) contains four pins that drive the circuit blocks and control the operations specified. The TAP facilitates the serial loading and unloading of instructions and data.
- The four pins of the TAP are:
  - TMS - Test Mode Select
  - TCK - Test Clock
  - TDI - Test Data Input
  - TDO - Test Data Output

### **JTAG Test Access Port (Contd)**

- The function of each TAP pin is as follows:
- TCK - this pin is the JTAG test clock. It sequences the TAP controller as well as all of the JTAG registers
- TMS - this pin is the mode input signal to the TAP Controller. The state of TMS at the rising edge of TCK determines the sequence of states for the TAP controller.

### **JTAG Test Access Port (Contd)**

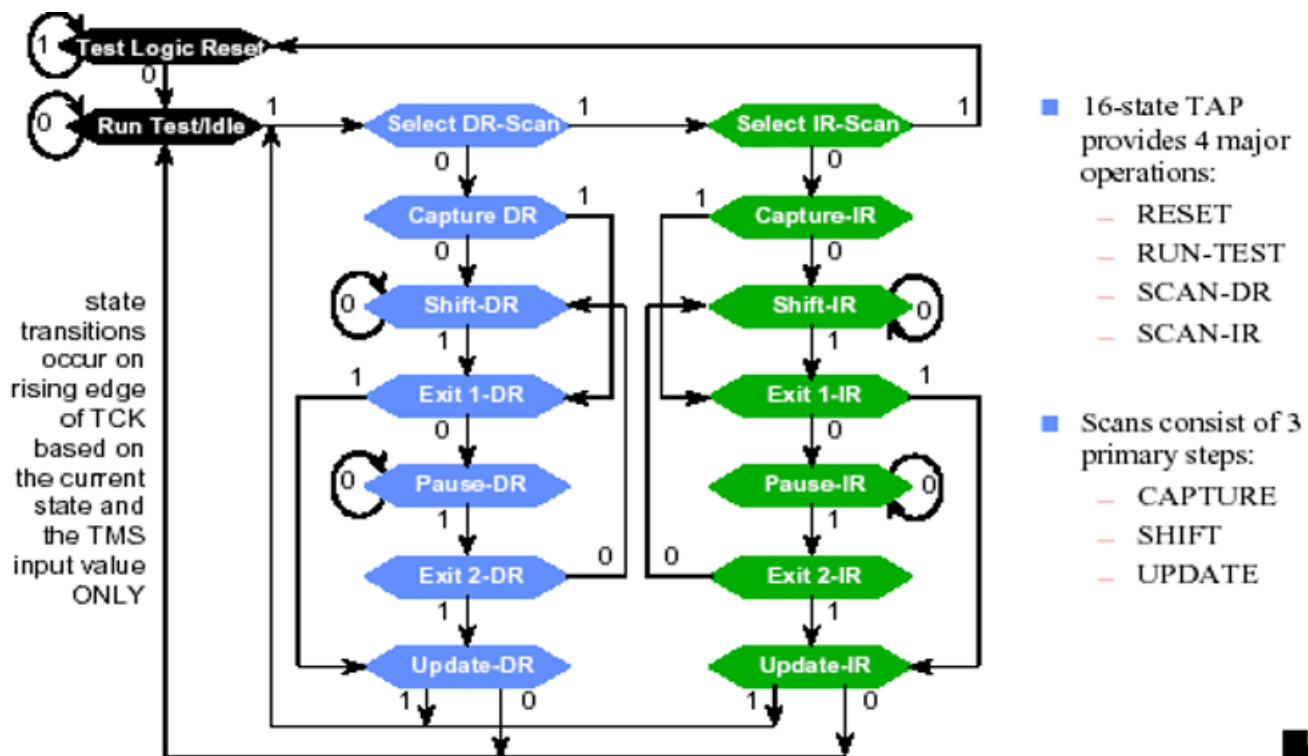
- TDI - this pin is the serial data input to all JTAG instruction and data registers. TDI is sampled into the JTAG registers on the rising edge of TCK.
- TDO - this pin is the serial data output for all JTAG instruction and data registers. TDO changes state on the falling edge of TCK and is only active during the shifting of data through the device. This pin is three-stated at all other times

### **JTAG TAP Controller**

- The JTAG TAP Controller is a
- 16-state finite state machine, that controls the scanning of data into the various registers of the JTAG architecture.
- The state of the TMS pin at the rising edge of TCK is responsible for determining the sequence of state transitions.

- There are two state transition paths for scanning the signal at TDI in to the device,
  - one for shifting in an instruction to the instruction register ,and ,
  - one for shifting data into the active data register as determined by the current instruction.

## JTAG TAP Controller logic



## Data Registers

- The Device ID register (IDR) reads-out an identification number which is hardwired into the chip.
- The Bypass register (BR) is a 1-cell pass-through register which connects the TDI to the TDO with a 1-clock delay to give test equipment easy access to another device in the test chain on the same board.
- The Boundary Scan register (BSR), intercepts all the signals between the core-logic and the pins.

## Test Process

- The standard test process for verifying a device or circuit board using boundary-scan technology is as follows:
- The tester applies test or diagnostic data on the input pins of the device.
- The boundary-scan cells capture the data in the boundary scan registers monitoring the input pins.

- Data is scanned out of the device via the TDO pin, for verification.
- Data can then be scanned into the device via the TDI pin.
- The tester can then verify data on the output pins of the device.

## **REAL JTAG APPLICATIONS**

- Scan access to chips, boards and systems for:
  - Desing verification and debugging
  - Manufacturing test
  - Hardware/Software integration
  - Field Test / Diagnostics
- Access buildt-in self-test (BIST)
- Access on-chip / in-circuit emulation (ONCE/ICE)
- Access in-system programming (ISP) of PLDs/EEPROMs
- Whatever else you can think of...

# Chapter 6

## OMAP - Application Prozessor

### 6.1 General Aspects

- OMAP-Application Prozessor provides a lot interfaces (like simple digital I/O but also complex systems as a camera interface)
- different interfaces are a key feature of every microcontroller system
- every interface is controlled by one or more registers, which are part of the standard memory space.
- most of the interfaces can generate an interrupt (which is handled according to an interrupt service routine and can be used to program some actions).
- every interface also has a flag which makes it possible to write programs to ask in a “polling method” for interaction with the interface.
- to reduce the number of external pins, mostly the input/output pins can be used in more than one mode.
- this means the number of interfaces are also reduced.

### 6.2 Digital Ports

- the OMAP-Application Prozessor has a lot of digital input/output lines (interfaces).
- these lines are grouped in 6 ports, every port with 32 input/output lines.
- every port is programmable as an input or an output port.
- a mix of input and output lines in one port is allowed.
- the output lines are driven a Hiz-mode (high impedance mode, this means a connection of two or more outputs lines are possible).
- 32 bit Ports (Programming direction)

| Port  | Port lines      | Direction        |
|-------|-----------------|------------------|
| GPIO1 | [31:0] (i/o)    | 0x48310034 (r/w) |
| GPIO2 | [63:34] (i/o)   | 0x49050034 (r/w) |
| GPIO3 | [95:64] (i/o)   | 0x49052034 (r/w) |
| GPIO4 | [127:96] (i/o)  | 0x49054034 (r/w) |
| GPIO5 | [159:128] (i/o) | 0x49056034 (r/w) |
| GPIO6 | [191:160] (i/o) | 0x49058034 (r/w) |

- after reset the direction is set to input (0xffffffff in the corresponding direction register)
- a bit reset on the correspondig bit in the direction register switch the direction to output
- 32 bit Ports (Read and write access)

| Port  | Port lines      | Input          | Output           |
|-------|-----------------|----------------|------------------|
| GPIO1 | [31:0] (i/o)    | 0x48310038 (r) | 0x4831003c (r/w) |
| GPIO2 | [63:34] (i/o)   | 0x49050038 (r) | 0x4905003c (r/w) |
| GPIO3 | [95:64] (i/o)   | 0x49052038 (r) | 0x4905203c (r/w) |
| GPIO4 | [127:96] (i/o)  | 0x49054038 (r) | 0x4905403c (r/w) |
| GPIO5 | [159:128] (i/o) | 0x49056038 (r) | 0x4905603c (r/w) |
| GPIO6 | [191:160] (i/o) | 0x49058038 (r) | 0x4905803c (r/w) |

- read access from input lines done as standard 32 bit access
- write access to output lines could be done as standard 32 bit write access
- a read access from the output lines gets the “old” contents (shadow register)
- with the registers “ClearDataOut” and “SetDataOut” an specific Clearbit- and Setbit-Operation is possible
- the specific bit which should be changed must be set in the correspondig “ClearDataOut” or SetDataOut“ register

| Port  | Port lines      | ClearDataOut     | SetDataOut       |
|-------|-----------------|------------------|------------------|
| GPIO1 | [31:0] (i/o)    | 0x48310090 (r/w) | 0x48310094 (r/w) |
| GPIO2 | [63:34] (i/o)   | 0x49050090 (r/w) | 0x49050094 (r/w) |
| GPIO3 | [95:64] (i/o)   | 0x49052090 (r/w) | 0x49052094 (r/w) |
| GPIO4 | [127:96] (i/o)  | 0x49054090 (r/w) | 0x49054094 (r/w) |
| GPIO5 | [159:128] (i/o) | 0x49056090 (r/w) | 0x49056094 (r/w) |
| GPIO6 | [191:160] (i/o) | 0x49058090 (r/w) | 0x49058094 (r/w) |

### 6.2.1 Push-Pull Ports

- The port can deliver large currents when set to ”high” (to source) and accept large currents when set to ”low” (sink).
- In the static case there is no current consumption
- Consequently signals can be switched at a high frequency (charging and discharging parasitic capacitances)
- Problem: When two push-pull ports are connected and set to different levels this creates a short from V++ to GND and fries the pin

### 6.2.2 Open-Collector / Drain Ports

- There is no possibility for shorts as in the Push-Pull case, even in the presence of programming and connection errors.
- Level changes can be implemented easily (3.3 - 5 V for example)
- At ”low” setting there is always a current flowing
- The flanks when switching are asymmetric
  - The low-high flank is relatively slow, as the current is limited by the large-ish pull-up resistor (RC-Charging curve)

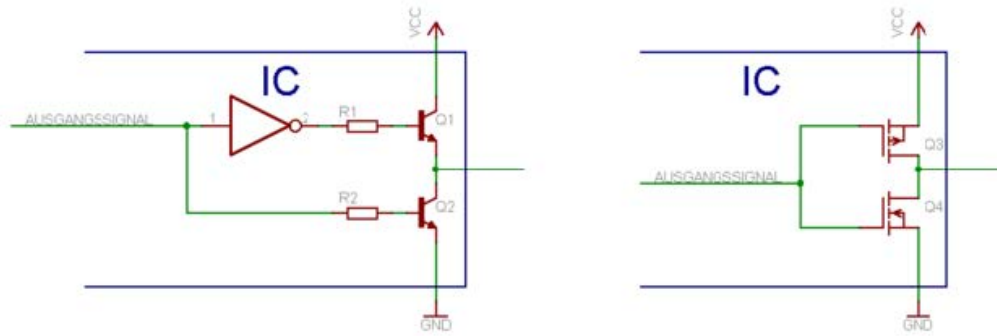


Figure 6.2.1: Port Configuration Push-Pull

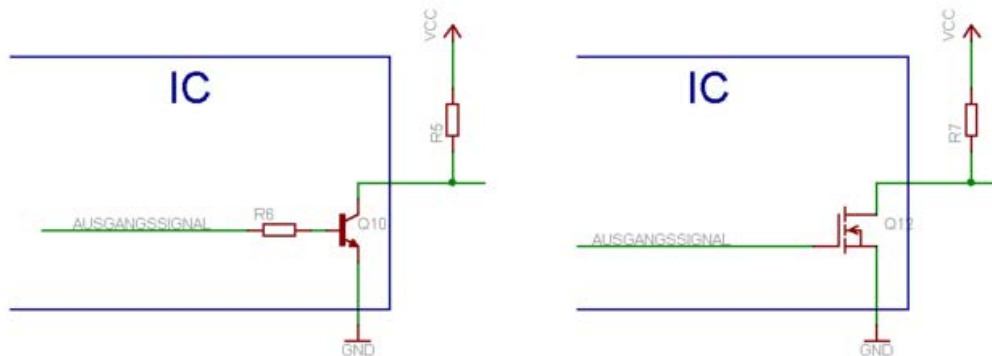


Figure 6.2.2: Port Configuration Open Collector / Open Drain

### 6.2.3 Tri-State Ports

- Besides "high" and "low" a port can also be set to high-impedance.
- This is usually called High-Z, Z or Tristate
- A port in this configuration presents itself as not being present.
- An arbitrary number of tristate ports can be connected.

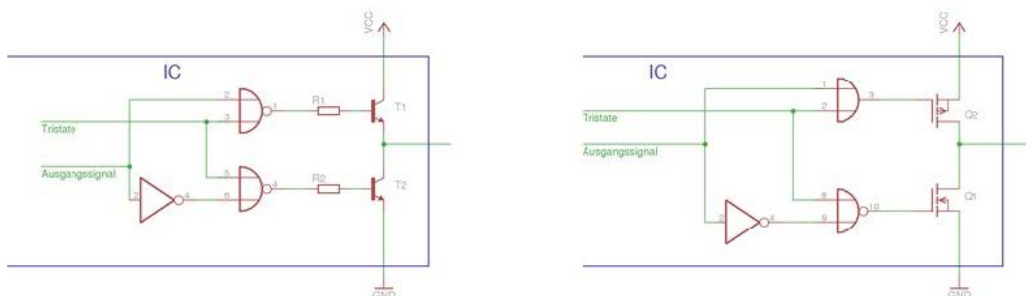


Figure 6.2.3: Port Configuration Tri-State

## 6.3 Serial Ports

- The OMAP-Application Prozessor supports different kinds of serial interfaces:

- programmable serial controller (UART) for standard asynchronous serial communication.
- universal serial bus (USB) for serial bus-oriented communication.
- inter-integrated circuit communication (I<sup>2</sup>C).

### UART universal asynchron receiver/transmitter

- 3 different UART-Channels:
  - UART 1, 2 asynchronous data transfer with 5,6,7,8 bits of data plus parity transfer.
  - UART 3 asynchronous data transfer IrDA
    - 1.0 SIR mode (SIR) with a baudrate in the range of 2400 - 115200 bps.
    - IrDA 1.1 MIR mode (MIR) with a baudrate ranging from 0.576 or 1.152 Mbps.
    - IrDA 1.1 FIR mode (FIR) with a baudrate of 4.0 Mbps.
- programmable baudrate generator

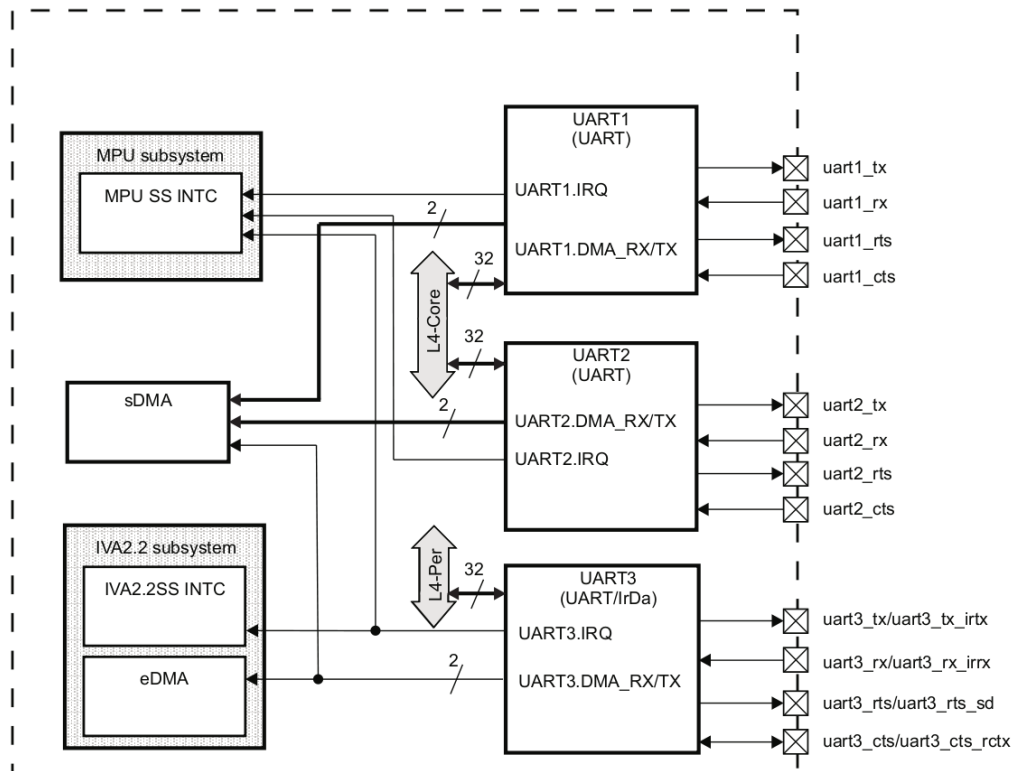


Figure 6.3.1: Blockdiagram UART

- the protocol of the UART-mode fix mostly a 8 bit transfer, 1 start bit and 1 or 2 stop bits.
- the protocol has to be the same for the transmit- and the receive-interface (no opportunity to check the protocol directly if the transmission failed).
- both the transmit- and the receive-interface can set flags and can generate interrupts if a transfer is finished.

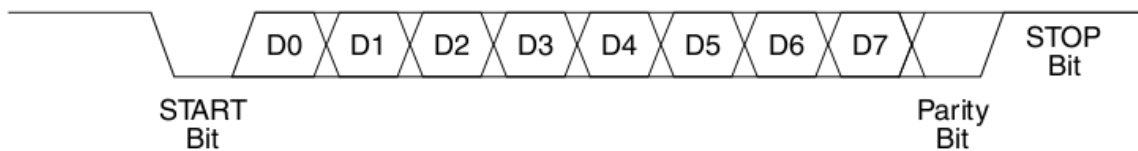


Figure 6.3.2: Protocol of a 8 Bit UART-Transfer

- no error-check methods in the UART-mode (only parity-checking is possible).
- the SIR-mode is similar to the UART-mode.
- every 0-bit generates an IR-pulse of a fixed duration (depends on the baudrate).

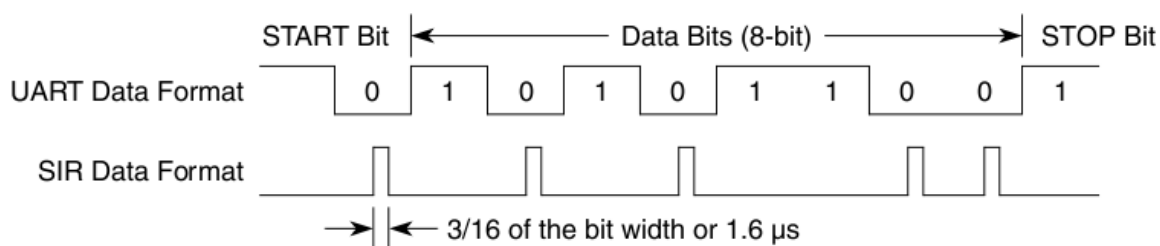


Figure 6.3.3: Protocol of a SIR-Transfer

- the MIR-mode has the same bit-coding as the SIR-mode, but with shorter IR-pulses (higher baudrates).
- in this mode the data is transferd in frames (HDLC-Frames)with a start-of-frame and stop-of-frame sequence (STA, STO).
- the frames are checked with an error check mechanism (CRC).

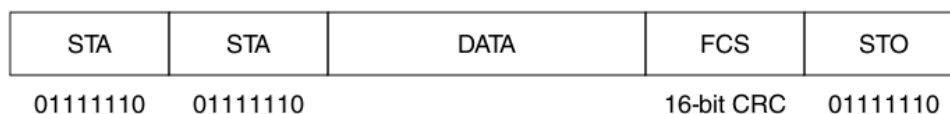
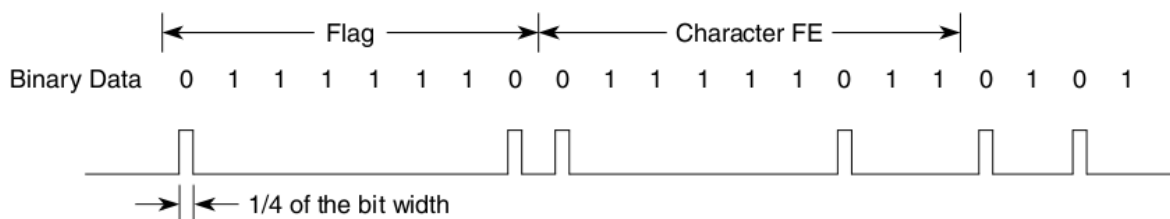


Figure 6.3.4: Protocol of a MIR-Transfer

- the FIR-mode use 4 PPM (4 pulse position modulation), each IR-pulse has a duration of 125 ns.
- the data transfer is done in frames.
- the frames are checked with an error check mechanism (CRC).

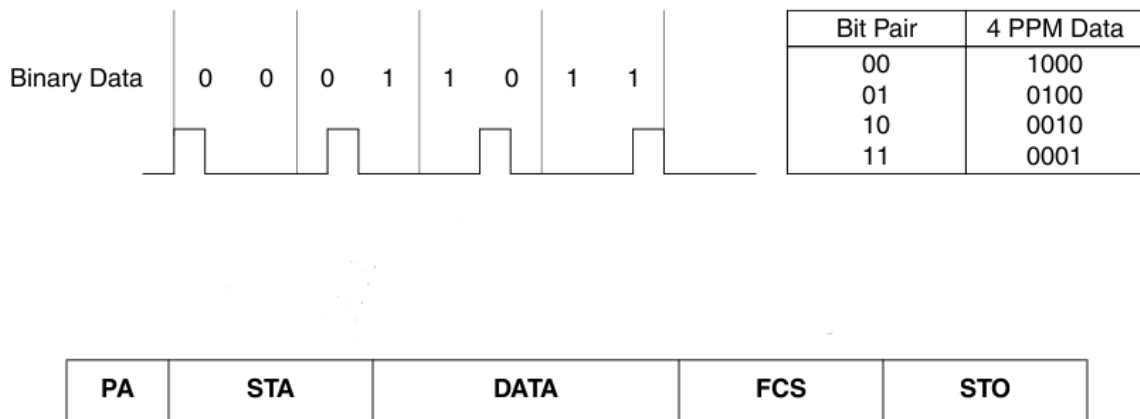


Figure 6.3.5: Protocol of a FIR-Transfer

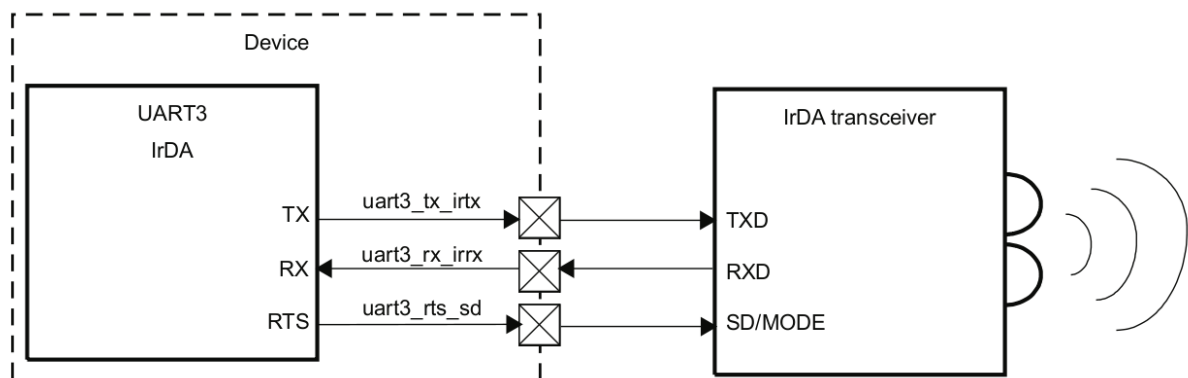


Figure 6.3.6: Example for an IrDA-Interface

## USB (universal serial bus)

- supports the high-speed operation up to 480 Mbps and full-speed operation at 12 Mbps (USB 2.0 Specification).
- burst-transfer, interrupt and isochronous transport modes.
- 4 kbytes of shared FIFO RAM and 1 kbyte descriptor RAM.

## I<sup>2</sup>C (inter-integrated circuit communication)

- an interface to connect hardware decices (e.g. LCD-Displays, keyboards, switches).
- multi-master operation.
- different serial clock frequencies are programmable.
- interrupt driven byte-by-byte data transfer possible.
- 3 different I<sup>2</sup>-Interfaces

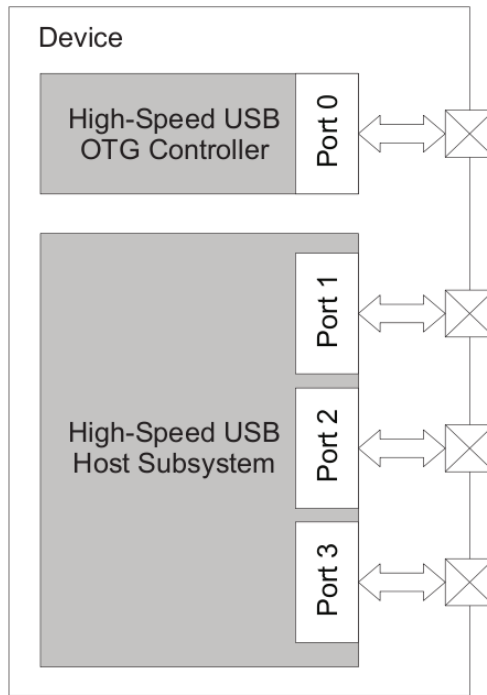


Figure 6.3.7: Blockdiagram USB

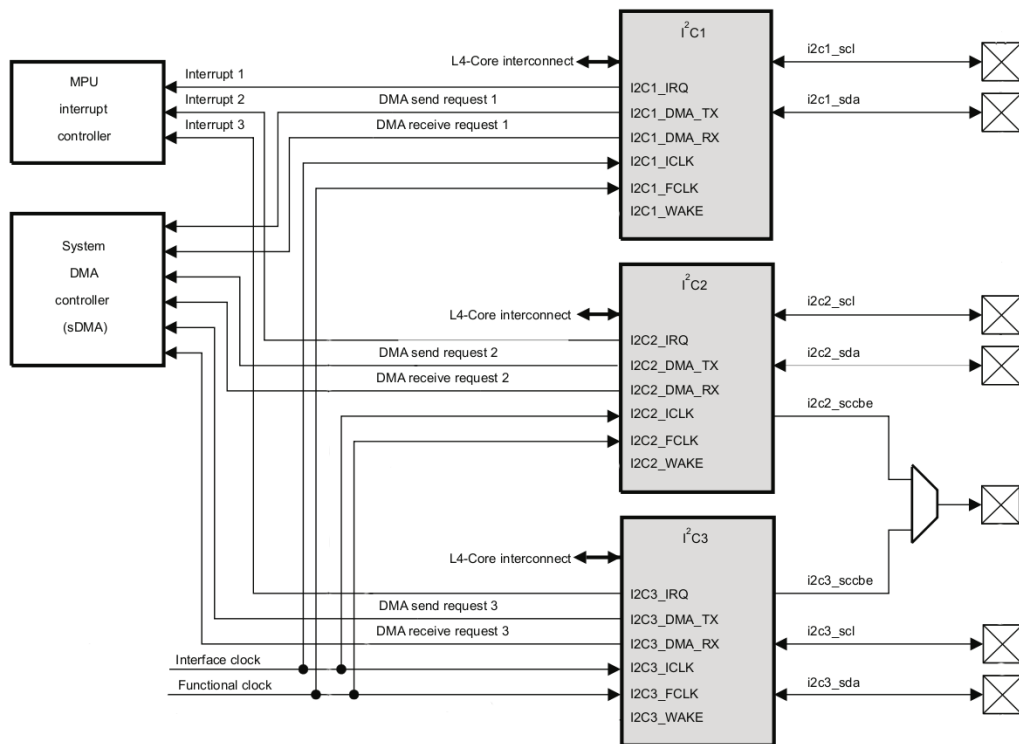


Figure 6.3.8: I2C-Blockdiagram

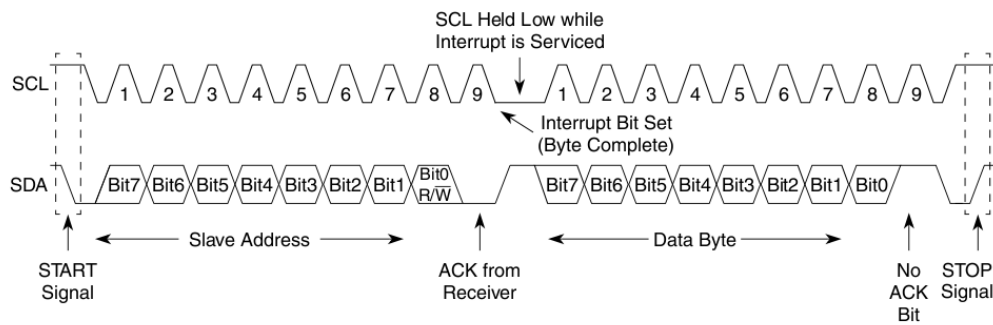


Figure 6.3.9: Typical Data Transfer Protocol of a I2C-Interface

## 6.4 Timers

### GPT (general purpose timer)

- implementation of 11 independent timer channels
- every timer has a wordlength of 32 bit and operates as a free running timer.
- the timer channels could operate in:
  - an input capture-mode (in this mode the counters run until the specified capture event occurs (rise, fall, or pulse) on appropriate Event\_Capture\_IN-signal)
  - the capture value are stored in a specific TCAR-register and could be read
  - an output compare-mode (in this mode the counters run until they reach the programmed terminal count value. At this point, the specified output event is generated (toggle, pulse high, or pulse low) on PWM\_OUT)
  - the compare value are stored in a specific TMAR-register
  - a PWM-mode (in this mode the user can program period and width values to create an adjustable, repeating output waveform on PWM\_OUT)
  - PWM-mode: suitable to emulate an analogue output signal (with an additional operational amplifier).

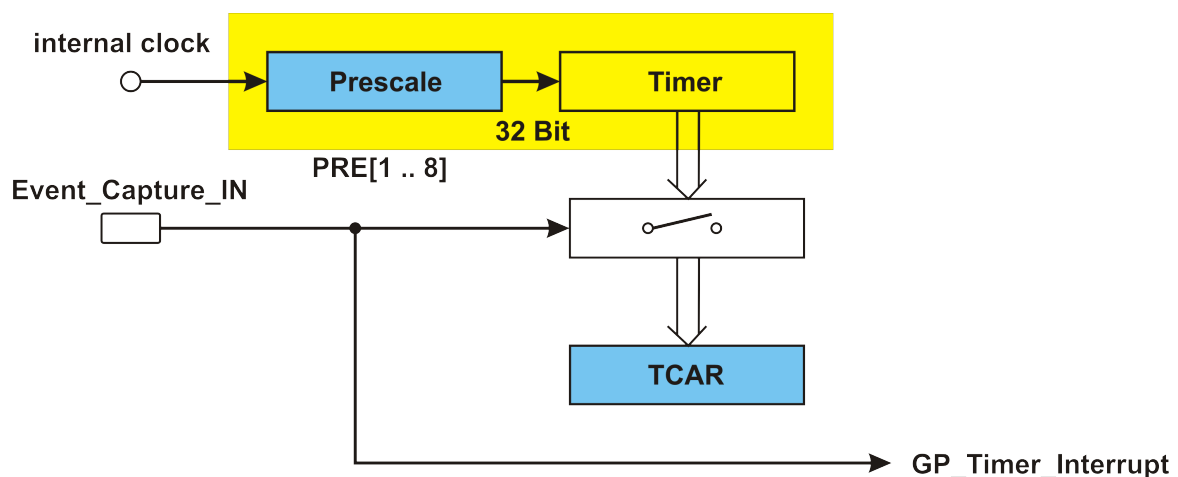


Figure 6.4.1: Blockdiagram of the Input Capture Mode

### Watchdog Timers

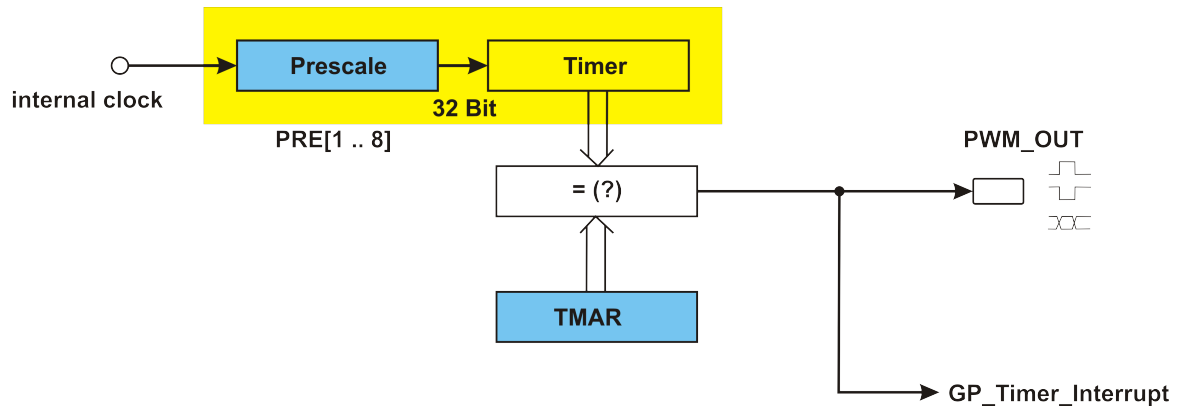


Figure 6.4.2: Blockdiagram of the Output Compare Mode

- two different timers with a specific clock signal (32 KHz)
- each timer consists of a 32-bit counter with an additional prescale (1 .. 128)
- the timers generate a special reset (watchdog) and a special interrupt signal (if not reloaded in time)
  - after they have reached zero, the timers reload and start the counting procedure one's more.

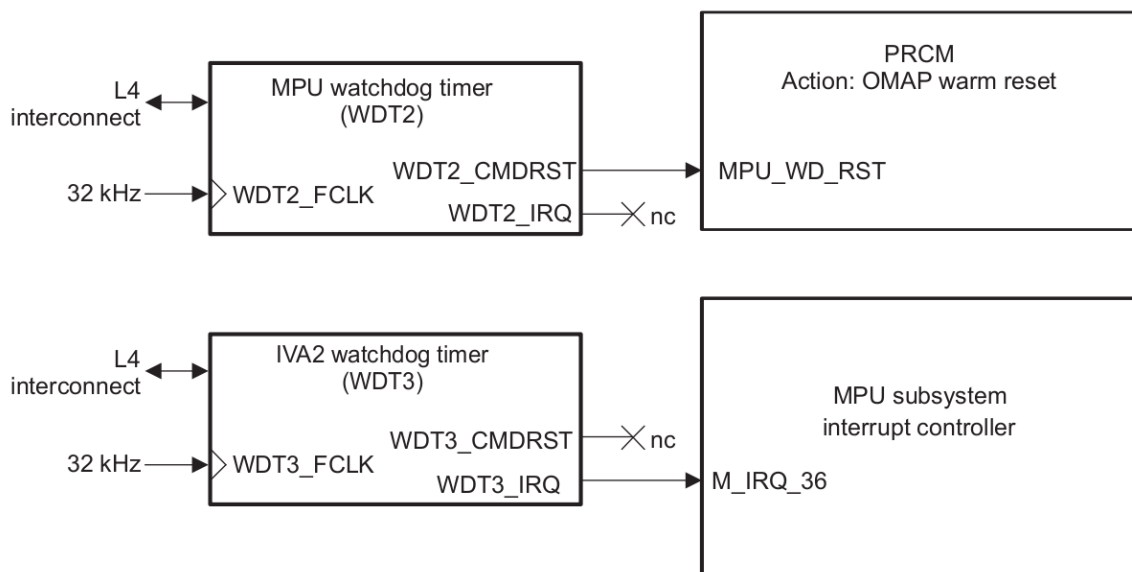


Figure 6.4.3: Blockdiagram Watchdog Timers

# Chapter 7

## Operating Systems

### 7.1 Operating Systems for Embedded Systems

Advances in microcontroller technology continue to open up new application fields for embedded systems. In parallel with this trend, the equipment controlled by embedded systems has become more sophisticated, often incorporating many functions in one product. As a result, embedded systems have grown in scale and complexity. Moreover, as products increasingly adopt digital technology, advanced processors have enabled more of the processing to be implemented in software, making embedded systems all the more important.

This means, in new application the developer is more and more interested to do the software development with the support of an operating system. Especially in large scale systems the effect on reducing development cost with an operating system is important.

Even in the small-scale embedded system field, the growing scale and complexity of software and the need for fast development turnaround time have made improving software productivity a pressing need. The use of C and other high-level languages have become increasingly common for this reason.

Real time operating systems fulfil all the requirements from the developer. They guarantee a response of the system in specific time. The development cycle becomes much shorter if the user could do the development on the target. Cross-development involves the downloading of the executable program on the target, testing it, changing it and repeating the same procedure once more for the next error is not an optimal development cycle. Real time operating systems offer a good support for the debugging.

There exist only a few drawbacks if you are looking for appropriate real time operating system. The first most important disadvantages of the most real time operating systems are the costs. The second one is that real time operating systems need virtual memory system which is in special not available in small microcontrollers.

The virtual memory system itself is really suitable for general purpose applications like a desktop PC. But for embedded systems a virtual memory system makes the application more difficult. In such systems really often an access to the hardware is necessary (control interfaces). These programming aspects could only be done in the so called “kernel-mode” with the result that such programs more dangerous (collapse of the operating system) and more difficult to test.

In this chapter we will discuss some aspects of operating systems and than come to the uCLinux or other “embedded Linux Systems”, operating systems for microcontroller with and without a virtual memory system.

- a lot of embedded systems runs without a operating system
  - only small algorithm (e.g. 'control the window' open/close system in a car).

- hard restrictions in the size of memory especially by 8- and 16-bit microcontrollers.
- costs of an operating system.
- reasons for the use of an operating system
  - generally the same reasons why we need one for a traditional computer.
  - but in the case of embedded systems not all services of an operating systems are needed for any device.
  - embedded systems have a large variety of requirements and environments.
    - critical applications with high functionality (medical applications, aircraft systems ..).
    - critical applications with small functionality (ABS, cardiac pacemaker ..).
    - not very critical applications with varying functionality (mobile phone microwave oven, washing machine ..).

### Special operating system (OS) for embedded systems

- a standard OS (like Windows or Linux) isn't suitable because :
  - monolithic kernel has too many features which are not necessary in a specific application.
  - monolithic kernel is not modular, fault-tolerant, configurable, modifiable.
  - needs too much memory space (virtual memory).
  - not designed for mission-critical applications.

### Key features of an OS for embedded systems

- configurability → no single OS will fit all needs, so configurability needed
  - simplest form → remove unused functions (by linker ?)
  - conditional compilation (using #if and #ifdef commands)
  - dynamic data might be replaced by static data
  - removes unnecessary functions or modules by a kernel programming (Linux, eCos, VxWORKS).
- handling of device drivers
  - access to interfaces.
  - device drivers handled by tasks instead of integrated drivers.
  - improve predictability; everything goes through scheduler.
  - in embedded systems mostly a lot of different interfaces are available, so not every device needs support by all versions of the OS.
- handling of interrupts
  - for standard OS: serious source of unreliability.
  - it should be possible to let interrupts directly start or stop tasks (by storing the tasks start address in the interrupt table).

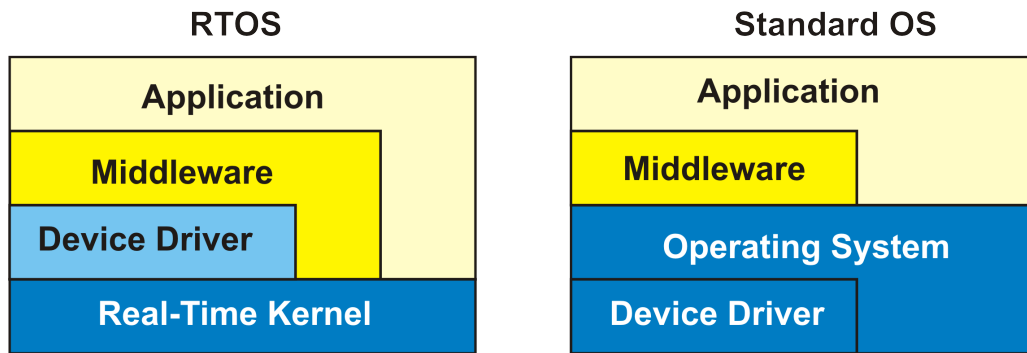


Figure 7.1.1: Integration of Devices Drivers in an OS

### Available OS for embedded systems

- **QNX**
  - real time operating system based on micro core technology.
  - suitable for small and large embedded systems.
  - tasks are separated in its one address space.
  - scheduling event driven (restrictions in time based scheduling) .
- **VxWORKS**
  - small VxWorks Kernel.
  - no virtual memory space (dangerous if a task is not cooperative).
  - scheduling time, priority and event driven.
  - implemented on a lot of microcontrollers.
- **OSEK/VDX**
  - scalable real time OS build for application in the automotive area.
  - event driven execution of tasks.
  - operates with fixed priorities.
  - tasks are grouped in basic tasks and extended tasks.
  - no virtual memory system.
- **eCos**
  - open source system.
  - scalable system (the configuration allows the application writer to impose their requirements on the run-time component).
  - OS and application software are specific for an embedded system.
  - scheduling is time and event driven.
  - no virtual memory system.
- **embedded Linux**
  - open source system.
  - scalable and portable (especially for 32 bit microcontrollers).
  - run with a non virtual and a virtual memory system.
  - scheduling is time and priority driven.
  - with the RTAI (real time application interface) also a event driven scheduling mechanism possible.
  - for small and large embedded systems.

## 7.2 Functions of RTOS

### FreeRTOS

- FreeRTOS is a real-time kernel/scheduler on top of which MCU applications can be built to meet their hard real-time requirements.
  - Allows MCU applications be organized as a collection of independent threads of execution.
  - Decides which thread should be executed by examining the priority assigned to each thread.
    - Assume a single core MCU where only a single thread can be executing at one time.

### The simplest case of task priority assignments

- Assign higher priorities (lower priorities) to threads that implement hard real-time (soft real-time) requirements
  - As a result hard real-time threads are always executed ahead of soft real-time threads.
- But priority assignment decision are not always that simple.
- In general task prioritization can help ensure an application meet its processing deadline.

### Why use a real-time kernel

- For a simple system many well-established techniques can provide an appropriate solution without the use of a kernel.
- For a more complex embedded application a kernel would be preferable.
- But where the crossover point occurs will always be subjective.
- Besides ensuring an application meets its processing deadline a kernel can bring other less obvious benefits.

### Benefits of using real-time kernel 1

- Abstracting away timing information
  - Kernel is responsible for execution timing and provides a time-related API to the application. This allows the application code to be simpler and the overall code size be smaller.
- Maintainability/Extensibility
  - Abstracting away timing details results in fewer interdependencies between modules and allows sw to evolve in a predictable way.
  - Application performance is less susceptible to changes in the underlying hardware.
- Modularity
  - Tasks are independent modules each of which has a well-defined purpose.
- Team development
  - Tasks have well-defined interfaces allowing easier development by teams

### Benefits of using real-time kernel 2

- Easier testing
  - Tasks are independent modules with clean interfaces they can be tested in isolation.
- Idle time utilization
  - The idle task is created automatically when the kernel is started. It executes whenever there are no application tasks to run.
  - Be used to measure spare processing capacity perform background checks or simply place the process into a low-power mode.
- Flexible interrupt handling
  - Interrupt handlers can be kept very short by deferring most of the required processing to handler tasks.

### Standard FreeRTOS features

- Pre-emptive or co-operative operation
- Very flexible task priority assignment
- Queues
- Binary/Counting / Recursive semaphores
- Mutexes
- Tick/Idle hook functions
- Stack overflow checking
- Trace hook macros
- Interrupt nesting

### FreeRTOS - RT and Embedded

- "Embedded" and "real-time" can mean different things to different people, so let's define them as FreeRTOS uses them.
- An embedded system is a computer system that is designed to do only a few things, like the system in a TV remote control, in-car GPS, digital watch, or pacemaker. Embedded systems are typically smaller and slower than general purpose computer systems, and are also usually less expensive. A typical low-end embedded system may have an 8-bit CPU running at 25MHz, a few KB of RAM, and maybe 32KB of flash memory. A higher-end embedded system may have a 32-bit CPU running at 750MHz, a GB of RAM, and multiple GB of flash memory.
- Real-time systems are designed to do something within a certain amount of time; they guarantee that stuff happens when it's supposed to.
- A pacemaker is an excellent example of a real-time embedded system. A pacemaker must contract the heart muscle at the right time to keep you alive; it can't be too busy to respond in time. Pacemakers and other real-time embedded systems are carefully designed to run their tasks on time, every time.

### Architecture

- FreeRTOS is a relatively small application. The minimum core of FreeRTOS is only three source (.c) files and a handful of header files, totalling just under 9000 lines of code, including comments and blank lines. A typical binary code image is less than 10KB.

- FreeRTOS's code breaks down into three main areas: tasks, communication, and hardware interfacing.
- Tasks: Almost half of FreeRTOS's core code deals with the central concern in many operating systems: tasks. A task is a user-defined C function with a given priority. `tasks.c` and `task.h` do all the heavy lifting for creating, scheduling, and maintaining tasks.
- Communication: Tasks are good, but tasks that can communicate with each other are even better! Which brings us to the second FreeRTOS job: communication. About 40% of FreeRTOS's core code deals with communication. `queue.c` and `queue.h` handle FreeRTOS communication. Tasks and interrupts use queues to send data to each other and to signal the use of critical resources using semaphores and mutexes.
- The Hardware Whisperer: The approximately 9000 lines of code that make up the base of FreeRTOS are hardware-independent; the same code runs whether FreeRTOS is running on the humble 8051 or the newest, shiniest ARM core. About 6% of FreeRTOS's core code acts a shim between the hardware-independent FreeRTOS core and the hardware-dependent code. We'll discuss the hardware-dependent code in the next section.

## Hardware Considerations

- The hardware-independent FreeRTOS layer sits on top of a hardware-dependent layer. This hardware-dependent layer knows how to talk to whatever chip architecture you choose.

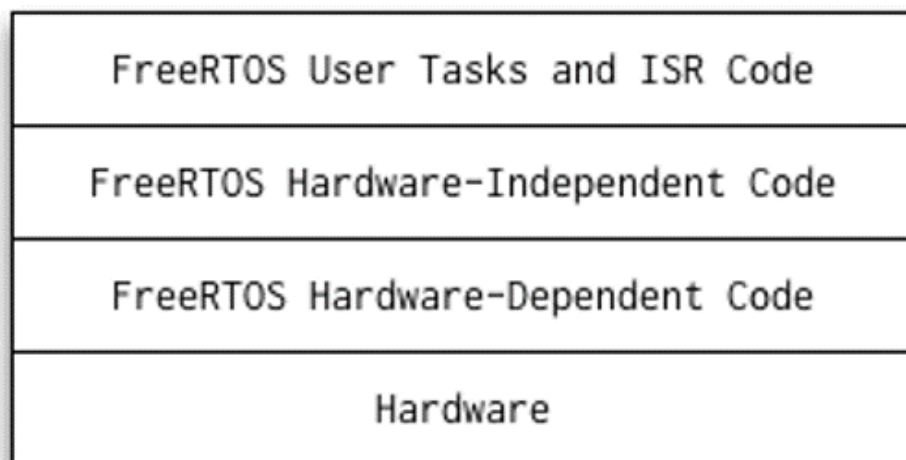


Figure 7.2.1: FreeRTOS Layers

## Configurations

- Configuration options are selected in `FreeRTOSConfig.h` by setting various `#defines`. Clock speed, heap size, mutexes, and API subsets are all configurable in this file, along with many other options. Here are a few examples that set the maximum number of task priority levels, the CPU frequency, the system tick frequency, the minimal stack size and the total heap size:

```

1 #define configMax_PRIORITIES ((unsigned portBASE_TYPE)5)
2 #define configCPU_CLOCK_HZ (12000000UL)
3 #define configTICK_RATE_HZ ((portTickType) 1000)
4 #define configMINIMAL_STACK_SIZE ((unsigned short) 100)
5 #define configTOTAL_HEAP_SIZE ((size_t) (4 * 1024))

```

## Hardware Dependency

- Hardware-dependent code lives in separate files for each compiler toolchain and CPU architecture.
- For example, if you're working with the IAR compiler on an ARM Cortex-M3 chip, the hardware-dependent code lives in the FreeRTOS/Source/portable/IAR/ARM\_CM3/ directory.
- portmacro.h declares all of the hardware-specific functions, while port.c and portasm.s contain all of the actual hardware-dependent code.
- The hardware-independent header file portable.h #include's the correct portmacro.h file at compile time. FreeRTOS calls the hardware-specific functions using #define'd functions declared in portmacro.h.

## Example

- Let's look at an example of how FreeRTOS calls a hardware-dependent function. The hardware-independent file tasks.c frequently needs to enter a critical section of code to prevent preemption. Entering a critical section happens differently on different architectures, and the hardware-independent tasks.c does not want to have to understand the hardware-dependent details.
- So tasks.c calls the global macro portENTER\_CRITICAL(), glad to be ignorant of how it actually works.
- Assuming we're using the IAR compiler on an ARM Cortex-M3 chip, FreeRTOS is built with the file
- FreeRTOS/Source/portable/IAR/ARM\_CM3/portmacro.h which defines portENTER\_CRITICAL like this:
- #define portENTER\_CRITICAL() vPortEnterCritical()

## Example 2

- vPortEnterCritical() is actually defined in
  - FreeRTOS/Source/portable/IAR/ARM\_CM3/port.c.
  - The port.c file is hardware-dependent, and contains code that understands the IAR compiler and the Cortex-M3 chip.
  - vPortEnterCritical() enters the critical section using this hardware-specific knowledge and returns to the hardware-independent tasks.c.
- The portmacro.h file also defines an architecture's basic data types. Data types for basic integer variables, pointers, and the system timer tick data type are defined like this for the IAR compiler on ARM Cortex-M3 chips:

```

1 \ite #define portBASE_TYPE long // Basic integer
 variable type
2 \ite #define portSTACK_TYPE unsigned long // Pointers to memory
 locations

```

```

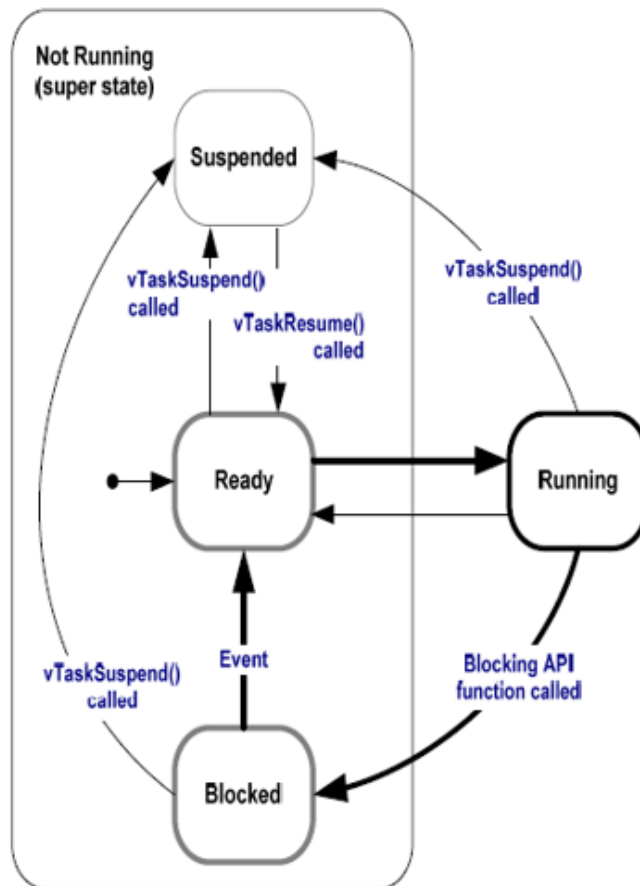
3 | \ite typedef unsigned portLONG portTickType; // The system timer
 | tick type

```

### Example 3

- This method of using data types and functions through thin layers of #defines may seem a bit complicated, but it allows FreeRTOS to be recompiled for a completely different system architecture by changing only the hardware-dependent files. And if you want to run FreeRTOS on an architecture it doesn't currently support, you only have to implement the hardware-dependent functionality which is much smaller than the hardware-independent part of FreeRTOS.
- As we've seen, FreeRTOS implements hardware-dependent functionality with C pre-processor #define macros. FreeRTOS also uses #define for plenty of hardware-independent code. For non-embedded applications this frequent use of #define is a cardinal sin, but in many smaller embedded systems the overhead for calling a function is not worth the advantages that "real" functions offer.

### Full task state machine



### Tasks - Task Control Block in tasks.c

```

1 |
2 | typedef struct tskTaskControlBlock
3 | {
4 | volatile portSTACK_TYPE *pxTopOfStack; /* Points to the location
 of the last item placed on the tasks stack. THIS MUST BE THE FIRST
 MEMBER OF THE STRUCT. */

```

```

5
6 xListItem xGenericListItem; /* List item
 used to place
7
 ready and
8
 blocked
 queues. */
9 xListItem xEventListItem; /* List item
 used to place
10
 the TCB in
 event lists. */
11 unsigned portBASE_TYPE uxPriority; /* The
 priority of the task
12
 where 0 is
13
 the lowest
 priority.
 */
14 portSTACK_TYPE *pxStack; /* Points to
 the start of
15
 the stack.
 */
16 signed char pcTaskName[configMAX_TASK_NAME_LEN]; /*
 Descriptive name given
17
 to the
 task when created.
18
 Facilitates debugging
19
 only. */
20
21 #if (portSTACK_GROWTH > 0)
22 portSTACK_TYPE *pxEndOfStack; /* Used for
 stack overflow
23
 checking
24
 on architectures
 where the
25
 stack grows up
 from low
 memory. */
26 #endif
27
28 #if (configUSE_MUTEXES == 1)
29 unsigned portBASE_TYPE uxBasePriority; /* The
 priority last
30
 assigned
31
 to the task -
 used by
32
 the priority
 inheritance mechanism. */
33 #endif
34
35 } tskTCB;

```

## Scheduling Tasks - Priorities and Ready List

- Each task has a user-assigned priority between 0 (the lowest priority) and the compile-time value of configMAX\_PRIORITIES-1 (the highest priority). For in-

stance, if configMAX\_PRIORITIES is set to 5, then FreeRTOS will use 5 priority levels: 0 (lowest priority), 1, 2, 3, and 4 (highest priority).

- FreeRTOS uses a "ready list" to keep track of all tasks that are currently ready to run. It implements the ready list as an array of task lists like this:

```
1 static xList pxReadyTasksLists[configMax\ _PRIORITIES]; /* Prioritised
 ready tasks. */
2 pxReadyTasksLists[0] is a list of all ready priority 0 tasks,
3 pxReadyTasksLists[1] is a list of all ready priority 1 tasks, and so on,
 all the way up to pxReadyTasksLists[configMax\ _PRIORITIES-1].
```

## Scheduling Tasks - System Tick

- The heartbeat of a FreeRTOS system is called the system tick. FreeRTOS configures the system to generate a periodic tick interrupt. The user can configure the tick interrupt frequency, which is typically in the millisecond range. Every time the tick interrupt fires, the vTaskSwitchContext() function is called. vTaskSwitchContext() selects the highest-priority ready task and puts it in the pxCurrentTCB variable like this:

```
1 /* Find the highest-priority queue that contains ready tasks. */
2 while(listLIST_IS_EMPTY(\&(amp; pxReadyTasksLists[uxTopReadyPriority
])))
3 {
4 configASSERT(uxTopReadyPriority);
5 --uxTopReadyPriority;
6 }
7
8 /* listGET_OWNER_OF_NEXT_ENTRY walks through the list, so the tasks
 of the same
9 priority get an equal share of the processor time. */
10 listGET_OWNER_OF_NEXT_ENTRY(pxCurrentTCB, \&(amp; pxReadyTasksLists[
 uxTopReadyPriority]));
```

## Scheduling Tasks - Ready List

- Before the while loop starts, uxTopReadyPriority is guaranteed to be greater than or equal to the priority of the highest-priority ready task. The while() loop starts at priority level uxTopReadyPriority and walks down through the pxReadyTasksLists array to find the highest-priority level with ready tasks. listGET\_OWNER\_OF\_NEXT\_ENTRY() then grabs the next ready task from that priority level's ready list.
- Now pxCurrentTCB points to the highest-priority task, and when vTaskSwitchContext() returns the hardware-dependent code starts running that task.
- Those nine lines of code are the absolute heart of FreeRTOS. The other 8900+ lines of FreeRTOS are there to make sure those nine lines are all that's needed to keep the highest-priority task running.

## Tasks - TCB

- The TCB stores the address of the stack start address in pxStack and the current top of stack in pxTopOfStack. It also stores a pointer to the end of the stack in

pxEndOfStack to check for stack overflow if the stack grows "up" to higher addresses. If the stack grows "down" to lower addresses then stack overflow is checked by comparing the current top of stack against the start of stack memory in pxStack.

- The TCB stores the initial priority of the task in uxPriority and uxBasePriority. A task is given a priority when it is created, and a task's priority can be changed. If FreeRTOS implements priority inheritance then it uses uxBasePriority to remember the original priority while the task is temporarily elevated to the "inherited" priority. (See the discussion about mutexes below for more on priority inheritance.)
- Each task has two list items for use in FreeRTOS's various scheduling lists. When a task is inserted into a list FreeRTOS doesn't insert a pointer directly to the TCB. Instead, it inserts a pointer to either the TCB's xGenericListItem or xEventListItem. These xListItem variables let the FreeRTOS lists be smarter than if they merely held a pointer to the TCB. We'll see an example of this when we discuss lists later.
- A task can be in one of four states: running, ready to run, suspended, or blocked. You might expect each task to have a variable that tells FreeRTOS what state it's in, but it doesn't. Instead, FreeRTOS tracks task state implicitly by putting tasks in the appropriate list: ready list, suspended list, etc. The presence of a task in a particular list indicates the task's state. As a task changes from one state to another, FreeRTOS simply moves it from one list to another.

## Tasks Setup

- We've already touched on how a task is selected and scheduled with the pxReadyTasksLists array; now let's look at how a task is initially created. A task is created when the xTaskCreate() function is called. FreeRTOS uses a newly allocated TCB object to store the name, priority, and other details for a task, then allocates the amount of stack the user requests (assuming there's enough memory available) and remembers the start of the stack memory in TCB's pxStack member.
- The stack is initialized to look as if the new task is already running and was interrupted by a context switch. This way the scheduler can treat newly created tasks exactly the same way as it treats tasks that have been running for a while; the scheduler doesn't need any special case code for handling new tasks.
- The way that a task's stack is made to look like it was interrupted by a context switch depends on the architecture FreeRTOS is running on, but this ARM Cortex-M3 processor's implementation is a good example:

## Tasks - Stack

```
1 unsigned int *pxPortInitialiseStack(unsigned int *pxTopOfStack,
2 pdTASK_CODE pxCode,
3 void *pvParameters)
4 {
5 /* Simulate the stack frame as it would be created by a context
6 switch interrupt. */
7 pxTopOfStack--; /* Offset added to account for the way the MCU uses
8 the stack on
9 entry/exit of interrupts. */
10 *pxTopOfStack = portINITIAL_XPSR; /* xPSR */
11 pxTopOfStack--;
12 *pxTopOfStack = (portSTACK_TYPE) pxCode; /* PC */
13 pxTopOfStack--;
14 *pxTopOfStack = 0; /* LR */
15 pxTopOfStack -= 5; /* R12, R3, R2 and R1. */
16 *pxTopOfStack = (portSTACK_TYPE) pvParameters; /* R0 */
```

```

15 pxTopOfStack -= 8; /* R11, R10, R9, R8, R7, R6, R5 and R4. */
16
17 return pxTopOfStack;
18 }

```

## Tasks - Stack

- The ARM Cortex-M3 processor pushes registers on the stack when a task is interrupted. `pxPortInitialiseStack()` modifies the stack to look like the registers were pushed even though the task hasn't actually started running yet. Known values are stored to the stack for the ARM registers xPSR, PC, LR, and R0. The remaining registers R1 – R12 get stack space allocated for them by decrementing the top of stack pointer, but no specific data is stored in the stack for those registers. The ARM architecture says that those registers are undefined at reset, so a (non-buggy) program will not rely on a known value.
- After the stack is prepared, the task is almost ready to run. First though, FreeRTOS disables interrupts: We're about to start mucking with the ready lists and other scheduler structures and we don't want anyone else changing them underneath us.
- If this is the first task to ever be created, FreeRTOS initializes the scheduler's task lists. FreeRTOS's scheduler has an array of ready lists, `pxReadyTasksLists`, which has one ready list for each possible priority level. FreeRTOS also has a few other lists for tracking tasks that have been suspended, killed, and delayed. These are all initialized now as well.
- After any first-time initialization is done, the new task is added to the ready list at its specified priority level. Interrupts are re-enabled and new task creation is complete.

## Lists

- The FreeRTOS list is a standard circular doubly linked list with a couple of interesting additions. Here's a list element:

```

1 struct xLIST_ITEM
2 {
3 portTickType xItemValue; /* The value being listed.
 In most cases used to sort the list in descending order. */
4 volatile struct xLIST_ITEM * pxNext; /* Pointer to the next
 xListItem in the list. */
5 volatile struct xLIST_ITEM * pxPrevious; /* Pointer to the
 previous xListItem in the list. */
6 void * pvOwner; /* Pointer to the object (
 normally a TCB) that contains the list item. There is therefore
 a two-way link between the object containing the list item and
 the list item itself. */
7 void * pvContainer; /* Pointer to the list in
 which this list item is placed (if any). */
8 };

```

## Lists

- Each list element holds a number, `xItemValue`, that is the usually the priority of the task being tracked or a timer value for event scheduling. Lists are kept in high-to-low priority order, meaning that the highest-priority `xItemValue` (the largest number) is at the front of the list and the lowest priority `xItemValue` (the smallest number) is at the end of the list.
- The `pxNext` and `pxPrevious` pointers are standard linked list pointers. `pvOwner` is a pointer to the owner of the list element. This is usually a pointer to a task's TCB object. `pvOwner` is used to make task switching fast in `vTaskSwitchContext()`: once the highest-priority task's list element is found in `pxReadyTasksLists`, that list element's `pvOwner` pointer leads us directly to the TCB needed to schedule the task.
- `pvContainer` points to the list that this item is in. It is used to quickly determine if a list item is in a particular list. Each list element can be put in a list, which is defined as:

```

1 typedef struct xLIST
2 {
3 volatile unsigned portBASE\ _TYPE uxNumberOfItems;
4 volatile xListItem * pxIndex; /* Used to walk through
 the list. Points to
5 the last item returned
 by a call to
6
7 pvListGetOwnerOfNextEntry (). */
8 volatile xMiniListItem xListEnd; /* List item that
 contains the maximum
9 possible item value ,
 meaning it is always
10 at the end of the list
 and is therefore
11 used as a marker. */
12 } xList;

```

## Lists

- The size of a list at any time is stored in `uxNumberOfItems`, for fast list-size operations. All new lists are initialized to contain a single element: the `xListEnd` element. `xListEnd.xItemValue` is a sentinel value which is equal to the largest value for the `xItemValue` variable: `0xffff` when `portTickType` is a 16-bit value and `0xffffffff` when `portTickType` is a 32-bit value. Other list elements may also have the same value; the insertion algorithm ensures that `xListEnd` is always the last item in the list.
- Since lists are sorted high-to-low, the `xListEnd` element is used as a marker for the start of the list. And since the list is circular, this `xListEnd` element is also a marker for the end of the list.
- Most "traditional" list accesses you've used probably do all of their work within a single `for()` loop or function call like this:

## Lists

```

1 for (listPtr = listStart; listPtr != NULL; listPtr = listPtr->next)
2 {
3 // Do something with listPtr here...
4 }

```

- FreeRTOS frequently needs to access a list across multiple `for()` and `while()` loops as well as function calls, and so it uses list functions that manipulate the `pxIndex` pointer to walk the list. The list function `listGET_OWNER_OF_NEXT_ENTRY()` does `pxIndex = pxIndex->pxNext`; and returns `pxIndex`. (Of course it does the proper end-of-list-wraparound detection too.) This way the list itself is responsible for keeping track of "where you are" while walking it using `pxIndex`, allowing the rest of FreeRTOS to not worry about it.

## Lists

- The `pxReadyTasksLists` list manipulation done in `vTaskSwitchContext()` is a good example of how `pxIndex` is used. Let's assume we have only one priority level, priority 0, and there are three tasks at that priority level. This is similar to the basic ready list picture we looked at earlier, but this time we'll include all of the data structures and fields.
- As you can see in Figure 3.3, `pxCurrentTCB` indicates that we're currently running Task B. The next time `vTaskSwitchContext()` runs, it calls `listGET_OWNER_OF_NEXT_ENTRY()` to get the next task to run. This function uses `pxIndex->pxNext` to figure out the next task is Task C, and now `pxIndex` points to Task C's list element and `pxCurrentTCB` points to Task C's TCB, as shown in Figure 3.4.
- Note that each struct `xListItem` object is actually the `xGenericListItem` object from the associated TCB.

## Queues

- FreeRTOS allows tasks to communicate and synchronize with each other using queues.
- Interrupt service routines (ISRs) also use queues for communication and synchronization.
- The basic queue data structure is:

```

1 typedef struct QueueDefinition
2 {
3 signed char *pcHead; /* Points to the
4 beginning of the queue storage area. */
5 signed char *pcTail; /* Points to the byte
6 at the end of the queue storage area.
7
8 One more byte is allocated than
9 necessary to store the queue items; this is
10 used as a marker. */
11 signed char *pcWriteTo; /* Points to the free
12 next place in the storage area. */
13 signed char *pcReadFrom; /* Points to the last
14 place that a queued item was read from.
15
16 */
17 xList xTasksWaitingToSend; /* List of tasks that
18 are blocked waiting to post onto this
19 queue. Stored in priority order. */

```

```

16 xList xTasksWaitingToReceive; /* List of tasks that
17 are blocked waiting to read from this
18 queue. Stored in priority order. */
19
20 volatile unsigned portBASE_TYPE uxMessagesWaiting; /* The
21 number of items currently in the
22 queue. */ unsigned portBASE_TYPE uxLength; /* The
23 length of the queue defined as
24 the number of items it
25 will hold, not the number of
26 bytes. */ unsigned portBASE_TYPE uxItemSize; /* The size
27 of each items that the queue
28 will hold. */
29 } xQUEUE;

```

## Queues

- This is a fairly standard queue with head and tail pointers, as well as pointers to keep track of where we've just read from and written to.
- When creating a queue, the user specifies the length of the queue and the size of each item to be tracked by the queue. pcHead and pcTail are used to keep track of the queue's internal storage. Adding an item into a queue does a deep copy of the item into the queue's internal storage.
- FreeRTOS makes a deep copy instead of storing a pointer to the item because the lifetime of the item inserted may be much shorter than the lifetime of the queue. For instance, consider a queue of simple integers inserted and removed using local variables across several function calls. If the queue stored pointers to the integers' local variables, the pointers would be invalid as soon as the integers' local variables went out of scope and the local variables' memory was used for some new value.
- The user chooses what to queue. The user can queue copies of items if the items are small, or the user can queue pointers to the items if the items are large. Note that in both cases FreeRTOS does a deep copy: if the user chooses to queue copies of items then the queue stores a deep copy of each item; if the user chooses to queue pointers then the queue stores a deep copy of the pointer. Of course, if the user stores pointers in the queue then the user is responsible for managing the memory associated with the pointers. The queue doesn't care what data you're storing in it, it just needs to know the data's size.

## Queues

- FreeRTOS supports blocking and non-blocking queue insertions and removals. Non-blocking operations return immediately with a "Did the queue insertion work?" or

”Did the queue removal work?” status. Blocking operations are specified with a timeout. A task can block indefinitely or for a limited amount of time.

- A blocked task - call it Task A - will remain blocked as long as its insert/remove operation cannot complete and its timeout (if any) has not expired. If an interrupt or another task modifies the queue so that Task A’s operation could complete, Task A will be unblocked. If Task A’s queue operation is still possible by the time it actually runs then Task A will complete its queue operation and return ”success”. However, by the time Task A actually runs, it is possible that a higher-priority task or interrupt has performed yet another operation on the queue that prevents Task A from performing its operation. In this case Task A will check its timeout and either resume blocking if the timeout hasn’t expired, or return with a queue operation ”failed” status.

## Queues

- It’s important to note that the rest of the system keeps going while a task is blocking on a queue; other tasks and interrupts continue to run. This way the blocked task doesn’t waste CPU cycles that could be used productively by other tasks and interrupts.
- FreeRTOS uses the `xTasksWaitingToSend` list to keep track of tasks that are blocking on inserting into a queue. Each time an element is removed from a queue the `xTasksWaitingToSend` list is checked. If a task is waiting in that list the task is unblocked.
- Similarly, `xTasksWaitingToReceive` keeps track of tasks that are blocking on removing from a queue. Each time a new element is inserted into a queue the `xTasksWaitingToReceive` list is checked. If a task is waiting in that list the task is unblocked.

## Semaphores and Mutexes

- FreeRTOS uses its queues for communication between and within tasks.
- FreeRTOS also uses its queues to implement semaphores and mutexes.
- What’s The Difference?
- Semaphores and mutexes may sound like the same thing, but they’re not. FreeRTOS implements them similarly, but they’re intended to be used in different ways. How should they be used differently?
- The correct use of a semaphore is for signaling from one task to another. A mutex is meant to be taken and released, always in that order, by each task that uses the shared resource it protects. By contrast, tasks that use semaphores either signal [”send” in FreeRTOS term or wait [”receive” in FreeRTOS term - not both.
- A mutex is used to protect a shared resource. A task acquires a mutex, uses the shared resource, then releases the mutex. No task can acquire a mutex while the mutex is being held by another task. This guarantees that only one task is allowed to use a shared resource at a time.
- Semaphores are used by one task to signal another task.
- For example, Task 1 may contain code to post (i.e., signal or increment) a particular semaphore when the ”power” button is pressed and Task 2, which wakes the display, pends on that same semaphore. In this scenario, one task is the producer of the event signal; the other the consumer.

## Semaphores and Mutexes

- FreeRTOS implements an N-element semaphore as a queue that can hold N items. It doesn't store any actual data for the queue items; the semaphore just cares how many queue entries are currently occupied, which is tracked in the queue's `uxMessagesWaiting` field. It's doing "pure synchronization", as the FreeRTOS header file `semphr.h` calls it. Therefore the queue has a item size of zero bytes (`uxItemSize == 0`). Each semaphore access increments or decrements the `uxMessagesWaiting` field; no item or data copying is needed.
- Like a semaphore, a mutex is also implemented as a queue, but several of the `xQUEUE` struct fields are overloaded using `#defines`:

```

1 \ite /* Effectively make a union out of the xQUEUE structure. */
2 \ite #define uxQueueType pcHead
3 \ite #define pxMutexHolder pcTail

```

- Since a mutex doesn't store any data in the queue, it doesn't need any internal storage, and so the `pcHead` and `pcTail` fields aren't needed. FreeRTOS sets the `uxQueueType` field (really the `pcHead` field) to 0 to note that this queue is being used for a mutex. FreeRTOS uses the overloaded `pcTail` fields to implement priority inheritance for mutexes.

## Priority Inheritance

- In case you're not familiar with priority inheritance, I'll quote Michael Barr again to define it, this time from his article, "Introduction to Priority Inversion":
- [Priority inheritance] mandates that a lower-priority task inherit the priority of any higher-priority task pending on a resource they share. This priority change should take place as soon as the high-priority task begins to pend; it should end when the resource is released.
- FreeRTOS implements priority inheritance using the `pxMutexHolder` field (which is really just the overloaded-by-`#define` `pcTail` field). FreeRTOS records the task that holds a mutex in the `pxMutexHolder` field. When a higher-priority task is found to be waiting on a mutex currently taken by a lower-priority task, FreeRTOS "upgrades" the lower-priority task to the priority of the higher-priority task until the mutex is available again.

## Task management

- Introduction and scope
- Main topics to be covered
  - How FreeRTOS allocates processing time to each task within an application
  - How FreeRTOS chooses which task should execute at any given time
  - How the relative priority of each task affects system behavior
  - The states that a task can exist in.

## More specific topics

- How to implement tasks
- How to create one or more instances of a task
- How to use the task parameter
- How to change the priority of a task that has already been created.

- How to delete a task.
- How to implement periodic processing.
- When the idle task will execute and how it can be used.

## Task functions

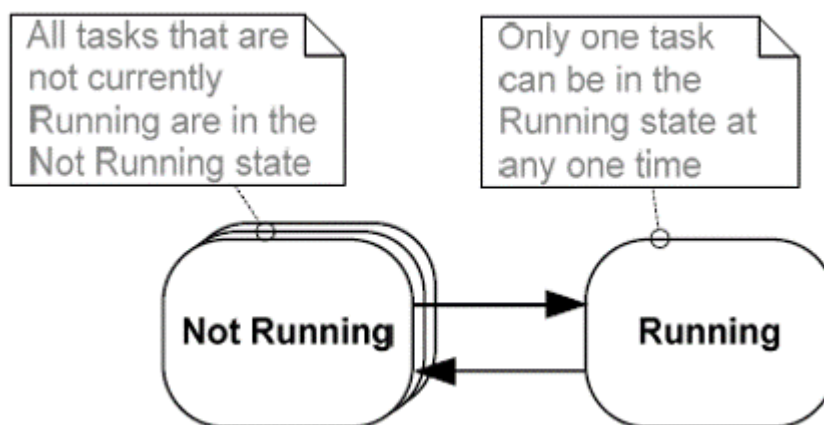
- Tasks are implemented as C functions.
  - Special: Its prototype must return void and take a void pointer parameter as the following
  - `void ATaskFunction (void *pvParameters);`
- Each task is a small program in its own right.
  - Has an entry point
  - Normally runs forever within an infinite loop
  - Does not exit

## Special features of task function

- FreeRTOS task
  - Must not contain a "return" statement
  - Must not be allowed to execute past the end of the function
  - If a task is no longer required it should be explicitly deleted.
  - Be used to create any number of tasks
    - Each created task is a separate execution instance with its own stack and its own copy of any automatic variables defined within the task itself.

## Top level task states

- A task can exist in one of two states: Running and Not Running
- Scheduler is the only entity that can switch a task in and out a running state.



- Running state: the processor is executing its code.
- Not Running state: the task is dormant its status having been saved ready for resuming execution the next time

## Creating Tasks

- `xTaskCreate()` API function

- the most fundamental component in a multitasking system
  - Probably the most complex of all API functions
  -

```

1 portBASE_TYPE xTaskCreate(
2 pdTASK_CODE pvTaskCode
3 const signed char * const pcName
4 unsigned short usStackDepth
5 void *pvParameters
6 unsigned portBASE_TYPE uxPriority
7 xTaskHandle *pxCreatedTask
8);

```

### All parameters

- pvTaskCode
  - a pointer to the function (just the function name) that implements the task.
- pcName
  - A descriptive name for the task. It is not used by FreeRTOS but a debugging aid.
  - configMAX\_TASK\_NAME\_LEN: the application defined constant that defines the maximum length a task name can task including the NULL terminator.

### All parameters

- usStackDepth
  - Each task has its own unique stack that is allocated by the kernel to the task when the task is created.
  - The value specifies the number of words the task stack can hold.
    - E.g. Cortex-M3 stack is 32 bits wide if usStackDepth is passed in as 100 400 bytes of stack space will be allocated (100\*4 bytes)
  - Size of the stack used by the idle task is defined by configMINIMAL\_STACK\_SIZE.
    - Adjustable w.r.t. applications

### All parameters

- pvParameters
  - The value assigned to pvParameters will be the values passed into the task.
- uxPriority
  - defines the priority at which the task will execute.
  - Priorities can be assigned from 0 which is the lowest priority to (configMAX\_PRIORITIES-1) which is the highest priority.
  - Passing a value above (configMAX\_PRIORITIES -1) will result in the priority being capped the maximum legitimate value.

### All parameters

- pxCreatedTask
  - pass out a handle to the created task then be used to refer the created task in API calls.

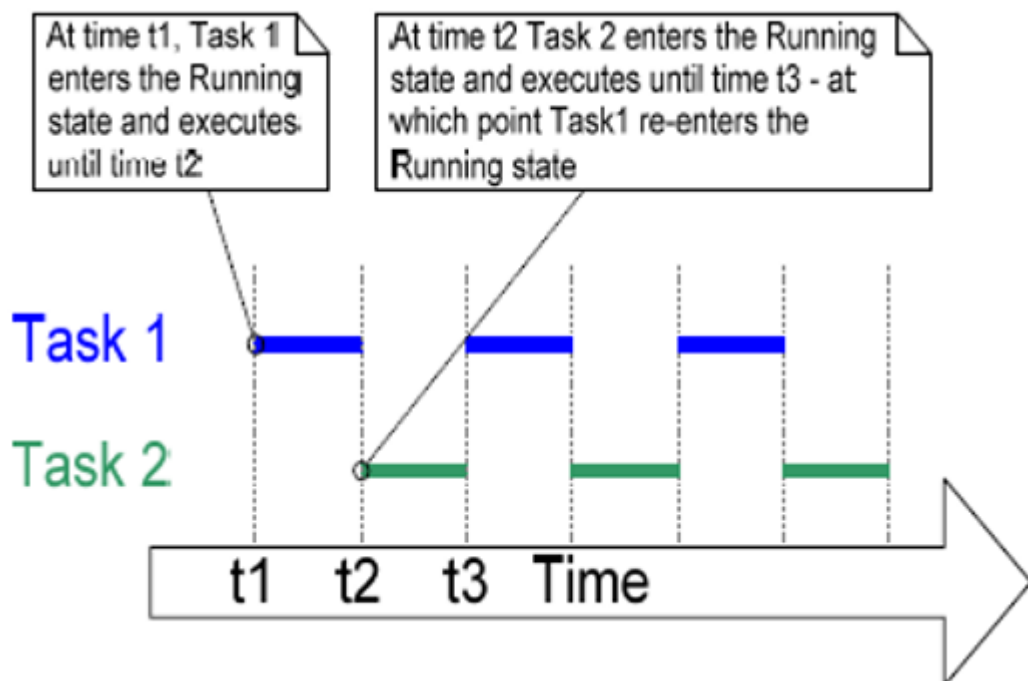
- E.g. change the task priority or delete the task
- Be set to NULL if no use for the task handle
- 
- Two possible return values
  - pdTRUE : task has been created successfully.
  - errCOULD\_NOT\_ALLOCATE\_REQUIRED\_MEMORY
    - Task has not been created as there is insufficient heap memory available for FreeRTOS to allocate enough RAM to hold the task data structures and stack.

### Example: Creating tasks

- To demonstrate the steps of creating two tasks then starting the tasks executing.
- Tasks simply print out a string periodically using a crude null loop to create the periodic delay.
- Both tasks are created at the same priority and are identical except for the string they print out.

### Execution pattern of two Example 1 tasks

- Both tasks are rapidly entering and exiting the Running state.
- Only one task can exist in the Running state at any one time.



### Example 2 Using the task parameter

- The two tasks in Example 1 are almost identical the only difference between them being the text string they print out.
- Remove such duplication by creating two instances of a single task implementation `vTaskFunction`.

- Each created instance will execute independently under the control of the FreeRTOS scheduler.
- The task parameter can then be used to pass into each task the string that it should print out.

### Example 2 Using the task parameter

- First define a char string pcTextForTask1
- static const char \*pcTextForTask1 = Task 1 is running.;
- Then in main(void) function change
- xTaskCreate( vTask1 Task 1 240 NULL 1 NULL );
- to
- xTaskCreate( vTaskFunction Task 1 240 (void\*)pcTextForTask1 1 NULL );
- Create a task function vTaskFunction(void \* pvParameters)
- char \*pcTaskName;
- pcTaskName = (char \*) pvParameters;
- 
- 

### Task priorities

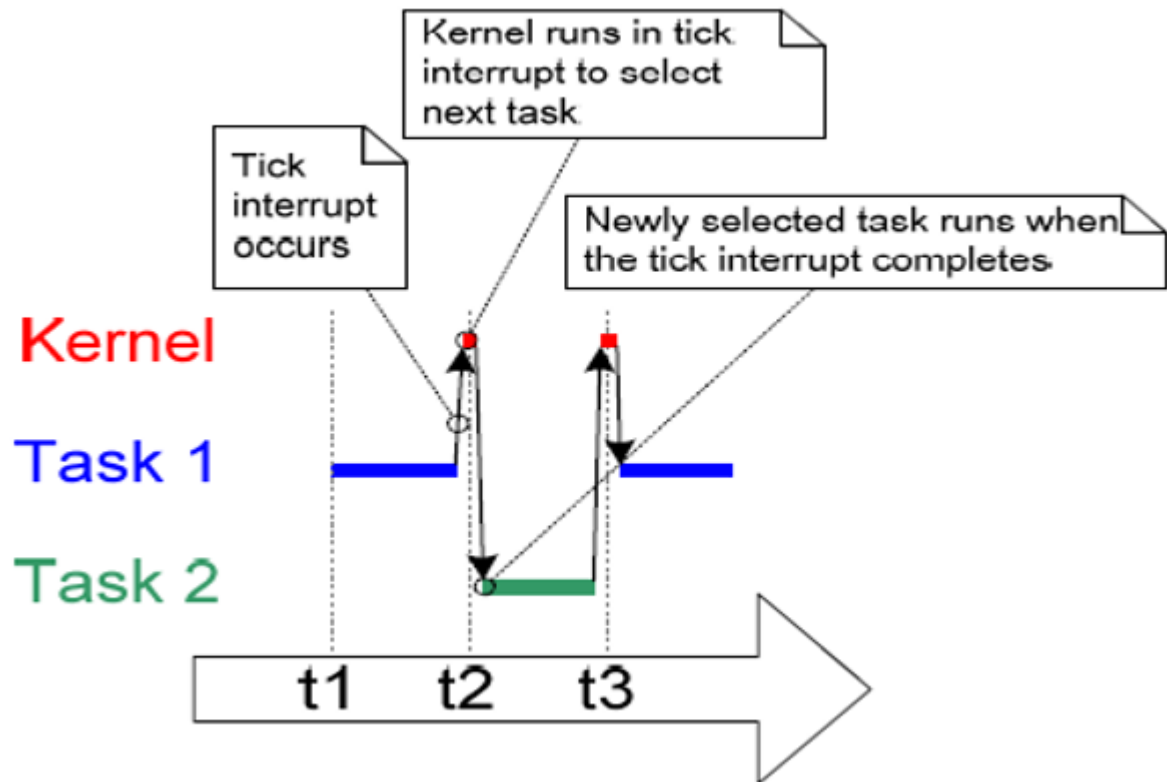
- uxPriority parameter of xTaskCreate() assigns an initial priority to the task being created.
  - It can be changed after the scheduler has been started by using vTaskPrioritySet() API function
  -
- configMax\_PRIORITIES in FreeRTOSConfig.h
  - Maximum number of priorities
  - Higher this value more RAM consumed
  - Range: [0(low) configMAX\_PRIORITIES-1(high)]
  - Any number of tasks can share the same priority

### Task priorities

- To select the next task to run the scheduler itself must execute at the end of each time slice.
  - Use a periodic interrupt called the tick (interrupt).
  - Effectively set the length of time slice by the tick interrupt frequency – configTICK\_RATE\_HZ in FreeRTOSConfig.h
- configTICK\_RATE\_HZ
  - If it is 100(Hz) the time slice will be 10 ms.
  - API always calls specify time in tick interrupts (ticks)
- portTICK\_RATE\_MS
  - Convert time delays from milliseconds into the number of tick interrupts.

### Task priorities

- When kernel itself is running the arrows in the above figure show the sequence of execution from task interrupt then from interrupt back to a next task.



### Example 3. Experimenting with priorities

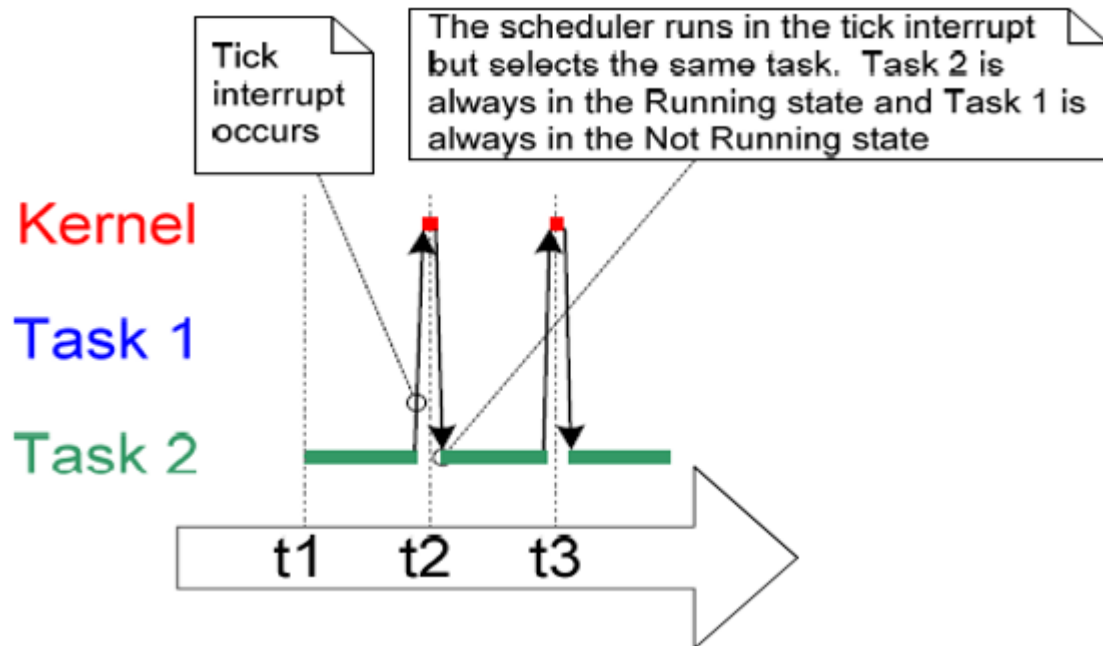
- Scheduler always ensures that the highest priority task is able to run is the task selected to enter the Running state.
- In Example 1 and 2 two tasks have been created at the same priority so both entered and exited the Running state in turn.
- In this example the second task is set at priority 2.

### Example 3. Experimenting with priorities

- In main() function change
  - `xTaskCreate( vTaskFunction Task 2 240 (void*)pcTextForTask2 1 NULL );`
  - `To`
  - `xTaskCreate( vTaskFunction Task 2 240 (void*)pcTextForTask2 2 NULL );`
  -

### Example 3. Experimenting with priorities

- The scheduler always selects the highest priority task that is able to run.
  - Task 2 has a higher priority than Task 1; so Task 2 is the only task to ever enter the Running state.
  - Task 1 is to be "starved" of processing time of Task 2.



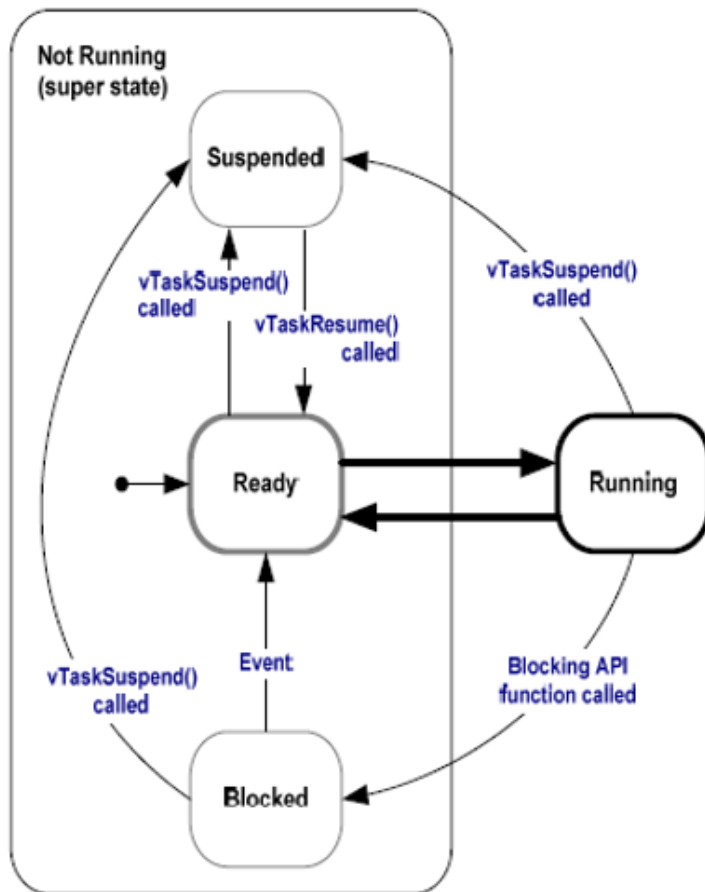
### "Continuous processing" task

- So far the created tasks always have work to perform and have never had to wait for anything
  - Always able to enter the Running state.
- This type of task has limited usefulness as they can only be created at the very lowest priority.
  - If they run at any other priority they will tasks of lower priority ever running at all.
- Solution: Event-driven tasks

### Expanding the "Not Running" state

- An event-driven task
  - has work to perform only after the occurrence of the event that triggers it
  - Is not able enter the Running state before that event has occurred.
- The scheduler selects the highest priority task that is able to run.
  - High priority tasks not being able to run means that the scheduler cannot select them and
  - Must select a lower priority task that is able to run.
- Using event-driven tasks means that
  - tasks can be created at different priorities without the highest priority tasks starving all the lower priority tasks.

### Full task state machine



### Blocked state

- Tasks enter this state to wait for two types of events
  - Temporal (time-related) events: the event being either a delay expiring or an absolute time being reached.
    - A task enter the Blocked state to wait for 10ms to pass
  - Synchronization events: where the events originate from another task or interrupt
    - A task enter the Blocked state to wait for data to arrive on a queue.
- Can block on a synchronization event with a timeout effectively block on both types of event simultaneously.
  - A task waits for a maximum of 10ms for data to arrive on a queue. It leaves the Blocked state if either data arrives within 10ms or 10ms pass with no data arriving.

### Suspended state

- Tasks in this state are not available to the scheduler.
  - The only way into this state is through a call to the `vTaskSuspend()` API function
  - The only way out this state is through a call to the `vTaskResume()` or `vTaskResumeFromISR()` API functions

### Ready state

- Tasks that are in the "Not Running" state but are not Blocked or Suspended are said to be in the Ready state.
  - They are able to run and therefore "ready" to run but are not currently in the Running state.

#### Example 4 Using the Block state to create a delay

- All tasks in the previous examples have been periodic
  - They have delayed for a period and printed out their string before delay once more and so on.
  - Delayed generated using a null loop
    - the task effectively polled an incrementing loop counter until it reached a fixed value.

#### Example 4 Using the Block state to create a delay

- Disadvantages to any form of polling
  - While executing the null loop the task remains in the Ready state "starving" the other task of any processing time.
  - During polling the task does not really have any work to do but it still uses maximum processing time and so wastes processor cycles.
- This example corrects this behavior by
  - replacing the polling null loop with a call to `vTaskDelay()` API function.
  - setting `INCLUDE_vTaskDelay` to 1 in `FreeRTOSConfig.h`

#### `vTaskDelay()` API function

- Place the calling task into the Blocked state for a fixed number of tick interrupts.
  - The Blocked state task does not use any processing time so processing time is consumed only when there is work to be done.
  - `void vTaskDelay(portTickType xTicksToDelay);`
  -
- `xTicksToDelay`: the number of ticks that the calling task should remain in the Blocked state before being transitioned back into the Ready state.
- E.g if a task called `vTaskDelay(100)` while the tick count was 10000 it enters the Blocked state immediately and remains there until the tick count is 10100.

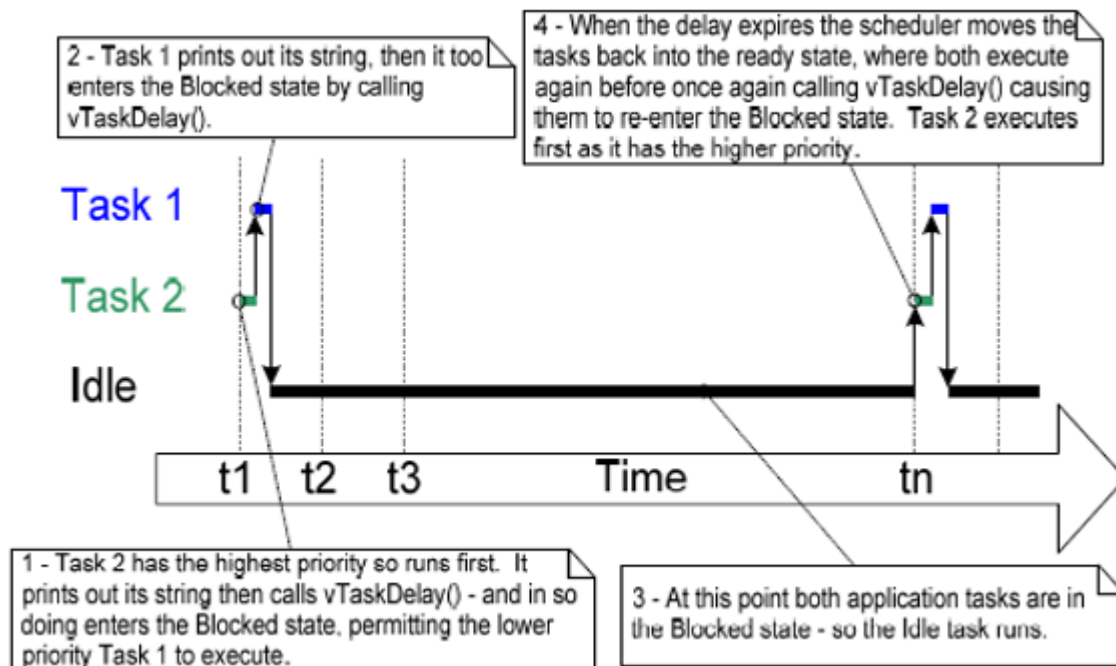
#### `vTaskDelay()` API function

- In void `vTaskFunction(void *pvParameters)`
  - Change a NULL loop
  - `for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ )`
  - To
  - `vTaskDelay(250 / portTICK_RATE_MS);`
  - `// a period of 250ms is being specified.`
  - Although two tasks are being created at different priorities both will now run.

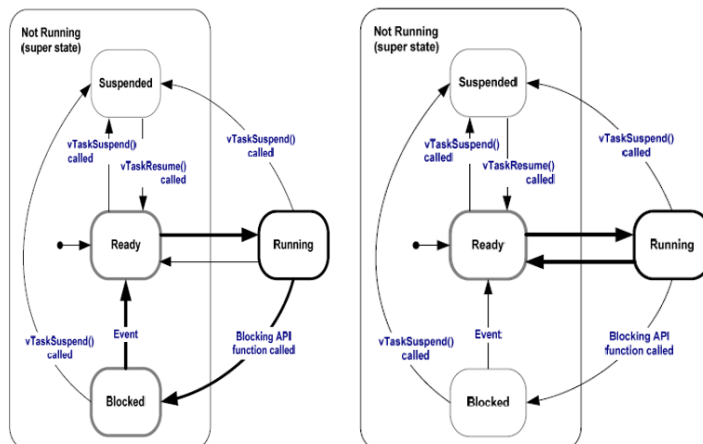
#### `vTaskDelay()` API function

- Each time the tasks leave the Blocked state they execute for a fraction of a tick period before re-entering the Blocked state.

- Most of the time no application tasks are able to run and so no tasks can be selected to enter the Running state.
- The idle task will run to ensure there is always at least one task that is able to run.



**Bold lines indicate the state transitions performed by the tasks in Example 4**



- Tasks 1+2
- Idle Task

### **vTaskDelayUntil() API Function**

- Parameters to vTaskDelayUntil()
  - specify the exact tick count value at which the calling task should be moved from the Blocked state into the Ready state.
  - Be used when a fixed execution period is required.
    - The time at which the calling task is unblocked is absolute rather than relative to when the function was called (as vTaskDelay())
  - void vTaskDelayUntil(

- portTickType \*pxPreviousWakeTime
- portTickType xTimeIncrement);

### **vTaskDelayUntil() prototype**

- pxPreviousWakeTime
  - Assume that vTaskDelayUtil() is being used to implement a task that executes periodically and with a fixed frequency.
  - Holds the time at which the task left the Blocked state.
  - Be used as a reference point to compute the time at which the task next leaves the Blocked state.
  - The variable pointed by pxPreviousWakeTime is updated automatically not be modified by application code other than when the variable is first initialized.

### **vTaskDelayUntil() prototype**

- xTimeIncrement
  - Assume that vTaskDelayUtil() is being used to implement a task that executes periodically and with a fixed frequency - set by xTimeIncrement.
  - Be specified in "ticks" . The constant portTICK\_RATE\_MS can be used to convert ms to ticks.

### **Example 5 Converting the example tasks to use vTaskDelayUntil()**

- Two tasks created in Example 4 are periodic tasks.
- vTaskDelay() does not ensure that the frequency at which they run is fixed
  - as the time at which the tasks leave the Blocked state is relative to when they call vTaskDelay().

### **Example 5 Converting the example tasks to use vTaskDelayUntil()**

- In void vTaskFunction(void \*pvParameters)
  - Change
  - vTaskDelay(250 / portTICK\_RATE\_MS);
  - // a period of 250ms is being specified.
  - To
  - vTaskDelayUntil( &xLastWakeTime (vTaskDelay(250 / portTICK\_RATE\_MS)));
  - /\*xLastWakeTime is initialized with the current tick count before entering the infinite loop. This is the only time it is written to explicitly. \*/
  - xLastWakeTime = xTaskGetTickCount();
  - /\*It is then updated within vTaskDelayUntil(); automatically \*/

### **Example 6 Combining blocking and non-blocking tasks**

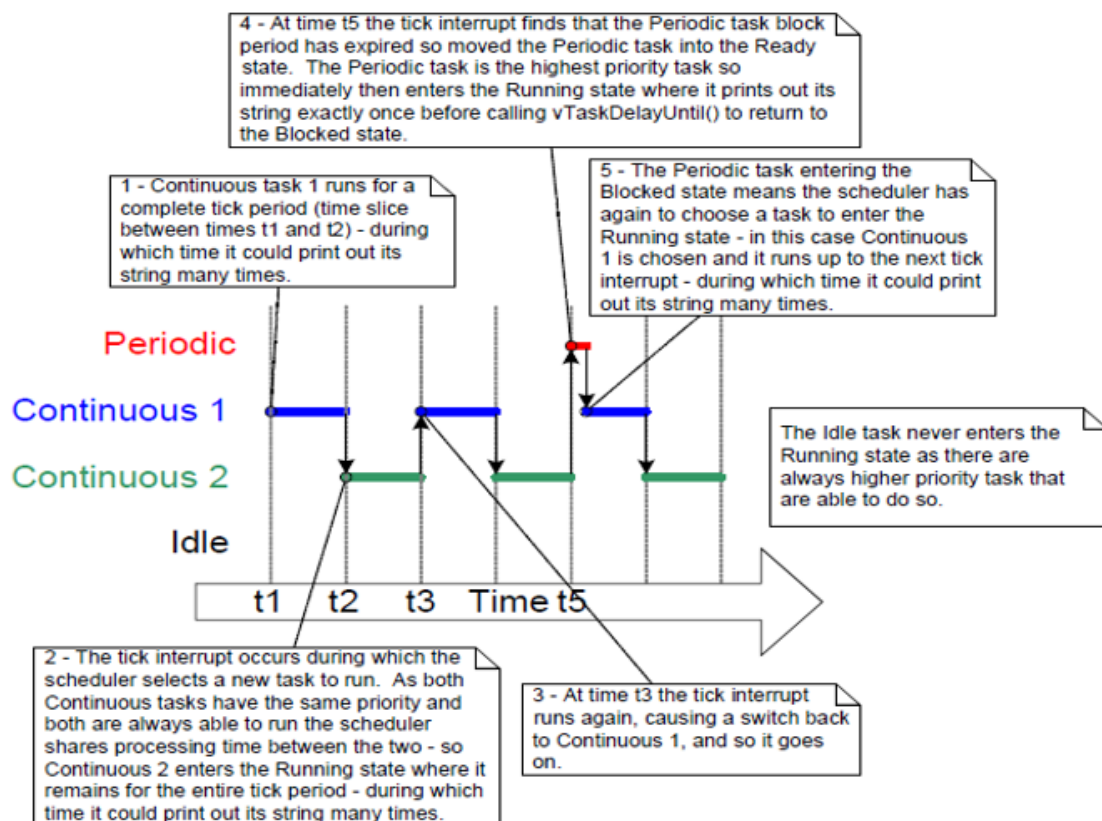
- Two tasks are created at priority 1.
  - Always be either the Ready or the Running state as never making any API function calls.
  - Tasks of this nature are called continuous processing tasks they always have work to do.
- A Third task is created at priority 2.

- Periodically prints out a string by using `vTaskDelayUntil()` to place itself into the Blocked state (blocking time 5) between each print iteration.
- 

## Combining blocking and non-blocking tasks

- `void vContinuousProcessingTask(void * pvParameters)`
- `char *pcTaskName;`
- `pcTaskName = (char *) pvParameters;`
- `for (;;) vPrintString(pcTaskName);`
- `for( ul = 0; ul < 0xffff; ul++ )`
- `void vPeriodicTask(void * pvParameters)`
- `portTickType xLastWakeTime;`
- `xLastWakeTime = xTaskGetTickCount();`
- `for (;;) vPrintString(Periodic task is running);`
- `vTaskDelayUntil(&xLastWakeTime`
- `(10/portTICK_RATE_MS));`

## Scheduling operations

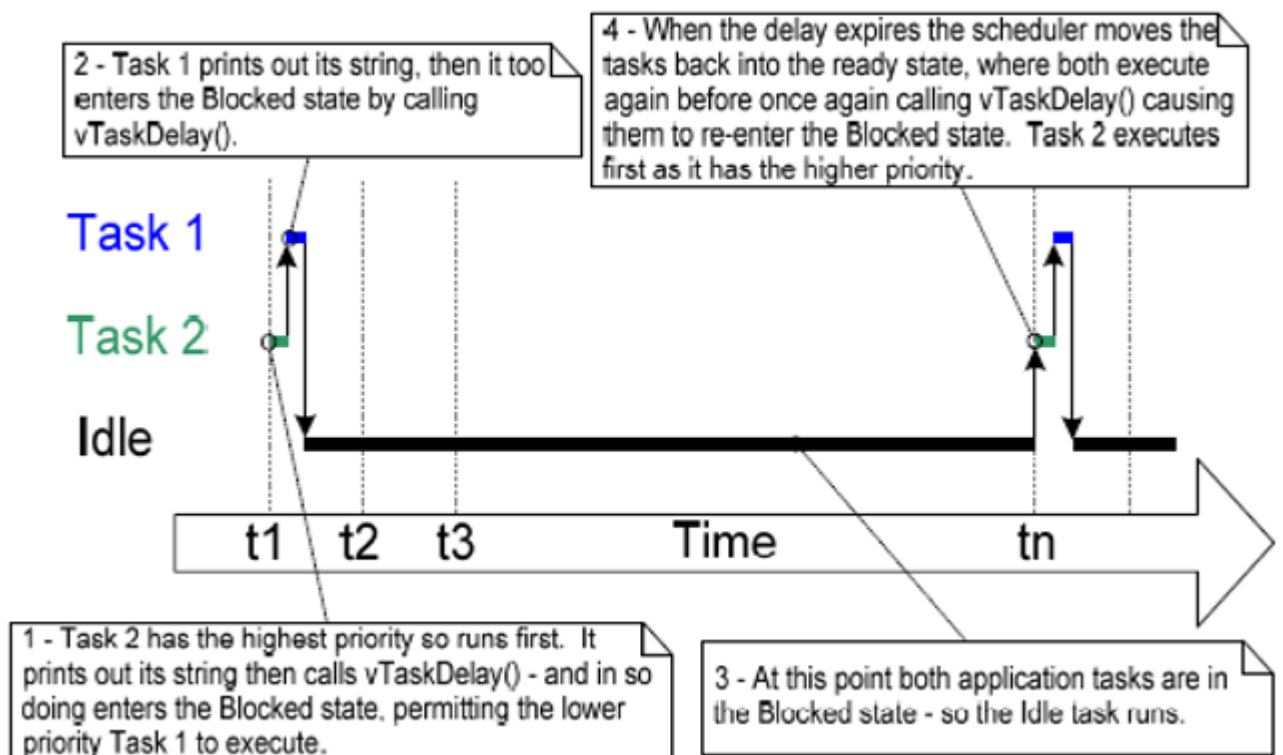


## Idle Task and the Idle task hook

- An idle task is automatically created by the scheduler when `vTaskStartScheduler()` is called.
  - Does very little more than sit in a loop
  - Has the lowest possible priority (zero) to ensure it never prevents a higher priority application task from entering the Running state
  - Be transitioned out of the Running state as soon as a higher priority task enters the Ready state

## Idle Task

- The Idle task is immediately swapped out to allow Task 2 to execute at the instant Task 2 leaves the Blocked state.
  - Task 2 pre-empts the idle task automatically.



## Idle Task Hook Functions

- Add application specific functionality directly into the idle task by the use of an idle hook
  - A function called automatically by the idle task once per iteration of the idle task loop
- Common uses for the Idle task hook
  - Executing low priority background or continuous processing
  - Measuring the amount of spare processing capacity
  - Placing the processor into a low power mode providing an automatic method of saving power whenever no application processing to be performed.

## Limitations on the implementation of Idle Task Hook Functions

- Rules idle task hook functions must adhere to
  - Must never attempt to block or suspend.
  - If the application uses `vTaskDelete()` the Idle task hook must always return to its caller within a reasonable time period.
    - Idle task is responsible for cleaning up kernel resources after a task has been deleted.
    - If the idle task remains permanently in the Idle hook function this clean-up cannot occur.
  - Idle task hook functions have the name and prototype as
    - `void vApplicationIdleHook(void);`

### Example 7 Defining an idle task hook function

- Set `configUSE_IDLE_HOOK` to 1
- Add following function
- `unsigned long ulIdleCycleCount = 0UL;`
- `/* must be called this name take no parameters and return void. */`
- `void vApplicationIdleHook (void)`
- `ulIdleCycleCount++;`
- In `vTaskFunction()` change
- `vPrintString(pcTaskName)` To
- `vPrintStringAndNumber(pcTaskName ulIdleCycleCount);`

### Idle task and Idle Task Hook

- Has the lowest priority possible to add functionality into the idle task vi idle hook
  - Execute continuous processing
  - Measuring spare processing capacity
  - Placing the processor into a low power mode
- Two rules
  - Never block or suspend; must return to its caller within a reasonable time period as it needs to clean up kernel resources after a deleted task.

### Change the priority of a task

- `vTaskPrioritySet(xTaskHandle pxTask unsigned portBASE_TYPE uxNewPriority);`
- Be used to change the priority of any task after the scheduler has been started.
- Available if `INCLUDE_vTaskPrioritySet` is set 1.
- Two parameters
  - `pxTask`: Handle of the task whose priority is being modified. A task can change its own priority by passing `NULL` in place of a valid task handle.
  - `uxNewPriority`: the priority to be set.

### Change the priority of a task

- `unsigned portBASE_TYPE uxTaskPriorityGet(xTaskHandle pxTask);`
  - Be used to query the priority of a task
  - Available if `INCLUDE_vTaskPriorityGet` is set 1
  - `pxTask`: Handle of the task whose priority is being modified. A task can query its own priority by passing `NULL` in place of a valid task handle.
  - Returned value: the priority currently assigned to the task being queried

### Example: Changing task priorities

- Demonstrate the scheduler always selects the highest Ready state task to run
  - by using the `vTaskPrioritySet()` API function to change the priority of two tasks relative to each other.
- Two tasks are created at two different priorities.
  - Neither task makes any API function calls that cause it to enter the Blocked state
    - So both are in either Ready or Running state.
    - So the task with highest priority will always be the task selected by the scheduler to be in Running state

### Expected Behavior of Example 8

- Task 1 is created with the highest priority to be guaranteed to run first. Task 1 prints out a couple of strings before raising the priority of Task 2 to above its own priority.
- Task2 starts to run as it has the highest relative priority.
- Task 2 prints out a message before setting its own priority back to below that of Task 1.
- Task 1 is once again the highest priority task so it starts to run and forcing Task 2 back into the Ready state.

### Example 8 Continued

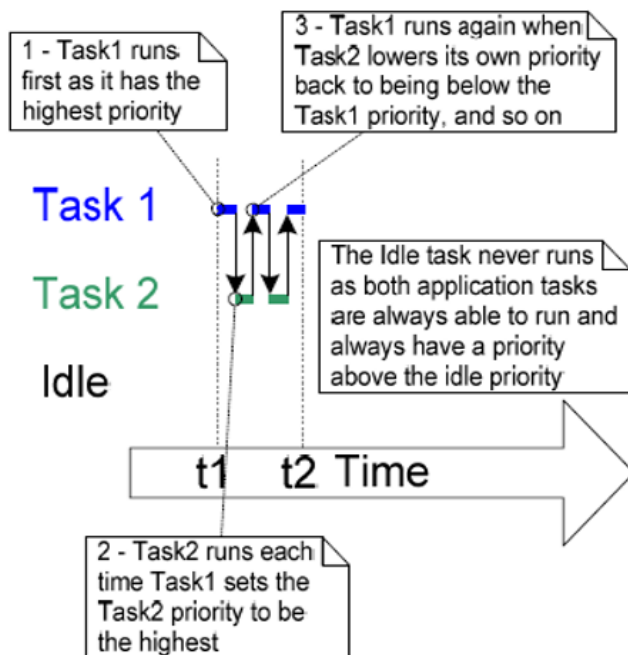
- Declare a global variable to hold the handle of Task 2.
  - `xTaskHandle xTask2Handle;`
- In `main()` function create two tasks
  - `xTaskCreate(vTask1 Task 1 240`
  - `NULL 2 NULL);`
  - `xTaskCreate(vTask2 Task 2 240`
  - `NULL 1 &xTask2Handle);`
- 

### Initialization

- Change `vTask1` by initialization
  - `unsigned portBASE_TYPE uxPriority;`
  - `uxPriority = uxTaskPriorityGet(NULL);`
- And adding to the infinite loop

- `vTaskPrioritySet(xTaskHandle1 (uxPriority+1));`
- Change `vTask2` by initialization
  - `unsigned portBASE_TYPE uxPriority;`
  - `uxPriority = uxTaskPriorityGet(NULL);`
- And adding to the infinite loop
  - `vTaskPrioritySet(NULL (uxPriority-2));`

### Task with changing priority: execution sequence



### Deleting a task

- Deleted tasks no longer exist and cannot enter the Running state again.
- Idle task is responsible to automatically free memory allocated by kernel to tasks that have been deleted.
  - Remember if applications use `vTaskDelete()` do not completely starve the idle task of all processing time.
- Note: any memory or other resource that the application task allocates itself must be freed explicitly by the application code.

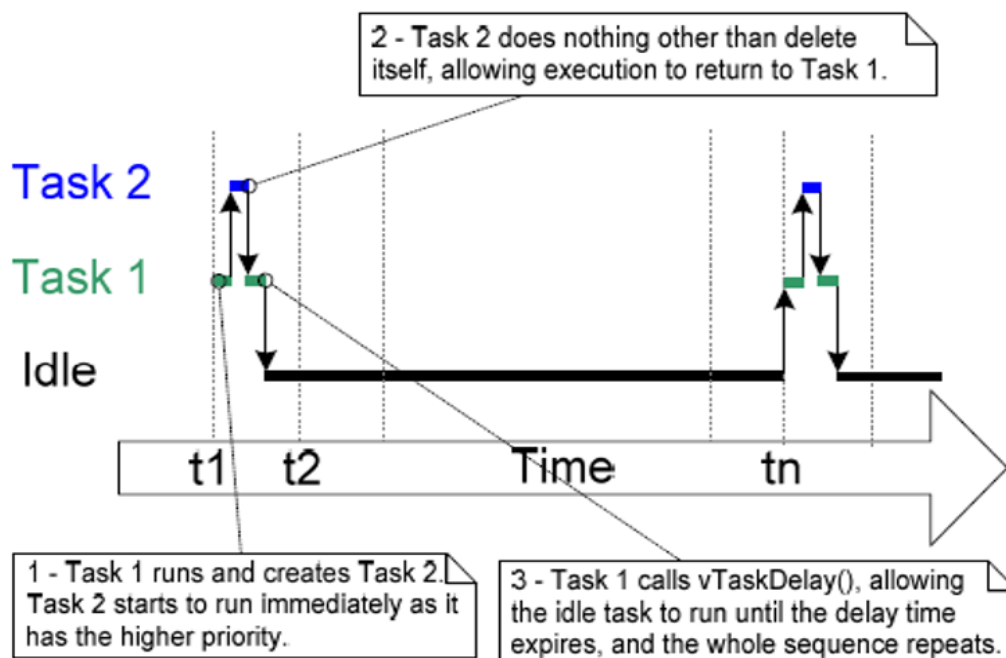
### `vTaskDelete()` API function

- Function prototype
  - `void vTaskDelete(xTaskHandle pxTaskToDelete);`
  - `pxTaskToDelete`: Handle of the task that is to be deleted. A task can delete itself by passing `NULL` in place of valid task handle.
- Available only when `INCLUDE_vTaskDelete` set 1

### Example: Deleting tasks (Behavior)

- Task 1 is created by main() with priority 1. When it runs it creates Task 2 at priority 2. Task 2 as the highest priority task starts to execute immediately.
- Task 2 does nothing but delete itself by passing NULL or its own task handle.
- When Task 2 has been deleted Task 1 is again the highest priority task so continues executing - at which point it calls vTaskDelay() to block for a short period.
- The idle task executes while Task 1 is in the blocked state and frees the memory that was allocated to the now deleted Task 2.
- When Task 1 leaves the blocked state it again becomes the highest priority Ready state task and preempts the Idle task. Then start from Step1 again.

### Execution sequence



### Memory Management

- Does not permit memory to be freed once it has been allocated.
- Subdivide a single array into smaller blocks. Total size of the array (heap) is set by configTOTAL\_HEAP\_SIZE
- xPortGetFreeHeapSize() returns the total amount of heap space that remains unallocated.

### Heap\_2.c

- Allow previously allocated blocks to be freed.
- Does not combine adjacent free blocks into a single large block.

### Summary - Prioritized pre-emptive scheduling

- Examples illustrate how and when FreeRTOS selects which task should be in the Running state.
  - Each task is assigned a priority.
  - Each task can exist in one of several states.
  - Only one task can exist in the Running state at any one time.
  - The scheduler always selects the highest priority Ready state task to enter the Running state.

### Fixed priority Pre-emptive scheduling

- Fixed priority
  - Each task is assigned a priority that is not altered by the kernel itself (only tasks can change priorities)
- Pre-emptive
  - A task entering the Ready state or having its priority altered will always pre-empt the Running state task if the Running state task has a lower priority.

### Tasks in the Blocked state

- Tasks can wait in the Blocked state for an event and are automatically moved back to the Ready state when the event occurs.
- Temporal events
  - Occur at a particular time e.g. a block time expires.
  - Generally be used to implement periodic or timeout behavior.
- Synchronization events
  - Occur when a task or ISR sends info to a queue or to one of the many types of semaphore.
  - Generally be used to signal asynchronous activity such as data arriving at a peripheral.
- 

### Semaphores & Queues

- Introduction to task synchronization
- Semaphores and mutexes of FreeRTOS
- Queues of FreeRTOS

### Why Synchronization?

- Synchronization may be used to solve:
  - Mutual exclusion
  - Control flow
  - Data flow
- Synchronization mechanisms include:
  - Message queues
  - Semaphores
  - Mutexs

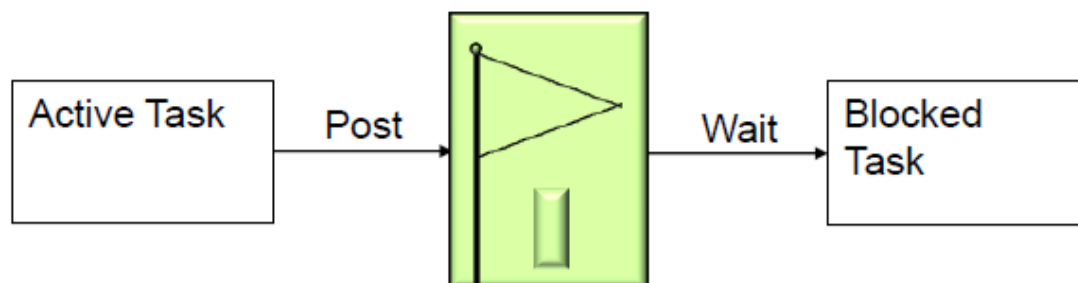
- Events
- Correct synchronization mechanism depends on the synchronization issue being addressed

### Mutual Exclusion

- Problem: multiple tasks may simultaneously need to access the same resource
  - Resource may be code data peripheral etc.
  - Need to allow the shared resource exclusively accessible to only one task at a time
- How to do?
  - Allowing only one task to lock the resource and the rest have to wait for the resource to be unlocked
  - Common mechanisms: lock/unlock mutex semaphore

### Control Flow Synchronization

- Problem: a task or ISR may need to resume the execution of one or more other tasks so that tasks execute in an application-controlled order
  - Mutual exclusion is used to prevent another task from running while control flow is used to allow another task to run often specific tasks
- How to do?
  - Common mechanisms: post/wait signal event



### Data Flow Synchronization

- Problem: a task or ISR may need to pass some data to one or more other specific tasks so that data may be processed in an application-specified order
- How to do?
  - May be accomplished indirectly through control flow synchronization
  - Common mechanisms: queues signal post/wait

### Outline

- Introduction to task synchronization
- Semaphores and mutexes of FreeRTOS
  - Binary semaphores
  - Counting semaphores
  - Mutex

- Queues of FreeRTOS

## Semaphores

- Semaphores are used to:
  - Control access to a shared resource (mutual exclusion)
  - Signal the occurrence of an event
  - Allow two tasks to synchronize their activities
- Basic idea
  - A semaphore contains a number of tokens. The code needs to acquire one in order to continue execution
  - If all the tokens of the semaphore are used the requesting task is suspended until some tokens are released by their current owners



## How Semaphores Work?

- A semaphore has:
  - Counter: maximum number of concurrent accesses
  - Queue: for tasks that wait for access
- If a task requests (waits for) a semaphore
  - if counter  $> 0$  then (1) the counter is decremented by 1 and (2) task gets the semaphore and proceed to do work
  - Else task is blocked and put in the queue
- If a task releases (posts) a semaphore
  - if there are tasks in the semaphore queue then appropriate task is readied according to queuing policy
  - Else counter is incremented by 1

## Binary Semaphores

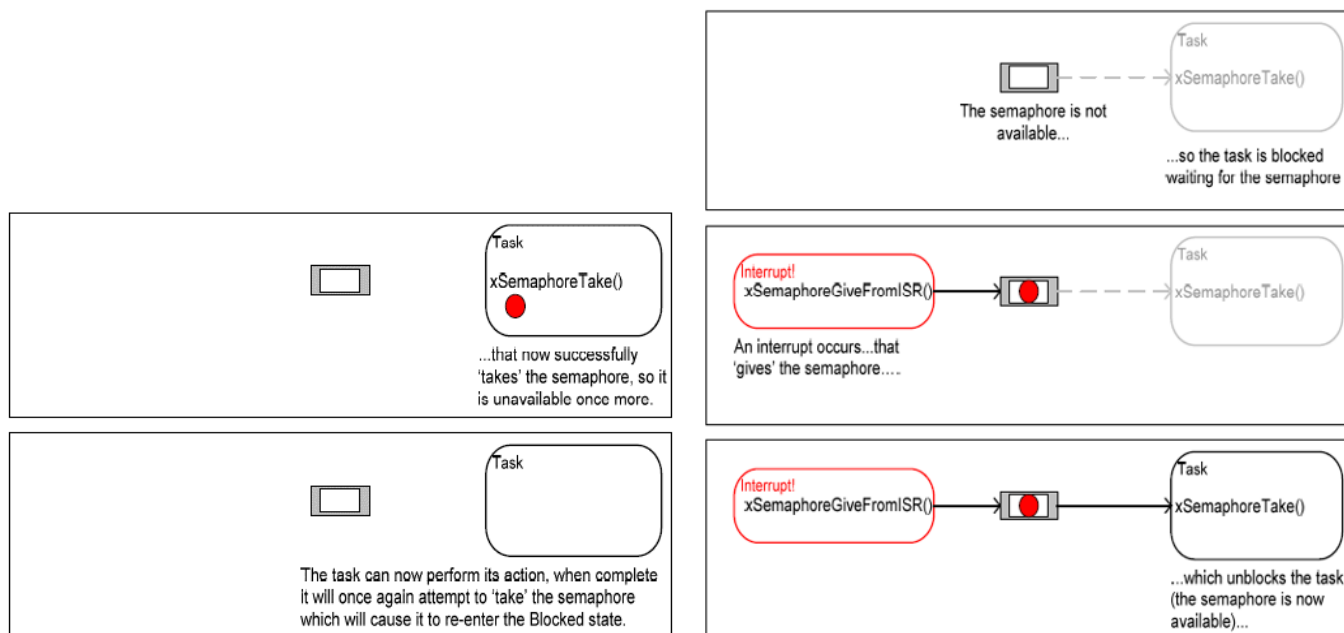
- Semaphores with counter = 1 used for mutual exclusion and synchronization
  - For synchronization purpose a binary semaphore can be think of as a queue that can only hold one item
    - The queue can only be empty or full (hence binary)
    - Tasks using the queue don't care what the queue holds only want to know if the queue is empty or full
    - If more than one task blocks on the same semaphore then the task with the highest priority will be the task that is unblocked the next time the semaphore becomes available

## Binary Semaphores and Interrupts

- The best way to handle complex events triggered by interrupts is to not do the code in the ISR
  - Create a task that is blocking on a binary semaphore
  - When the interrupt happens the ISR just sets (gives) the semaphore and exits
  - Task can now be scheduled like any other
    - No need to worry about nesting interrupts and interrupt priority
  - This is called Deferred Interrupt Processing

## Binary Semaphores and Interrupts

- Figure from Using the FreeRTOS Real Time Kernel (a pdf book) fair use claimed.



## Create a Binary Semaphore

- `SemaphoreHandle_t xSemaphoreCreateBinary(void);`
- Function to create a binary semaphore
  - The semaphore is created in the 'empty' state
  - A binary semaphore need not be given back once obtained so task synchronization can be implemented by one task/interrupt continuously 'giving' the semaphore while another continuously 'takes' the semaphore
  - Binary semaphores are assigned to variables of type `SemaphoreHandle_t` and can be used in any API function that takes a parameter of this type

## Set a Binary Semaphore

- `xSemaphoreGiveFromISR(`
- `SemaphoreHandle_t xSemaphore`
- `signed BaseType_t *pxHigherPriorityTaskWoken)`
- Set (give) a semaphore

- Can be used from an ISR
- xSemaphore: handle to the semaphore being released
- pxHigherPriorityTaskWoken: set to pdTRUE if giving the semaphore caused a task of a higher priority to unblock causing a context switch

## Reset a Binary Semaphore

- xSemaphoreTake( xSemaphoreHandle xSemaphore
- portTickType xBlockTime)
- Reset (take) a semaphore
  - xSemaphore: handle to the semaphore being taken
  - xBlockTime: time in ticks to wait for the semaphore to become available
    - A block time of zero can be used to poll the semaphore

## Example of Binary Semaphores

```
vSemaphoreHandle binary_sem; //Global handler void one_sec_isr(void) // an ISR xSemaphoreGive
```

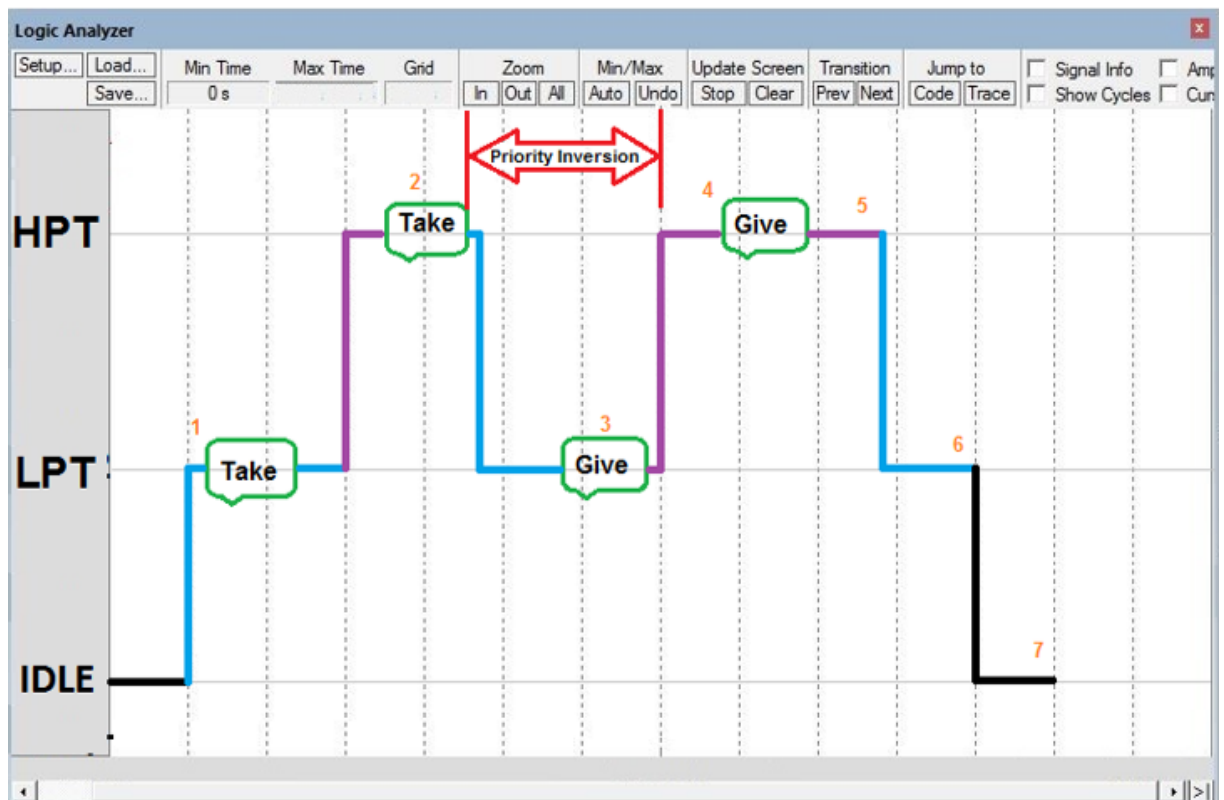
## Counting Semaphores

- Typically used for two things:
- Counting events:
  - An event handler will "give" a semaphore each time an event occurs and a handler task will 'take' a semaphore each time it processes an event
- Resource management:
  - The count value indicates number of available resources
  - To get a resource a task must obtain (take) a semaphore
  - When a task finishes with the resource it 'gives' the semaphore back
- SemaphoreHandle\_t xSemaphoreCreateCounting( UBaseType\_t uxMaxCount UBaseType\_t uxInitialCount)

## Priority Inversion

- LPT starts running and acquires the semaphore.
- Now HPT is created and it preempts LPT and starts running. It makes the request to acquire the semaphore. Since the semaphore is already with LPT HPT goes to blocked state.
- LPT starts executing again and releases the semaphore.
- Immediately the HPT comes out of the blocked state and starts executing.
- It releases the semaphore and runs for some time and deletes itself.
- Now control goes back to LPT which completes its job and deletes itself.
- Finally the scheduler is left out with the idle task and it keeps running.

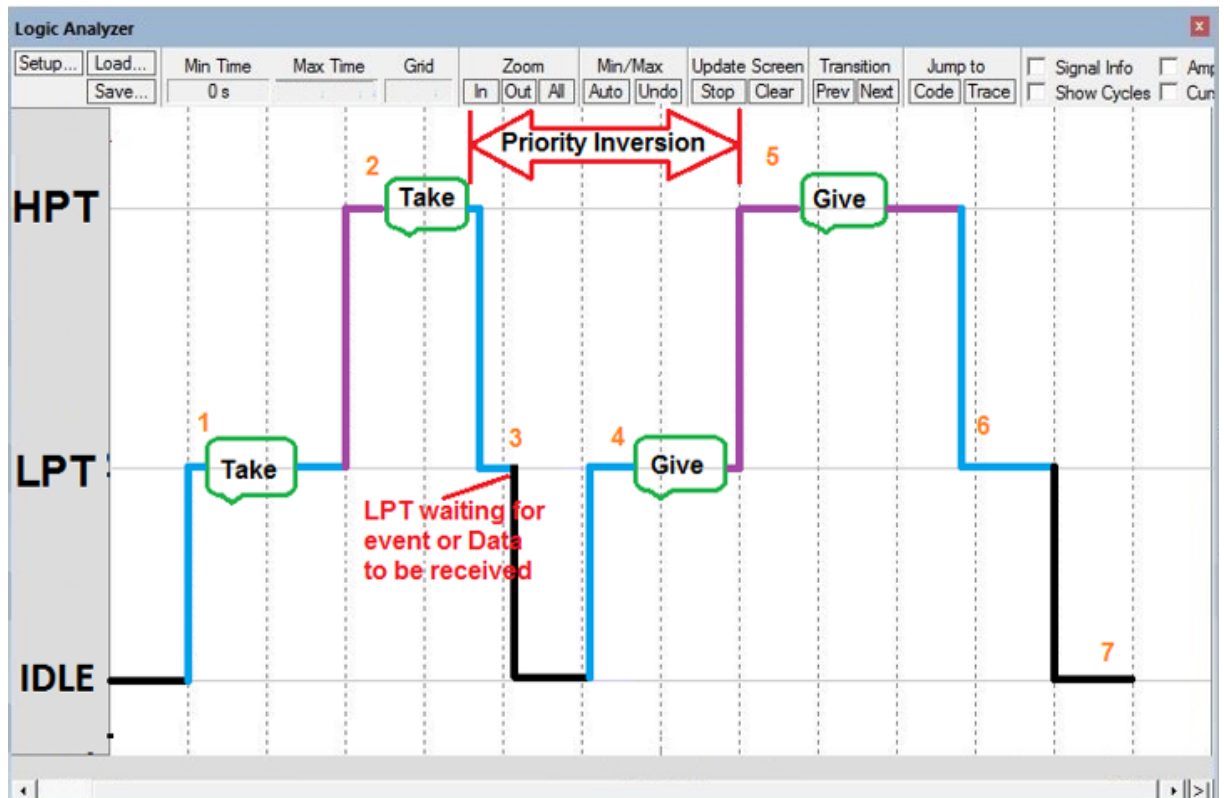
## Priority Inversion



### Extended Priority Inversion

- LPT starts running and acquires the semaphore.
- Now HPT is created and it preempts LPT and starts running. It makes the request to acquire the semaphore. Since the semaphore is already with LPT HPT goes to blocked state.
- LPT starts executing again and waits for an event(packet to be received).
- Now the Idle task starts running. Now the HPT is starved because of LPT and Idle task.
- LPT comes out of blocked state as the event(wait time/package is received) has occurred. Now it releases the semaphore.
- Immediately the HPT comes out of the blocked state and starts executing. It releases the semaphore and runs for some time and deletes itself.
- Now control goes back to LPT which completes its job and deletes itself.
- Finally the scheduler is left out with the idle task and it keeps running.

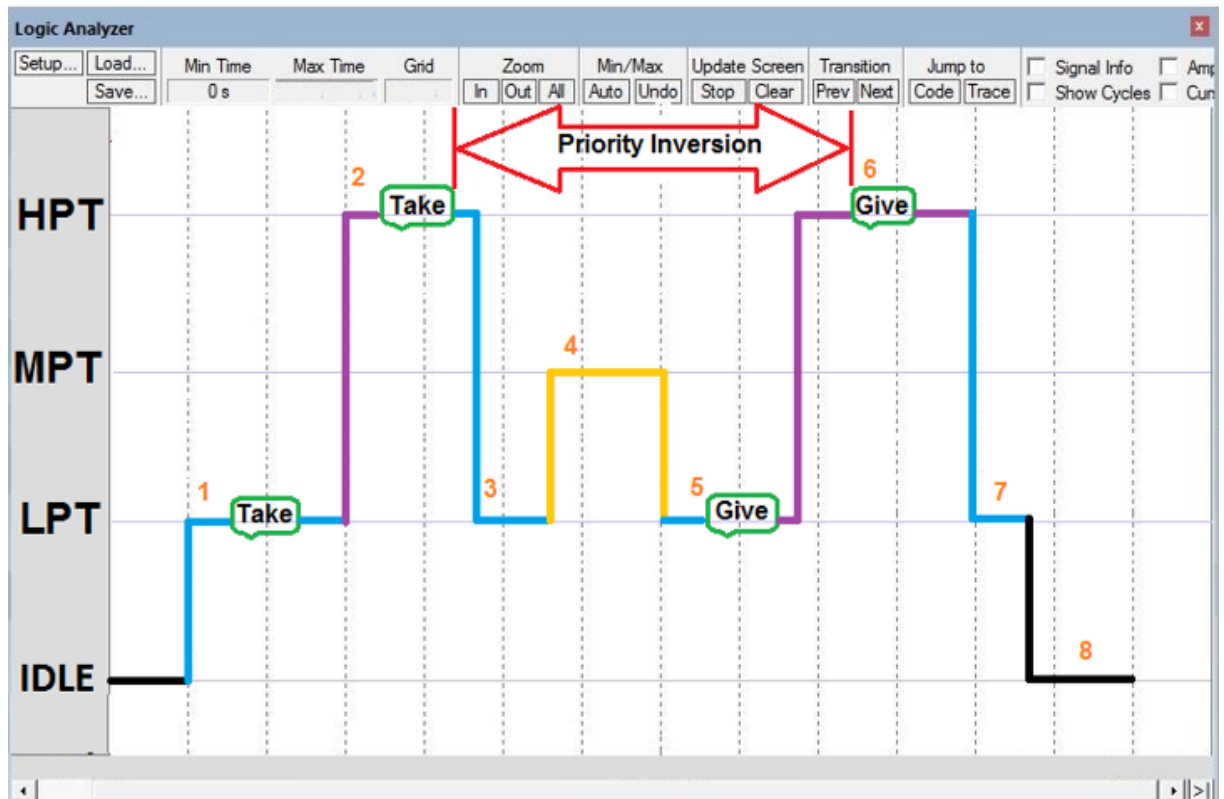
### Extended Priority Inversion



### Worst Case Priority Inversion

- LPT starts running and acquires the semaphore.
- Now HPT is created and it preempts LPT and starts running. It makes the request to acquire the semaphore. Since the semaphore is already with LPT HPT goes to blocked state.
- LPT starts executing again and creates an MPT tasks.
- MPT preempts the LPT and starts running. At this point the HPT is starved because of LPT and MPT. MPT completes its job and deletes itself.
- LPT comes out of blocked state and releases the semaphore.
- Immediately the HPT comes out of the blocked state and starts executing. It releases the semaphore and runs for some time and deletes itself.
- Now control goes back to LPT which completes its job and deletes itself.
- Finally the scheduler is left out with the idle task and it keeps running.

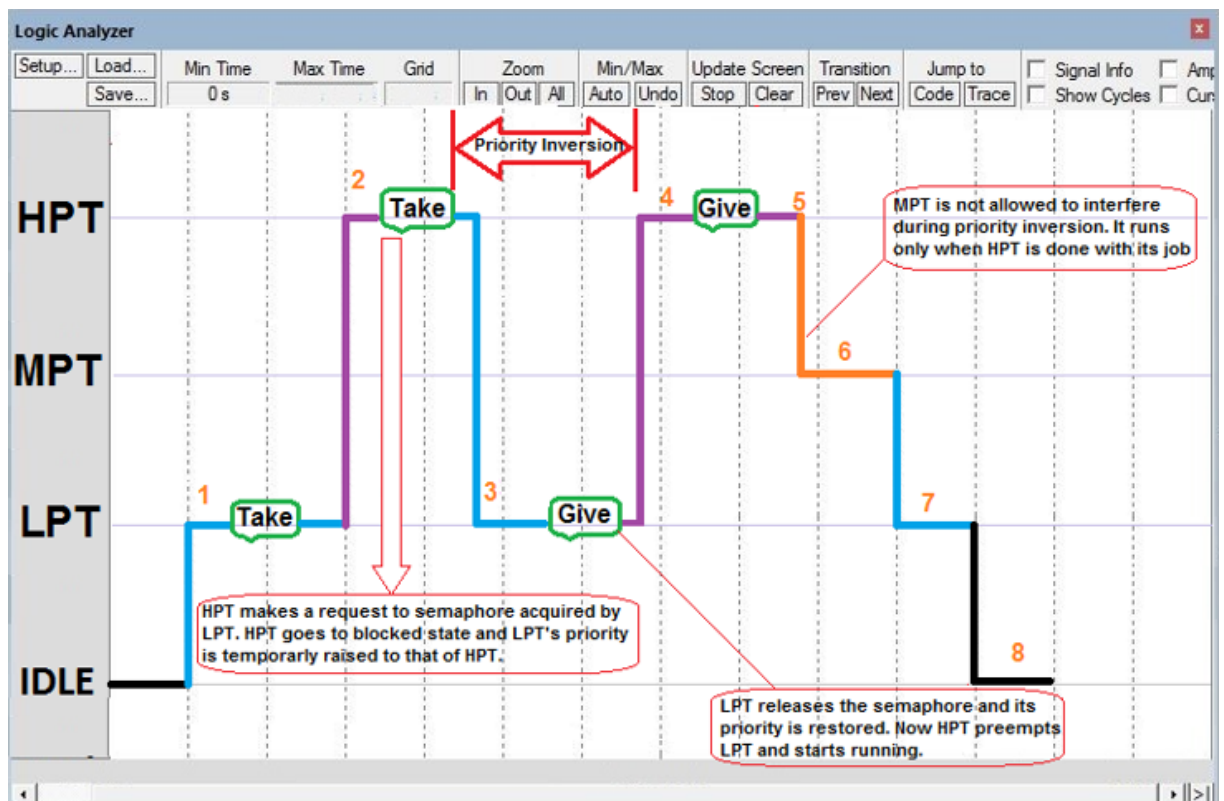
### Worst Case Priority Inversion



## Priority Inheritance

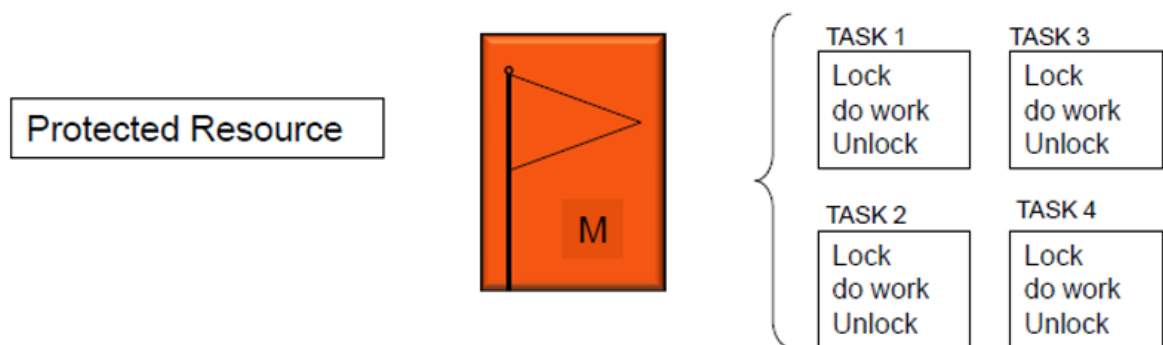
- As soon as the HPT makes a request to semaphore which is allocated to an LPT then the priority of LPT is temporarily increased to that of HPT. Once the LPT releases the semaphore its priority will be restored. Because of this the MPT is never allowed to interfere during the priority inversion(between LPT&HPT). -j, Mutex!
- LPT starts running and acquires the semaphore.
- Now HPT is created and it preempts LPT and starts running. It makes the request to acquire the semaphore. Since the semaphore is already with LPT HPT goes to blocked state and LPT's priority is temporarily raised to that of HPT.
- LPT starts executing again and creates an MPT tasks. It keeps running as its priority is currently higher than MPT. It releases the semaphore and its priority is restored back to lower value.
- Immediately the HPT comes out of the blocked state and starts executing.
- HPT releases the semaphore and runs for some time and deletes itself.
- Now scheduler is left with LPTMPT and IDLE task. MPT starts running and deletes itself after completing its job.
- Now control goes back to LPT which completes its job and deletes itself.
- Finally the scheduler is left out with the idle task and it keeps running.

## Priority Inheritance



## Mutex

- Mutexes are used for mutual exclusion so that only one task at a time uses a shared resource e.g. file data device ...
  - To access the shared resource a task locks the mutex associated with the resource
  - The task owns the mutex until it unlocks the mutex



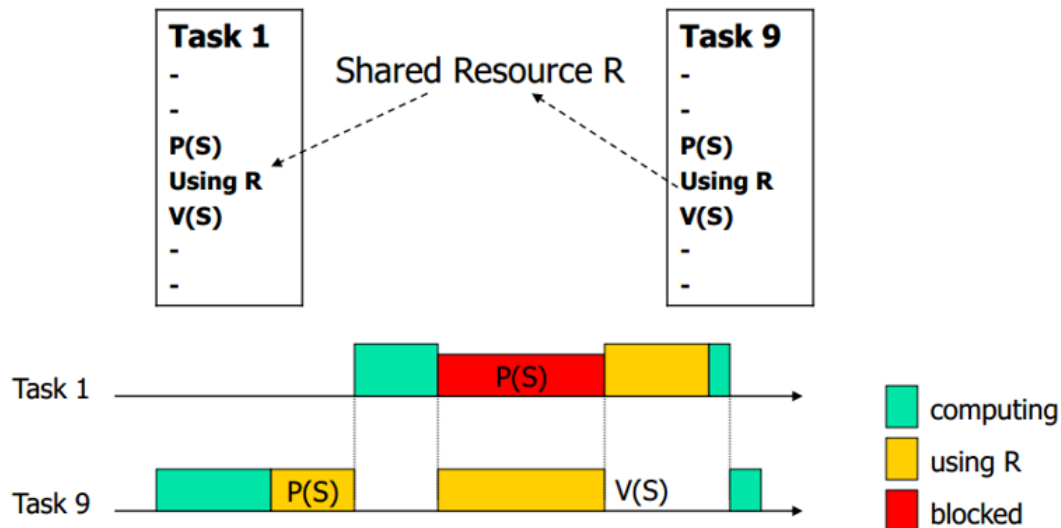
## Mutex

- Mutex acts like a token used to guard a resource
  - When a task wishes to access the resource it must first obtain ('take') the token
  - When the task has finished with the resource it must 'give' the token back - allowing other tasks the opportunity to access the same resource
- Mutexes are binary semaphores that include a priority inheritance mechanism

- If a high priority task blocks while attempting to obtain a mutex (token) that is currently held by a lower priority task then the priority of the task holding the token is temporarily raised to that of the blocking task

## Priority Inversion

- Assume priority of T1 > priority of T9
- If T9 has exclusive access T1 has to wait until T9 releases resource inverting priority can raise priority of T9



- T1 has higher priority and preempts T9
- Critical section
- (critical section)

## Example of Mutex (1/3)

```
xSemaphoreHandle gatekeeper = 0; //Global handler
void task_1(void *p){
while(1){
if(xSemaphoreTake(gatekeeper 1000)){
puts(Task 1 got access);
access_precious_resource(); // critical section
xSemaphoreGive(gatekeeper); }
else{
puts(Task 1 failed to get access within 1000ms);
}
vTaskDelay(1000);
/*Without delay Task 1 will get key immediately
after releasing the key */
}
}
```

## Example of Mutex (2/3)

```

void task_2(void *p){
while(1){
if(xSemaphoreTake(gatekeeper 1000)){
puts(Task 2 got access);
access_precious_resource(); // critical section
xSemaphoreGive(gatekeeper); }
else{
puts(Task 2 failed to get access within 1000ms);
}
vTaskDelay(1000);
/*Without delay task 2 will get key immediately
after releasing the key */
}
}
}

```

### Example of Mutex (3/3)

```

int main(void){
gatekeeper = xSemaphoreCreateMutex();

/*Create tasks with priority 1 for both users*/
xTaskCreate(task_1 (signed char*)) t1 1024
NULL 1 NULL);
xTaskCreate(task_2 (signed char*)) t2 1024
NULL 1 NULL);

vTaskStartScheduler();
return 0;
}

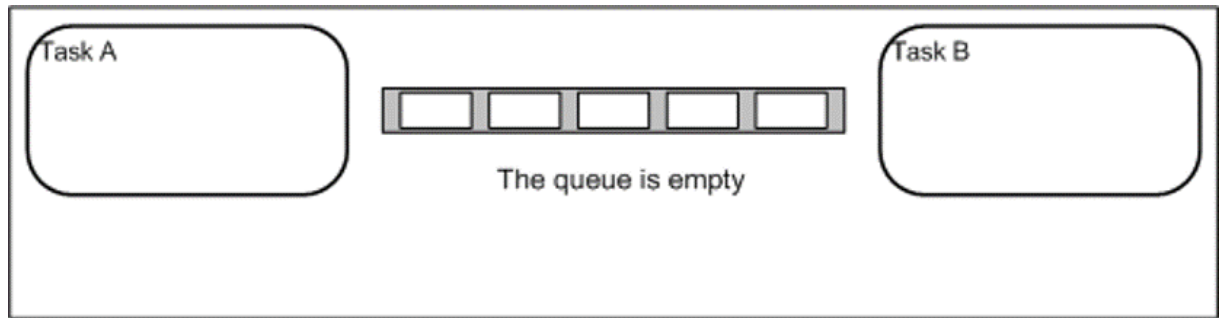
```

### Outline

- Introduction to task synchronization
- Semaphores and mutexs of FreeRTOS
- Queues of FreeRTOS

### FreeRTOS Queues

- Queues are the primary form of inter-task communications in FreeRTOS
  - Can be used to send messages between tasks and between interrupts and tasks
  - In most cases used as thread safe FIFO (First In First Out) buffers with new data being sent to the back of the queue although data can also be sent to the front.



## FreeRTOS Queues

- Queues store a finite number of fixed-size data
  - May be read and written by different tasks but do not belong to any task
  - # of items and item size determined at queue create time
  - Sending/receiving items are by copy not reference
- Queue functions
  - Create queues: `xQueueCreate()` `vQueueDelete()`
  - Send/receive data to/from queues: `xQueueSend()` `xQueueSendToBack()` `xQueueReceive()` `xQueueReceiveFromISR()`
  - Queue management/number of items in a queue
  - Blocking on a queue/effect of priority

## Blocking on Queues

- Queue APIs permit a block time to be specified
- When read from an empty queue
  - Task will be placed into the Blocked state
  - Until data is available on the queue or block time expires
- When write to a full queue
  - Task will be placed into the Blocked state
  - Until space is available in the queue or block time expires
- If more than one task block on the same queue then the task with the highest priority will be the task that is unblocked first

## Queue Creation

- `QueueHandle_t xQueueCreate(`
- `UBaseType_t uxQueueLength`
- `UBaseType_t uxItemSize);`
- Creates a new queue instance
  - Allocates queue storage and returns a handle
  - `uxQueueLength`: maximum number of items that the queue can contain
  - `uxItemSize`: number of bytes that each item in the queue will require
    - Items are queued by copy not by reference so this is the number of bytes that will be copied for each posted item
  - Each item on the queue must be the same size

## Send Data through Queues

- BaseType\_t xQueueSend(QueueHandle\_t xQueue
- const void \* pvItemToQueue
- TickType\_t xTicksToWait);
- Post an item on a queue
  - The item is queued by copy not by reference
  - Must not be called from an interrupt service routine
  - xQueue: queue handle to which the item is to be posted
  - pvItemToQueue: pointer to item to be placed on queue
  - xTicksToWait: max. time (in ticks) that task should block waiting for space to become available should it is full
  - If INCLUDE\_vTaskSuspend is set to "1" then specifying the block time as port-MAX\_DELAY will block task indefinitely

## Receive Data through Queues

- BaseType\_t xQueueReceive(
- QueueHandle\_t xQueue void \*pvBuffer
- TickType\_t xTicksToWait);
- Receive an item from a queue
  - The item is received by copy so a buffer of adequate size must be provided
  - Must not be used in an interrupt service routine
  - xQueue: queue handle from which to receive item
  - pvBuffer: pointer to the buffer into which the received item will be copied.
  - xTicksToWait: max. amount of time the task should block waiting for an item should the queue be empty

## Example of Queues (1/2)

```
xQueueHandle Global_Queue_Handle = 0; //Global Handler
void sender_task(void *p){ int i=0;
while(1){
printf(Send if(!xQueueSend(Global_Queue_Handle &i 1000))
printf(Failed to send to queue);
i++;
vTaskDelay(3000); }
}
void receiver_task(void *p){
int rx_int = 0;
while(1){
if(xQueueReceive(Global_Queue_Handle &rx_int1000))
printf(Received else puts(Failed to receive data from queue); }
}
```

## Example of Queues (2/2)

```

int main(void){

Global_Queue_Handle = xQueueCreate(3 sizeof(int));

/*Create tasks with priority 1 for both users*/
xTaskCreate(sender_task (signed char*)) sx
1024 NULL 1 NULL);
xTaskCreate(receiver_task (signed char*)) rx
1024 NULL 1 NULL);

vTaskStartScheduler();
return 0;
}

```

## 7.3 Functions of Embedded Linux

- the general idea of an “embedded Linux” is to store an image (Kernel plus necessary modules) in a non-volatile memory (Flash-EEPROM, SD-Card).
- after a reset (or power on reset) the image is copied in the RAM and executed.
- the size of the copied image is also known as a “footprint”.
- the additional feature of embedded CLinux (Linux for microcontrollers) is has the opportunity to work without a virtual memory system (VM).
  - suitable for processors without MMU.
  - low memory footprint.
  - according to the processors low power consumption applications are possible.
- with the ARM-Processors (MMU is available) embedded Linux could run with a virtual memory system or with a non virtual memory system.

### Boot sequence

- the boot procedure of embedded Linux differs from a standard boot procedure.
- this procedure offers two advantages:
  - the development process of building a Linux system is nearly the same as in a standard Linux (the same tools are available).
  - the execution time of programs which runs in a RAM is much more faster than in a Flash-EPROM).
- boot procedure starts immediately after the reset (an interaction with the user isn’t necessary).

### Scalable behaviour of embedded Linux

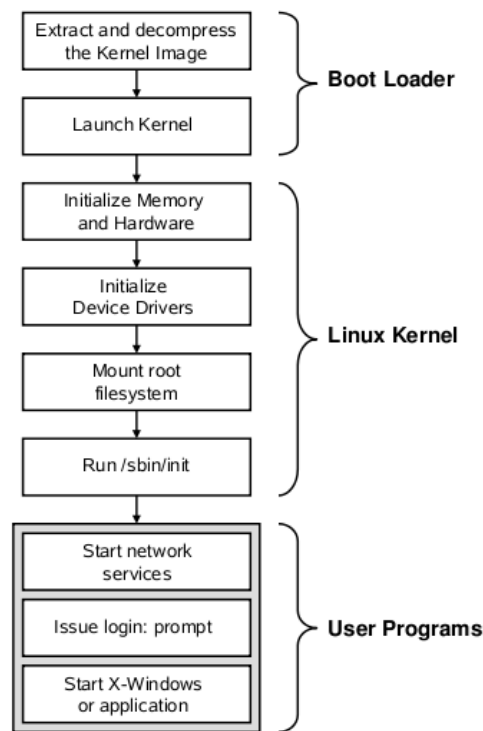


Figure 7.3.1: Boot Procedure

- building of a new image (binaries of embedded Linux systems) starts with a configuration part.
- first part is the selection of the “vendor”.
- specification of hardware details of the development kit (RAM addresses, ROM addresses).
- the next step is the configuration of necessary modules and drivers.

### Build procedure

- after the configuration (make menuconfig)
  - start the build process with “make dep”.
  - and “make”.
- the Kernel-compilation need a special compiler (gcc-version for the ARM-Processor) und a special binutils (as, ld objtools for the ARM-Processor).
- the result of the Kernel-compilation is a image which is available in two versions.
  - file “linux.bin or Kernel.img” .
    - a loadable image.
    - could be downloaded (serial or usb-interface, needs a monitor program on the target side ).
    - execute after a poweron reset or with a jump on a specific address (monitor function).
    - useful for test of a configuration.
- file “romfs”

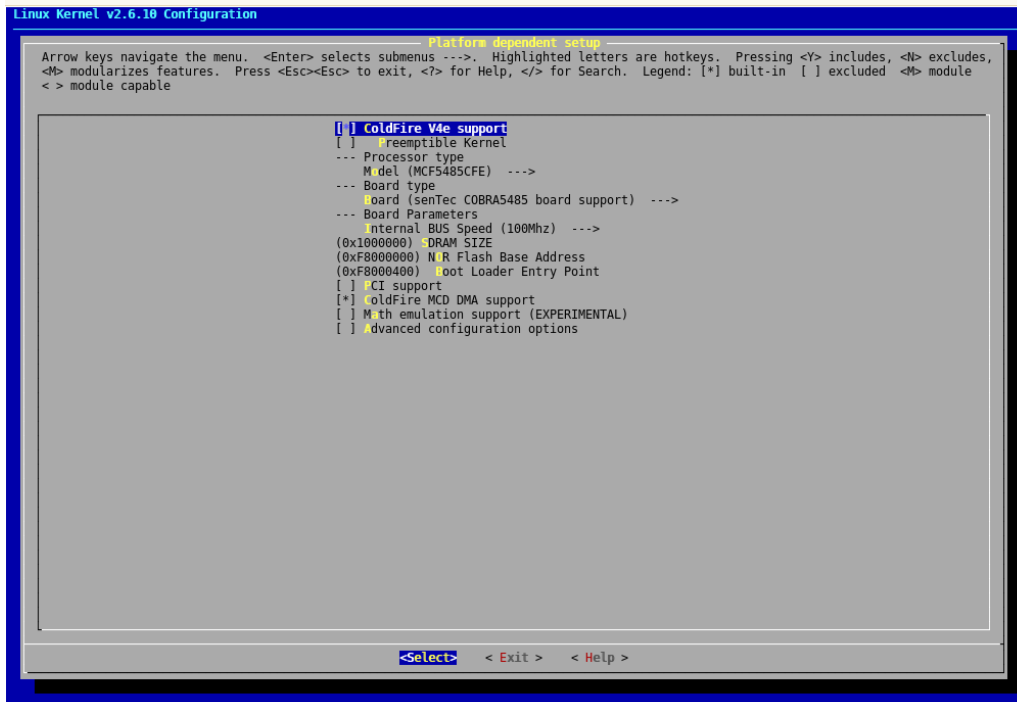


Figure 7.3.2: Specification of Hardware Details

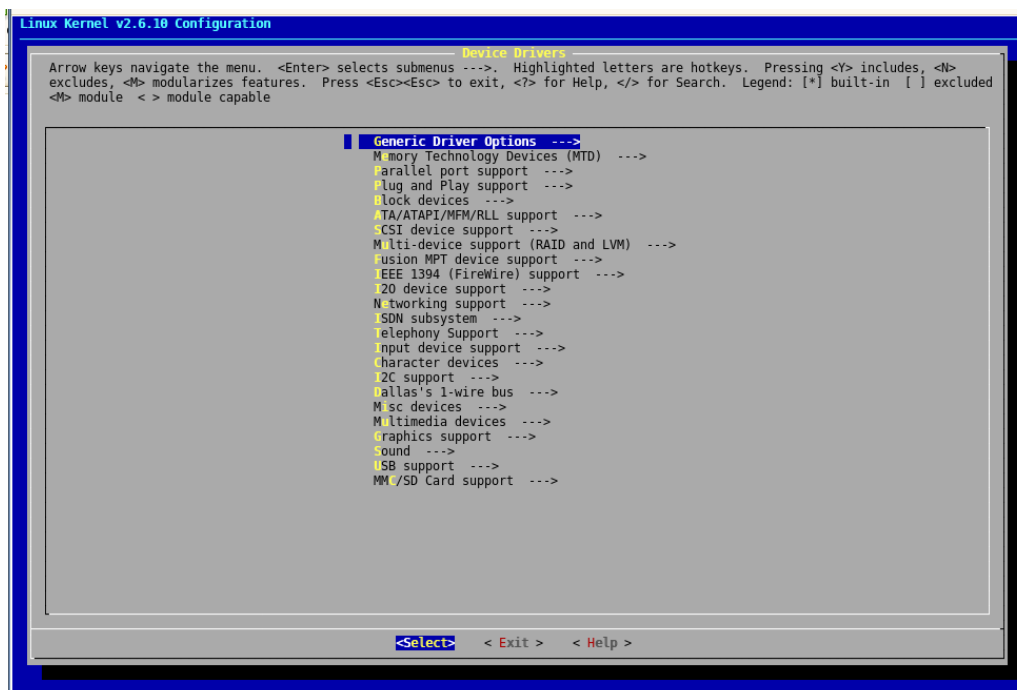


Figure 7.3.3: Configuration through a Selection of Moduls or Drivers

- a image which could be loaded in the FlashEPROM.
- need a “Flash”-load program like CF-Flasher.
- embedded Linux starts after the next reset or power-on-reset with the mentioned boot sequence.

### Working with embedded Linux in the target

- establish a serial communication link between the target and the PC.
  - works as standard input/output (stdio) for messages from embedded Linux.
  - on the PC-side, a terminal program like “minicom” for Linux or ”Hyperterminal” for Windows is useful.

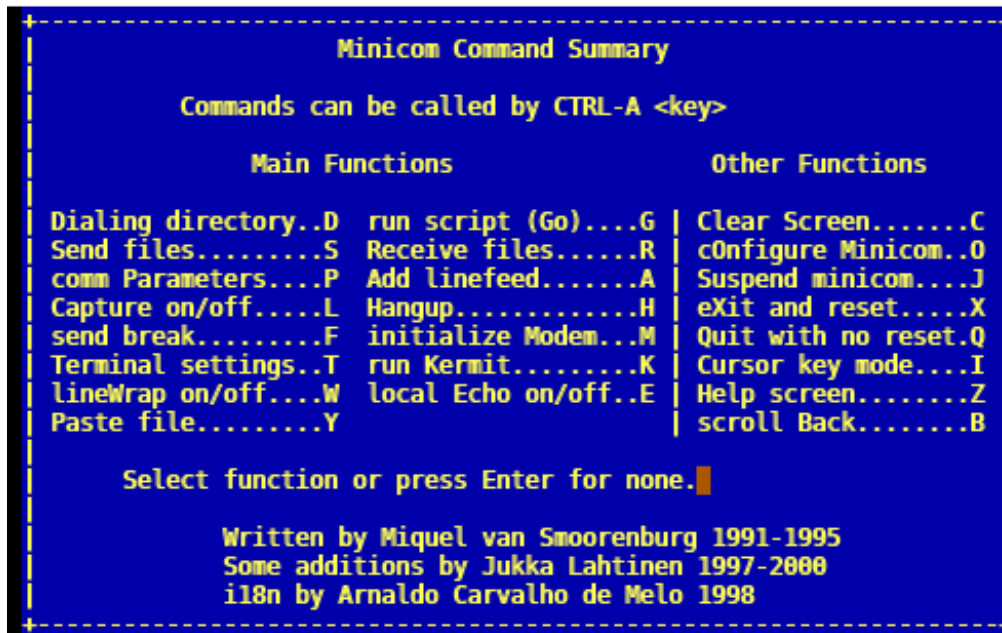


Figure 7.3.4: Serial Communication Tool Minicom

- if an ethernet link is established
  - generate a remote connection (via ssh)
    - execution of commandline instructions on the target
  - mount PC partition by NFS
    - target-filesystem can be mounted in the development-filesystem
    - execute programs on the development-side and on the target-side

### Restrictions when using non VM-systems

- ARM-Processors could operate with a VM (virtual memory) and a non VM System.
- in the case of working without a VM-System:
  - a user program has full access to the whole memory space.
  - as a result, no segmentation fault is possible.
  - no dynamic heap → instead uCLinux uses a global memory pool (kernel free memory pool).

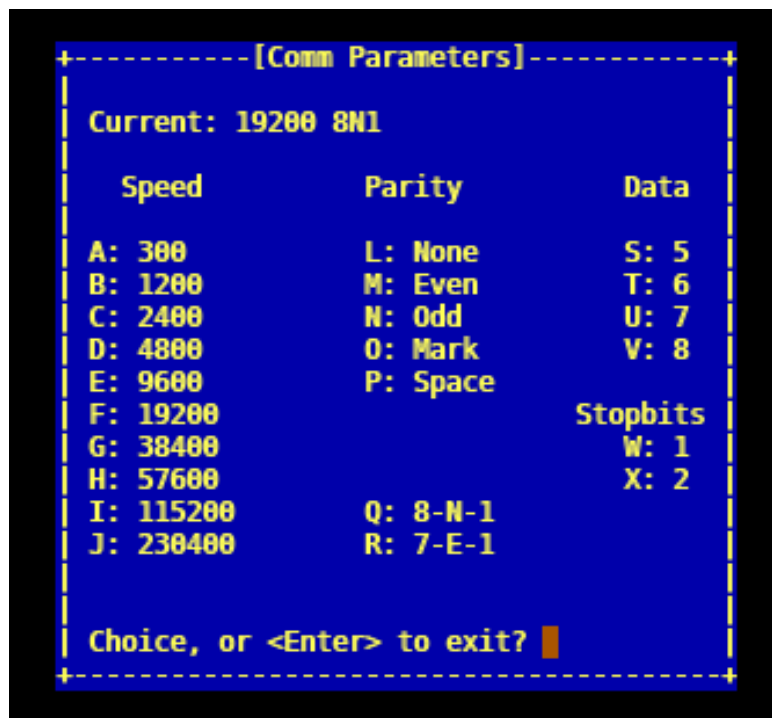


Figure 7.3.5: Configuration of Minicom

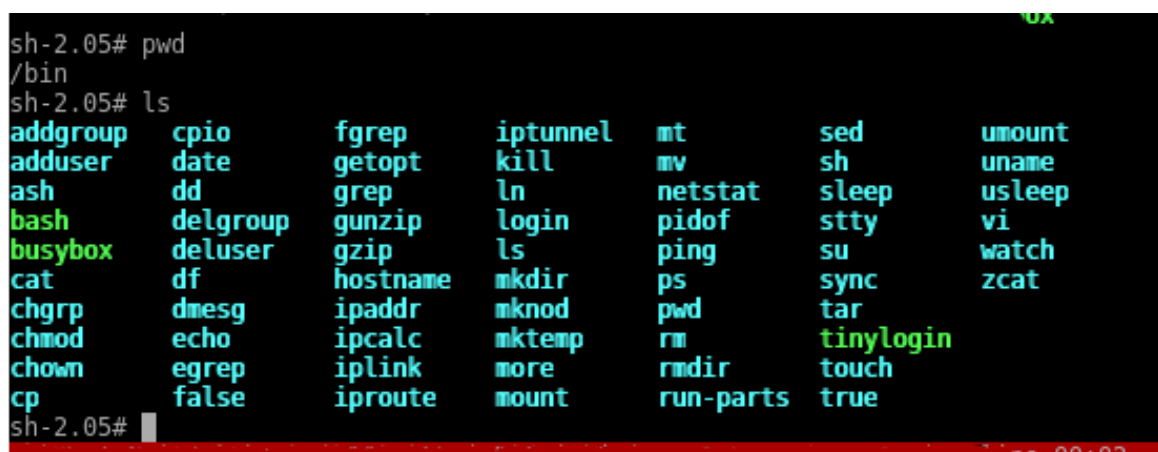


Figure 7.3.6: Standard Commandline Interface on the Target-side

- no obvious implementation of malloc due to lack of heap.
- no fork system call → an implementation of “copy on write” strategy isn’t possible (an alternative is the old BSD vfork and all its limits).

## 7.4 Cross Compilation

- cross compilation is widely used for programming in embedded systems.
- the programs are written, linked and simulated on a PC (for example) for an embedded system.
- then these executable programs have to be downloaded and started on the target under the control of a monitor program (mostly with a serial connection between the PC and the target).
- after the test in the target the executable program could be stored in the EPROM and executed without a monitoring program

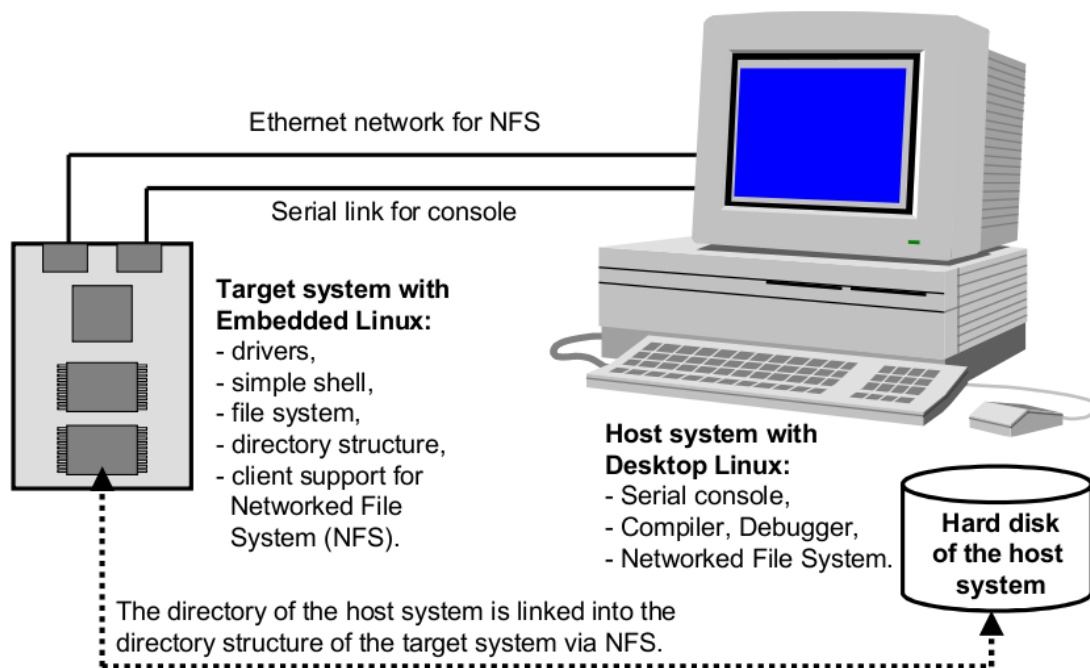


Figure 7.4.1: Cross Compilation

### Cross compilation with embedded Linux

- in embedded Linux systems the cross compilation is done for the same reasons as for a standard system.
- the advantage in this case is that the same debugging tools that are available for the test on the PC are also available on tests on the target system (remote debugging).
- to compile a program for the ARM-Processor a specific toolchain is necessary

- the same toolchain is also used to build the embedded Linux System for the target.
- to generate all parts of the toolchain the standard C-compiler and C-libraries are also involved:

| Tool              | Description                                                                              |
|-------------------|------------------------------------------------------------------------------------------|
| arm-linux-gcc     | C-Compiler for ARM-Processor                                                             |
| arm-linux-as      | Assembler for ARM-Processor                                                              |
| arm-linux-ld      | Linker for ARM-Processor                                                                 |
| arm-linux-ar      | Archiver, tool to build, extract and show the contents of libraries                      |
| arm-linux-nm      | extracts the symbols of an executable file                                               |
| arm-linux-objdump | extract the contents of an executable file (e.g. disassemble files)                      |
| arm-linux-objcopy | tool to generate loadable files from the output of the linker (e.g. “x.bin” or “x.ihex”) |

## Executable files

- standard output of the linker (“x.elf”) isn’t downloadable.
  - this file needs a VM-System for loading and execution.
- instead of “x.elf” the tool arm-linux-objcopy generates a “x.bin” and/or “x.ihex”-file.
  - both files are directly downloadable and don’t need a VM-System for execution.
  - this means both file are linked on an absolute address.
  - the “x.bin”-format is a binary format (could not be read with an editor).
  - the “x.ihex”.format is a text format which is readable with any standard editor.

```

5/DBG_Led/Led.elf
ELF Header:
 Magic: 7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00
 Class: ELF32
 Data: 2's complement, big endian
 Version: 1 (current)
 OS/ABI: UNIX - System V
 ABI Version: 0
 Type: EXEC (Executable file)
 Machine: MC68000
 Version: 0x1
 Entry point address: 0x20000
 Start of program headers: 52 (bytes into file)
 Start of section headers: 9112 (bytes into file)
 Flags: 0x0
 Size of this header: 52 (bytes)
 Size of program headers: 32 (bytes)
 Number of program headers: 3
 Size of section headers: 40 (bytes)
 Number of section headers: 15
 Section header string table index: 12
bm@linux-04z2:/usr/local/m68k-elf/LinuxBSP_20061018/toolchain/m68k-linux/gcc-3.4.0-glibc-2.3.2/bin>

```

Figure 7.4.2: Information in a “x.elf”-file

## 7.5 User Programming in uCLinux

- user programming implies that a program runs in the user mode of the ARM-Processor.
- if the VM-System is activated → the user doesn't have an access to an absolute memory address.
  - especially a direct access to the registers of interfaces isn't allowed.
- the restrictions imply also that an access to R-, S-, D-registers, CPSR and etc. are not allowed.
- these access could only be done in the supervisor mode (kernel program).
- Development of program should be done in three steps:
  - first try to generate a program with a cross-compilation for the target, load and then test it.
  - then merge the program together with an image of the operating system and load it.
  - lastly, if necessary implement a "autoexec" function in the init process of the embedded Linux.

### Cross-compilation with embedded Linux

- the cross-compilation with embedded Linux has the same procedure as the standard cross-compilation for ARM-Processors.
- in preparation for the next two steps it is helpful to use a separate directory which holds all the necessary files (C-files, Header-files, makefile).
- put this directory in the ARM-Linux-file-system '/armLinux/user/' .
- after the compilation mount the directory with NFS in the uCLinux for the target and execute and test the program.

### Download together with an image of embedded Linux

- the second step is to integrate the program in an image of uCLinux.
- additional to the cross-compilation, the program also has to be integrated in the build procedure of uCLinux.
  - the "central"-makefile for building all the user applications (/armLinux/user/makefile) should be expanded with an instruction like:  
DIRSc\$(CONFIG\_USER\_CAN) += can4linux (present line)  
DIRSc\$(CONFIG\_USER\_PROGRAM) += program (new line)
  - to make the program visible in configuration tool (make menuconfig) of arm-Linux the file 'config.in' (/armLinux/user/config.in) should be expanded by:  
bool 'can4linux' CONFIG\_USER\_CAN (present line)  
bool 'program' CONFIG\_USER\_PROGRAM (new line)

- to get some more information about the program and how somebody can do the configuration the file `Configure.help` (`/armLinux/user/Documentation/Configure.help`) should expand by `CONFIG_USER_CAN`

This package contains some examples using `can4linux` device driver.

#### `CONFIG_USER_PROGRAM`

This program adds my first functions to `uCLinux`. I'm not sure whether somebody can use these functions.

- if in the configuration step, the program is selected, the resulting image contains the program
- to test the image (`armLinux`) and the program once more, it can be downloaded with the monitor
- the downloaded `armLinux` (including the program) could be started after a re-boot or a reset

### **Autostart of a program with embedded Linux**

- if the program (which was downloaded together with the `armLinux`-image) has to be used, the user has to start it.
- embedded Linux offers an automatic start of programs (function of the program init which execute the script file `/etc/inittab`).
- the script-file `inittab` ask an another skript-file `rcS` (placed in `/etc/rc.d/rcS`) and this file switch over to `rc.local` (`/etc/rc.d/rc.local`).
- an auto start could be done by addition in the file `rc.local` (but this means that the user has to change this file and this is obviously not an autostart, especially not for the first time).
- to do a 'real' auto start of the file `/armLinux/defconfigs/vendors/senTec/etc/rc.d/rc.local` has to be changed and then a new image has to be created.
- when this image is loaded in the `FlushEPROM` (with the program `CFFlusher`) then after a reset the program is automatical started without an interaction with the user.